

On-Device AI

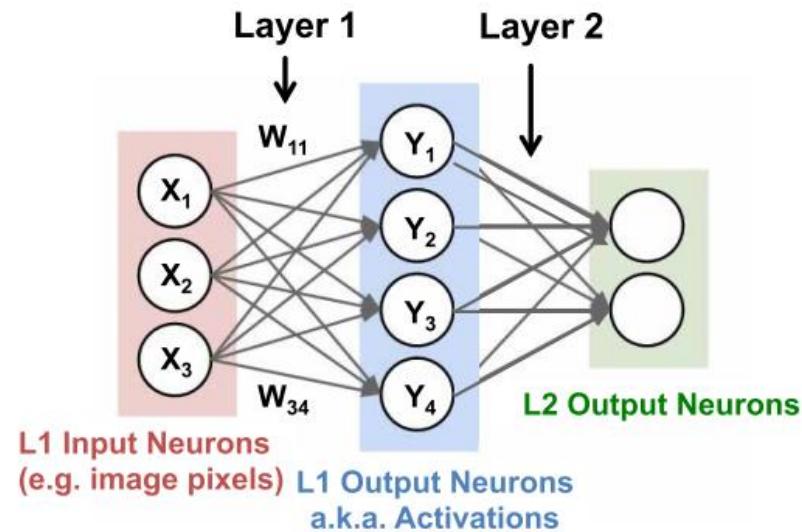
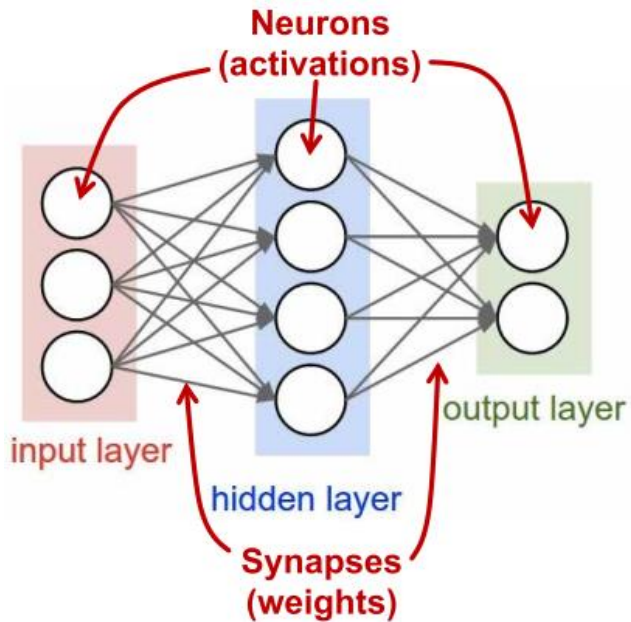
Part 1

성균관대 신동균 교수

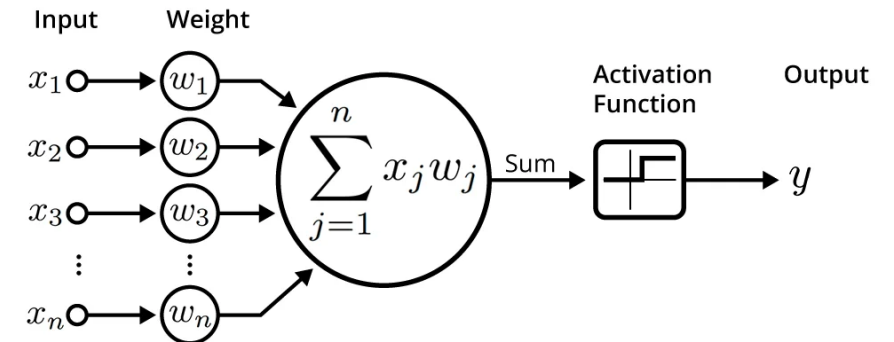
강의 구성

시간	주제	내용	실습
1-1	AI Workload & Performance	CNN, LLM (prefill/decode, attention, KV cache) FLOPs/Memory, GEMM/GEMV, Roofline	
1-2	On-Device AI	Small Models, llama.cpp, ExecuTorch, NPU	
1-3	Quantization	Data Representation, Linear/Non-Linear Quantization, Quantization Granularity, PTQ/QAT	CNN Quant Llama.cpp Quant 기법
1-4	Advanced Quantization	SmoothQuant, GPTQ, AWQ, Rotation (QuaRot, SpinQuant), QLora	LMM Quant
2-1	Pruning & Sparsity	Unstructured, Structured, N:M, LLM pruning (Wanda, SparseGPT)	LLM pruning
2-2	Knowledge Distillation	Response-based KD, Feature-based KD, LLM KD	Knowledge Distillation
2-3	Efficient LLM Inference	KV Quantization (KIVI, TurboQuant) KV Pruning (H2O, SnapKV, Quest) Speculative Decoding (Eagle, MTP, DFlash)	SnapKV, Eagle, DFlash
2-4	HW-aware Optimization	Tiling/Fusion, FlashAttention	

Deep Neural Network



$$y = \text{ReLU}(\sum w_i x_i + b)$$



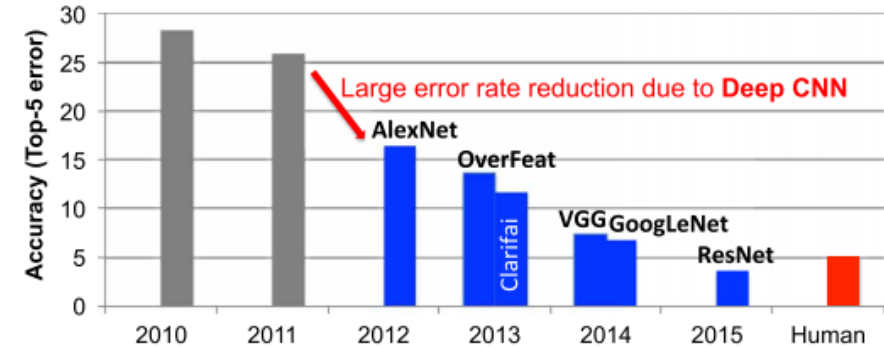
An illustration of an artificial neuron. Source: Becoming Human.

<https://insights.sei.cmu.edu/blog/deep-learning-going-deeper-toward-meaningful-patterns-in-complex-data/>

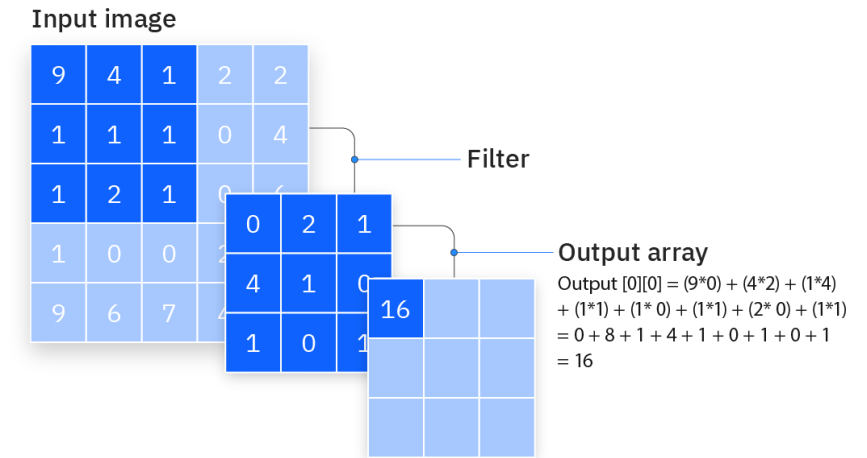
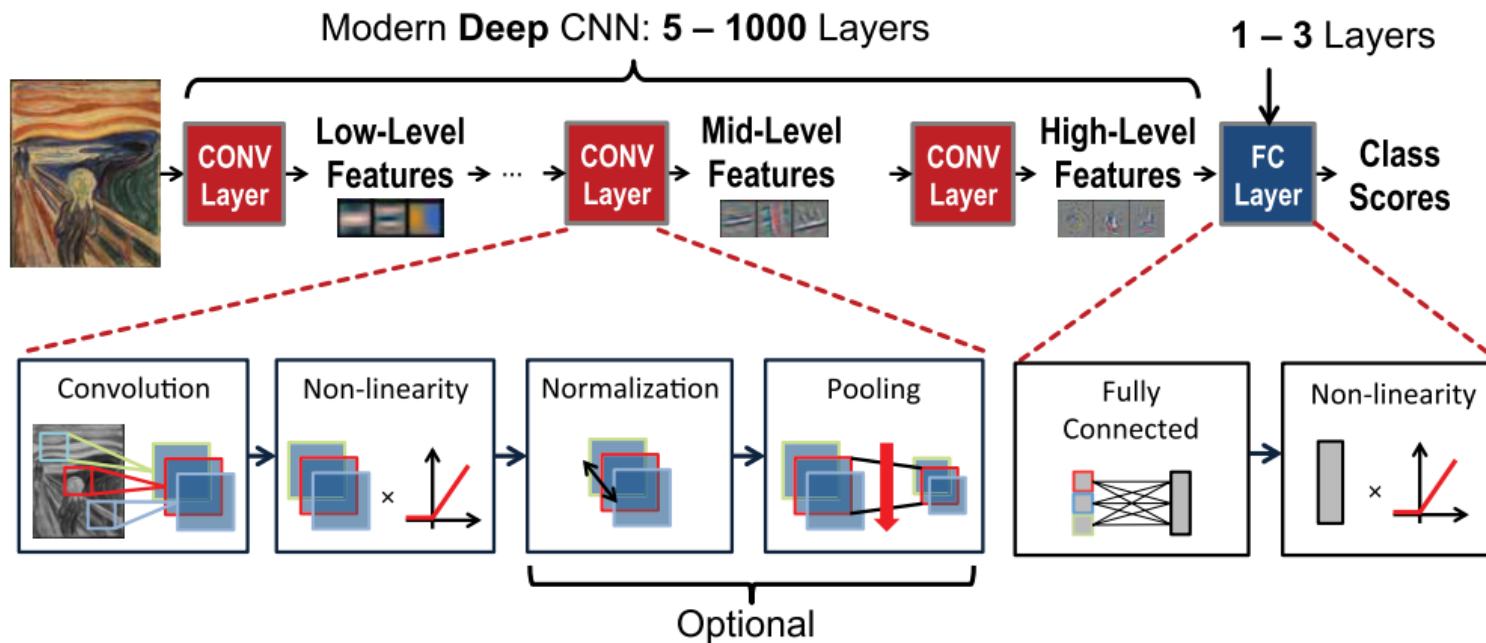
Efficient Processing of Deep Neural Networks: A Tutorial and Survey [Sze et al. 2017]

Convolutional Neural Networks (CNNs)

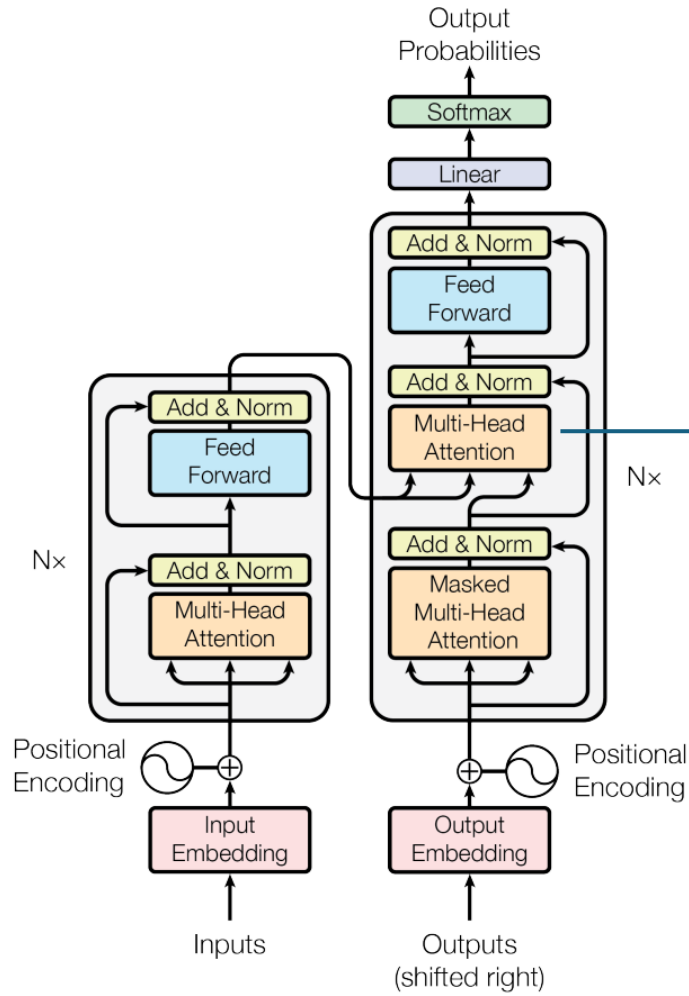
- Processing of 2-D features
- Conv layer, Pooling layer, FC layer
- Each layer generates a successively higher-level abstraction of the input data, called a feature map (fmap)



Results from the ImageNet Challenge

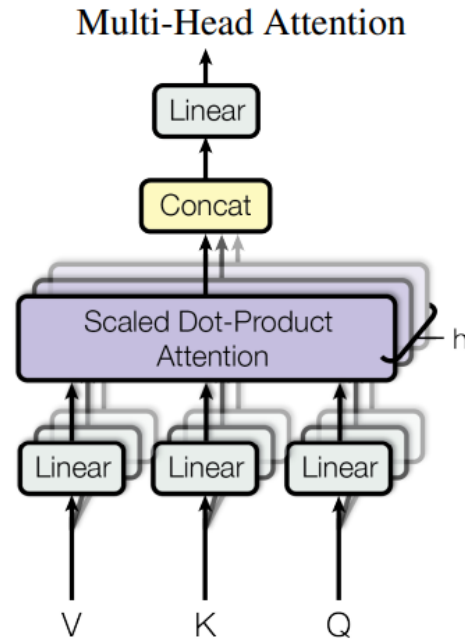


Transformer

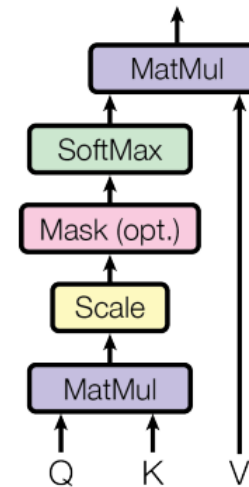


Encoder

Decoder



Scaled Dot-Product Attention



$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V = Z$$

The equation shows the matrix multiplication of Q (purple) and K^T (orange) divided by $\sqrt{d_k}$, followed by a **softmax** operation, then multiplied by V (blue) to produce the final output Z (pink).

<https://benjaminwarner.dev/2023/07/01/attention-mechanism>

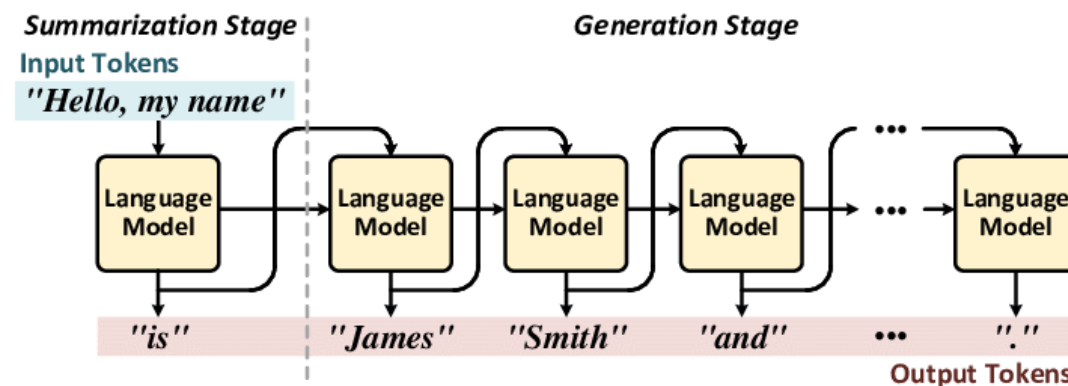
LLM Operations

- **Prefill phase**

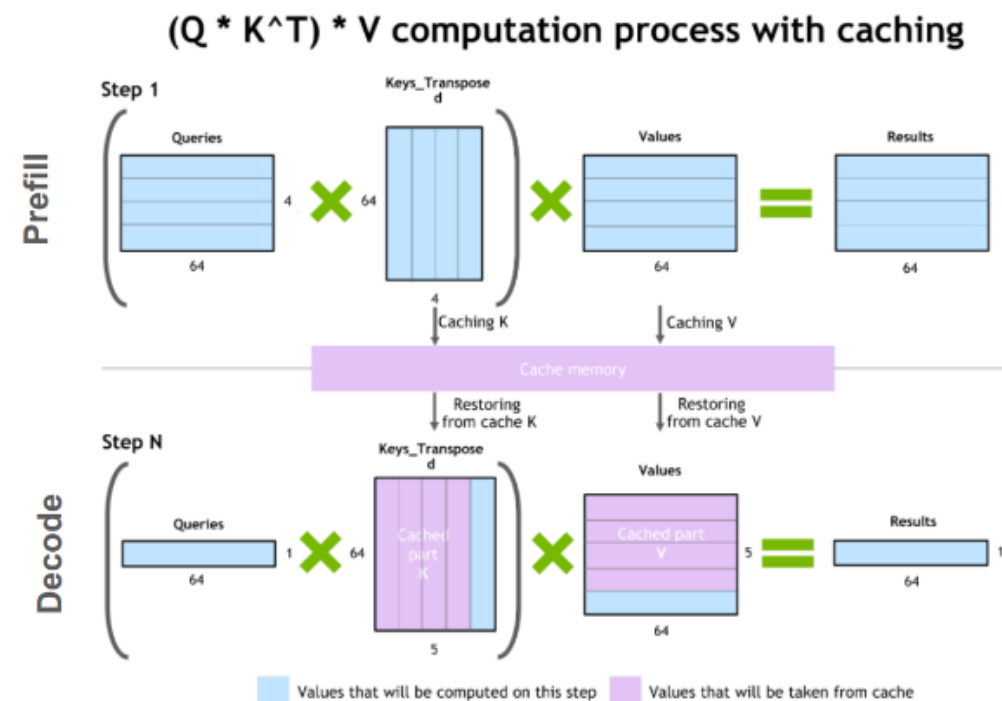
- Process the input tokens to compute the intermediate states (keys and values)
- **Masked GEMM**

- **Decode phase**

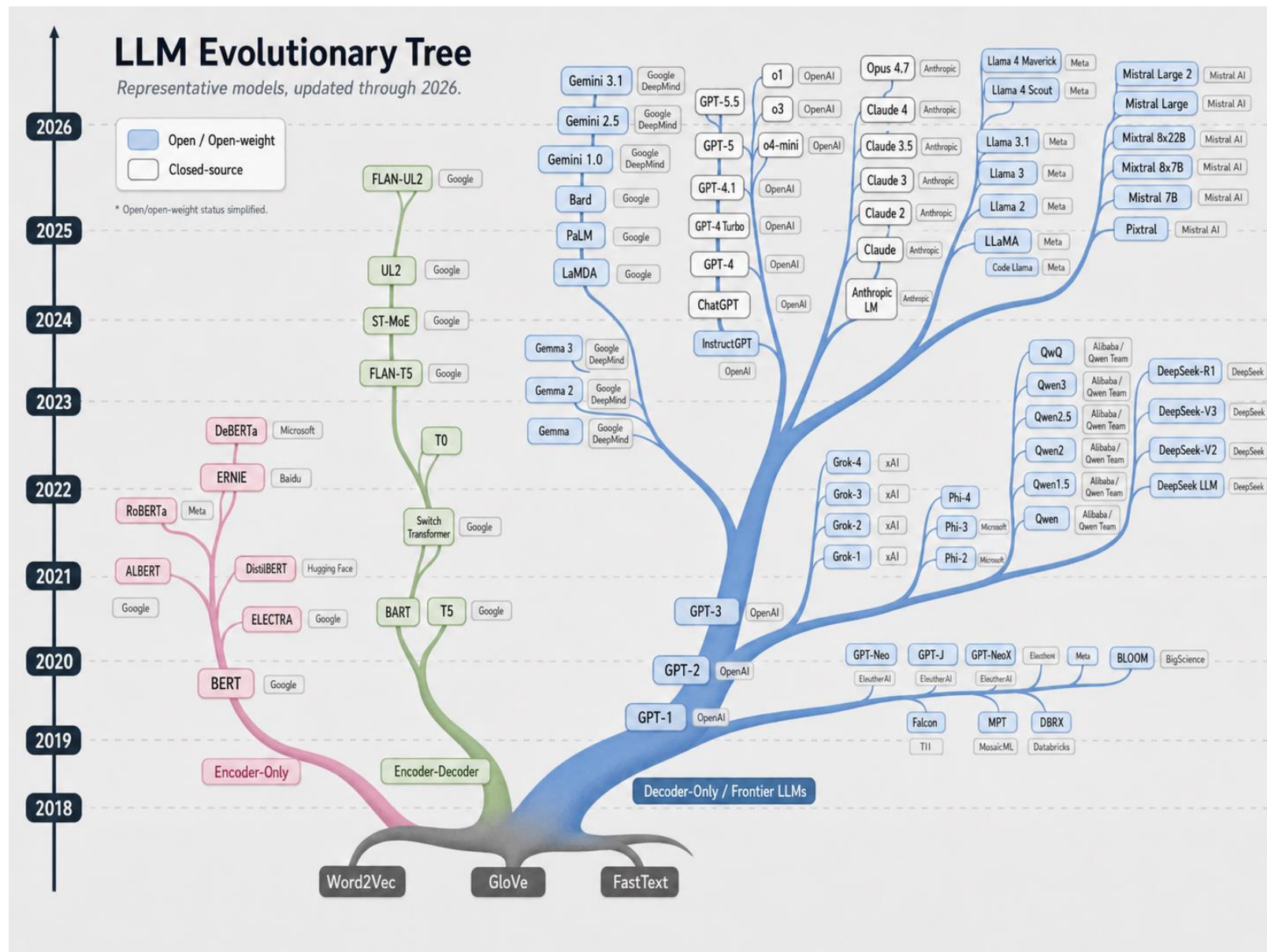
- Generate output tokens **autoregressively** one at a time.
- Each sequential output token needs to know all the previous iterations' output states (**KV Cache**).
- **GEMV** (underutilize GPU)
- Memory-bound: data (weights, keys, values, activations) are transferred to the GPU from memory
- Can improve GPU utilization by **request batching**



<https://arxiv.org/pdf/2209.10797>



Large Language Models



Vision Language Model

비전-언어 모델(VLM)의 진화 과정: 픽셀 매칭에서 멀티모달 에이전트로



Cloud AI vs. On-Device AI



대용량 AI 모델

고성능 컴퓨팅 환경

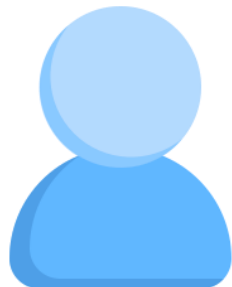
클라우드 인프라



Cloud GPU Server

데이터 입력

AI 연산 결과



User

데이터 입력
AI 연산 결과



Device



User

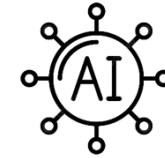
데이터 입력
AI 연산 결과



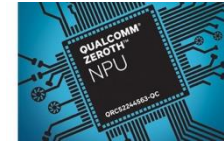
Device



1. 빠른 응답
2. 보안 강화
3. 네트워크 연결 불필요
4. 비용 절감
5. 개인화된 서비스



경량화된 AI 모델



지능형 반도체
(NPU, PIM)

Memory Capacity



S26
16/12GB

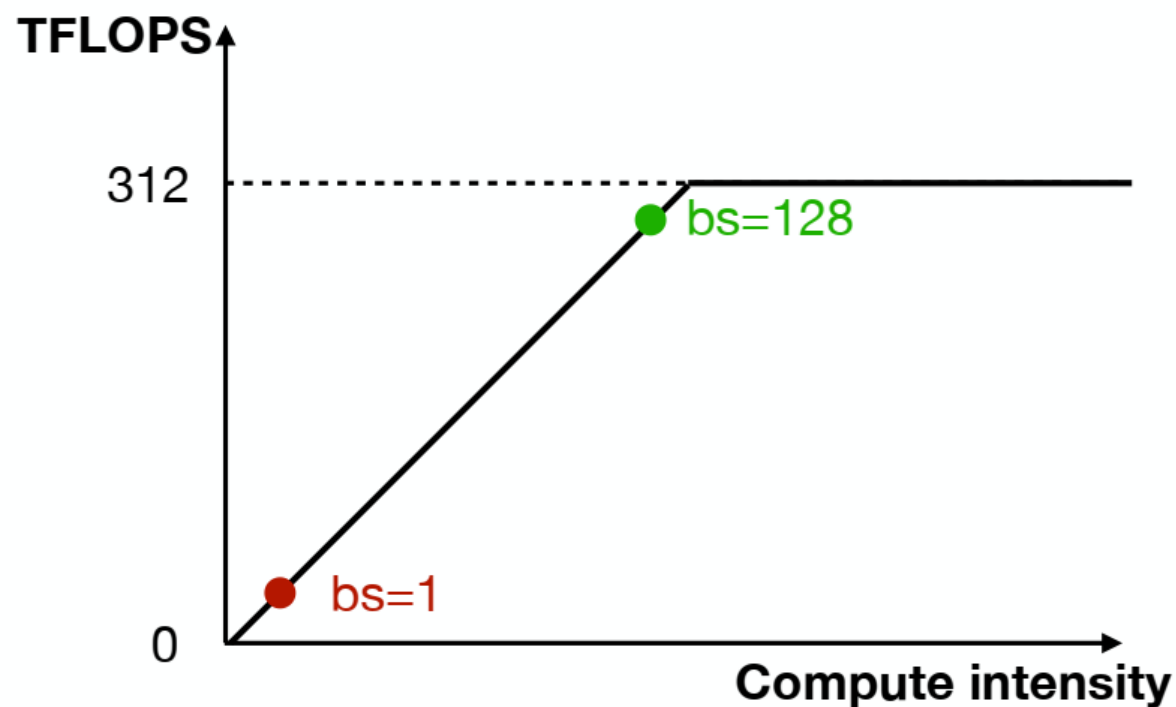


S24
12/8GB

Model	Parameters	FP16	INT4
Gemma4	2.3B, 4.5B, 12B	4, 8, 24GB	1, 2, 6GB
Llama3.2	1B, 3B, 11B	2, 6, 22GB	0.5, 1.5, 5.5GB

Performance

$$P = \min(P_{peak}, BW \times AI)$$



Arithmetic Intensity= FLOPs / Memory Traffic

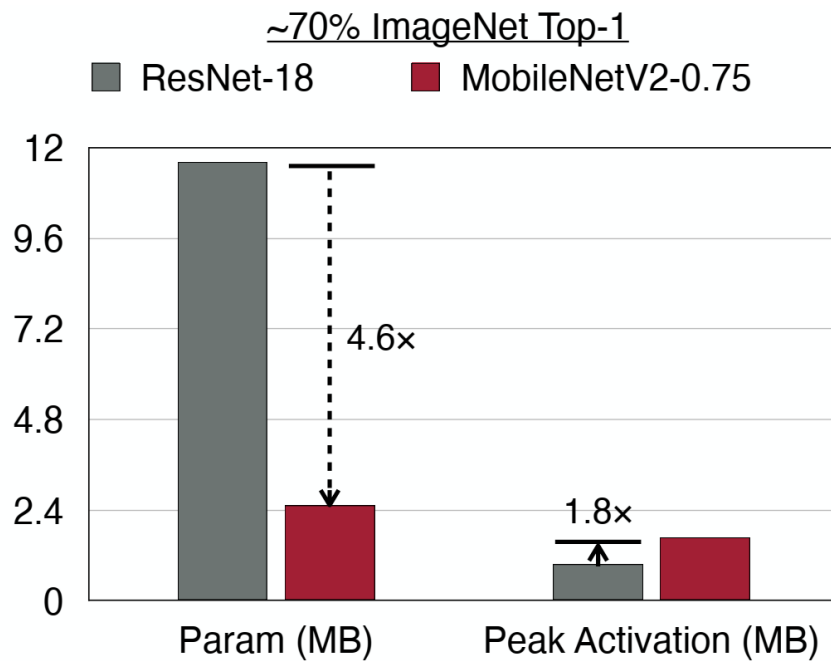
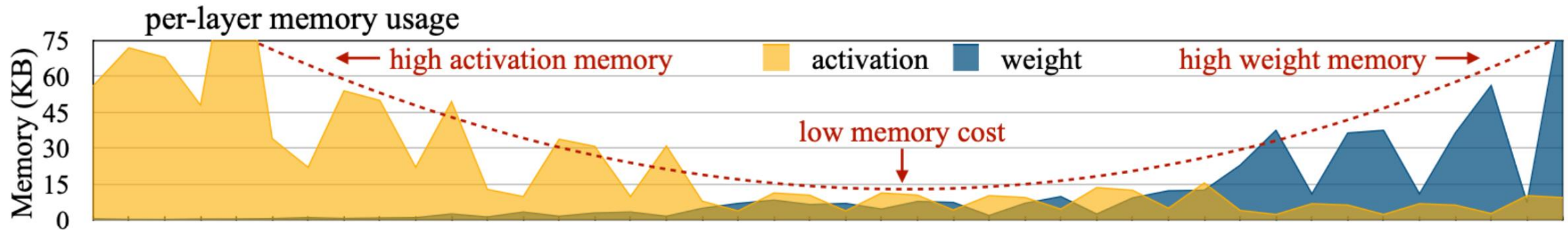
- CNN: Conv, GEMM
 - ✓ high data reuse
 - ✓ often compute-bound
- LLM Prefill: GEMM
 - ✓ relatively high arithmetic intensity
 - ✓ compute utilization 중요
- LLM Decode: GEMV
 - ✓ low weight reuse at batch=1
 - ✓ memory-bandwidth bound

Parameters, Activations, FLOPs

AlexNet	C x H x W = #Activations		#Parameters (bias is ignored) $c_{out} \times c_{in} \times k^2 / g$	MACs (one MAC = two FLOPs) $c_{out} \times c_{in} \times k^2 \times h \times w / g$
Image (3x244x224)	3x244x224	= 150,528		
11x11 Conv, channel 96, stride 4, pad 2	150,528+290,400=440,928 (peak) 96x55x55	= 290,400	96x3x11x11 = 24,848	96x3x11x11x55x55 = 105,415,200
3x3 MaxPool, stride 2	96x27x27	= 69,984		
5x5 Conv, channel 256, pad 2, groups 2	256x27x27	= 186,624	256x96x5x5/2 = 307,200	256x96x5x5x27x27/2 = 223,948,800
3x3 MaxPool, stride 2	256x13x13	= 43,264		
3x3 Conv, channel 384, pad 1	384x13x13	= 64,896	384x256x3x3 = 884,736	384x256x3x3x13x13 = 149,520,384
3x3 Conv, channel 384, pad 1, groups 2	384x13x13	= 64,896	384x384x3x3/2 = 663,552	384x384x3x3x13x13/2 = 112,140,288
3x3 Conv, channel 256, pad 1, groups 2	256x13x13	= 43,264	256x384x3x3/2 = 442,368	256x384x3x3x13x13/2 = 74,760,192
3x3 MaxPool, stride 2	256x6x6	= 9,216		
Linear, channel 4096	4096	= 4,096	$c_{out} \times c_{in} (k = 1)$ 4096x(256x6x6) = 37,748,736	$c_{out} \times c_{in} (k, h, w = 1)$ 4096x(256x6x6) = 37,748,736
Linear, channel 4096	4096	= 4,096	4096x4096 = 16,777,216	4096x4096 = 16,777,216
Linear, channel 1000	1000	= 1,000	1000x4096 = 4,096,000	1000x4096 = 4,096,000
Total 932,264 = 3.7MB (32bits)			Total 61M = 224MB (32bits)	Total 724M = 1.4G FLOPs

From <https://efficientml.ai>

Memory



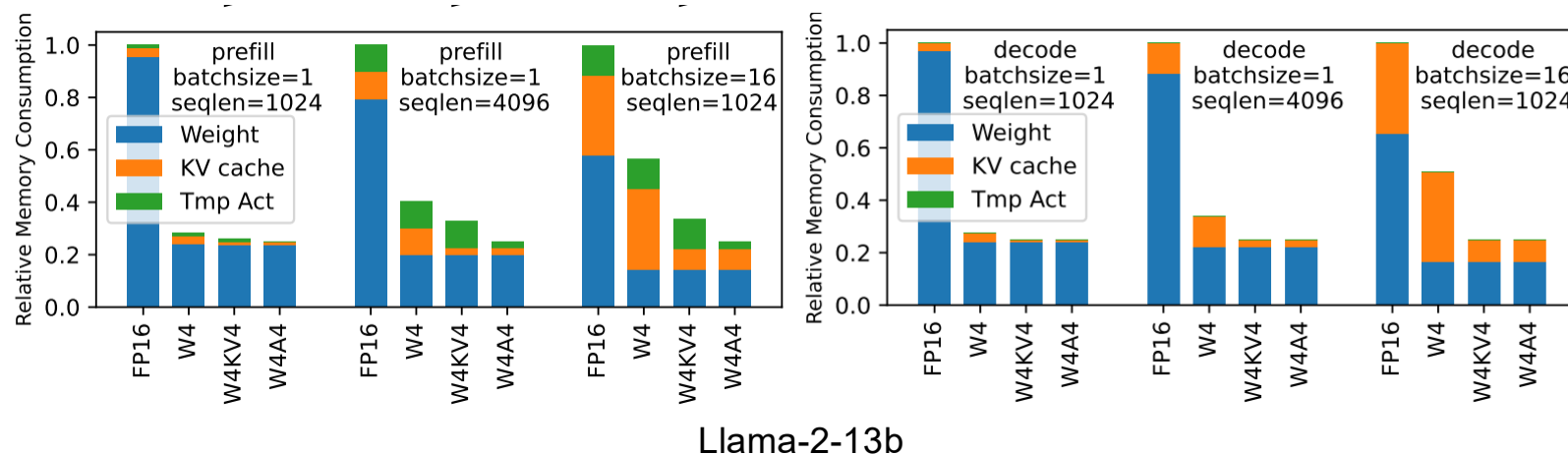
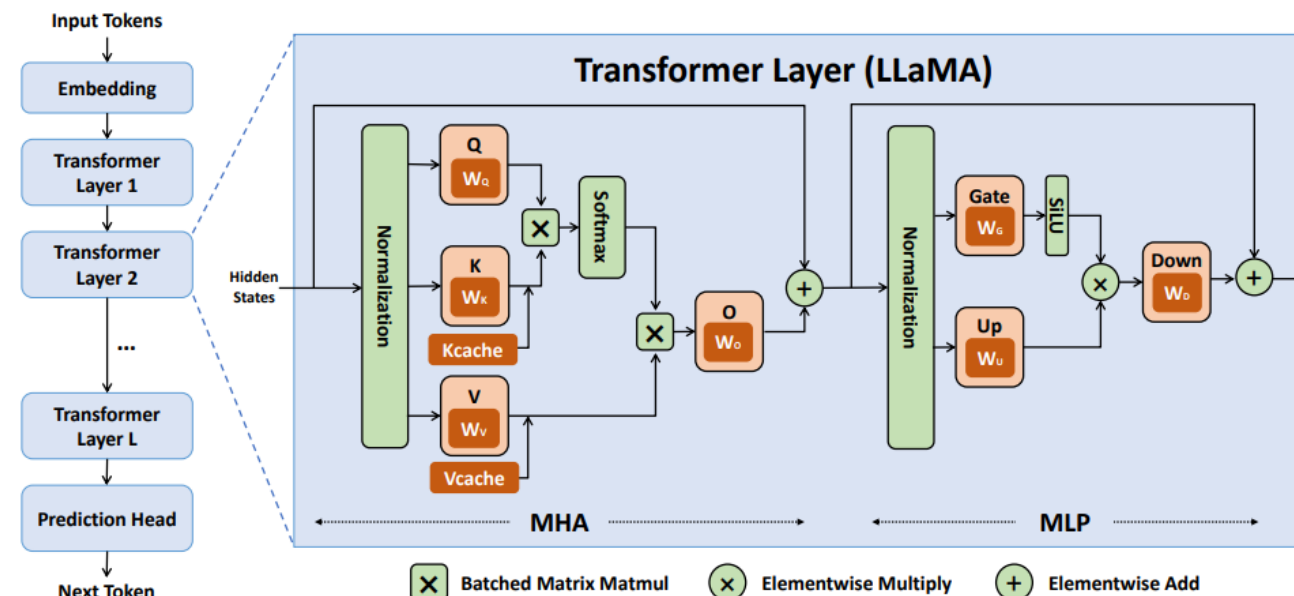
MobileNetV2 reduces model size
but not peak memory.

* All parameters and activations are Integer numbers (8 bits).

Memory - LLM Inference

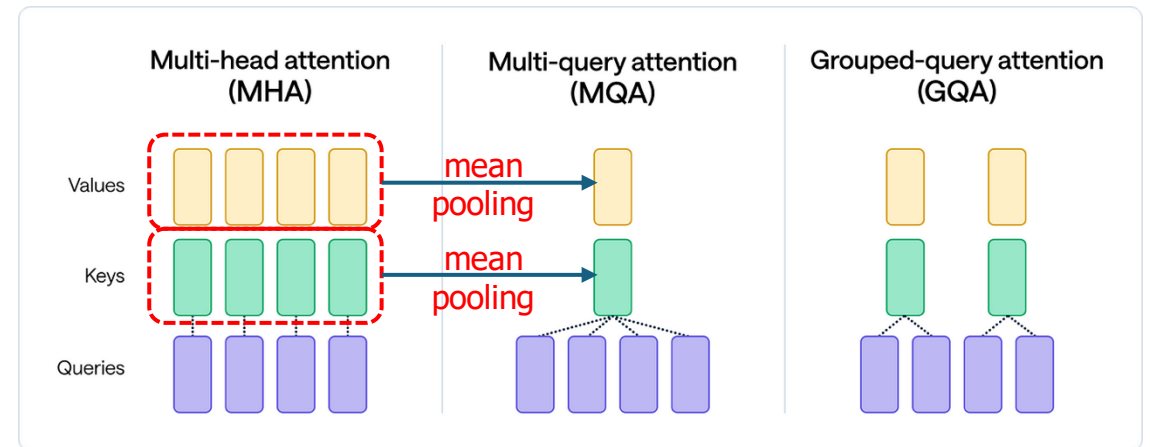
Table 1. Analysis for layers in Llama-2-7b using the Roofline model of Nvidia A6000 GPU. In this example, the sequence length is 2048 and the batch size is 1.

Layer Name	OPs	Memory Access	Arithmetic Intensity	Max Performance	Bound
Prefill					
q_proj	69G	67M	1024	155T	compute
k_proj	69G	67M	1024	155T	compute
v_proj	69G	67M	1024	155T	compute
o_proj	69G	67M	1024	155T	compute
gate_proj	185G	152M	1215	155T	compute
up_proj	185G	152M	1215	155T	compute
down_proj	185G	152M	1215	155T	compute
qk_matmul	34G	302M	114	87T	memory
sv_matmul	34G	302M	114	87T	memory
softmax	671M	537M	1.25	960G	memory
norm	59M	34M	1.75	1T	memory
add	8M	34M	0.25	192G	memory
Decode					
q_proj	34M	34M	1	768G	memory
k_proj	34M	34M	1	768G	memory
v_proj	34M	34M	1	768G	memory
o_proj	34M	34M	1	768G	memory
gate_proj	90M	90M	1	768G	memory
up_proj	90M	90M	1	768G	memory
down_proj	90M	90M	1	768G	memory
qk_matmul	17M	17M	0.99	762G	memory
sv_matmul	17M	17M	0.99	762G	memory
softmax	328K	262K	1.25	960G	memory
norm	29K	16K	1.75	1T	memory
add	4K	16K	0.25	192G	memory



Memory - LLM Inference

- Llama2-7B, 16-bit precision (FP16 or BF16)
 - Model weights: $7B \times FP16 \approx 14$ GB.
 - **KV cache**: $2 \times B \times S \times L \times D \times P$
 - B: Batch Size, S: Sequence Length, L: #Layers, D: Hidden Dimension, P: Precision
 - if batch size = 1, $2 \times 1 \times 4096 \times 32 \times 4096 \times 2B \sim 2$ GB (0.5MB/token)
- How to reduce KV cache?
 - Reduce batch size? Low GPU utilization
 - Reduce sequence length (token pruning, sparse attention)? Accuracy drop
 - Reduce layers? Smaller models usually have less layers
 - Reduce attention heads?
 - Multi-query attention (MQA),
 - Grouped-query attention (GQA)
 - Reduce precision? KV Quantization
- How about memory offloading?



On-Device AI Enablers

Efficient AI Models

Small Models, Model Compression

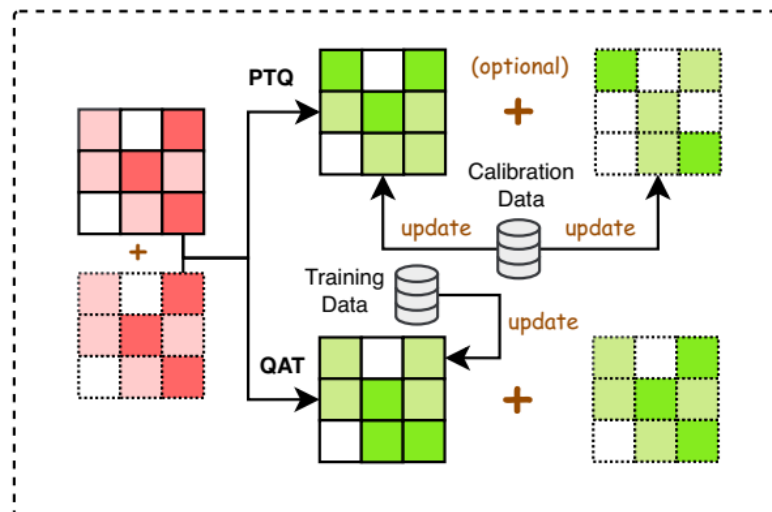
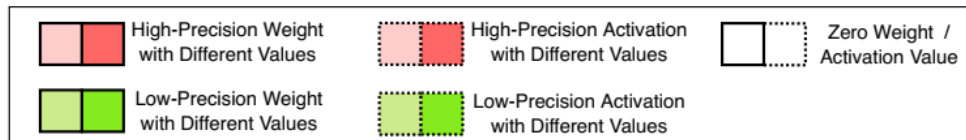
Edge AI Frameworks

Llama.cpp ExecuTorch, MLC-LLM,
Google LiteRT, ONNX Runtime,
Apple MLX, Intel OpenVINO GenAI

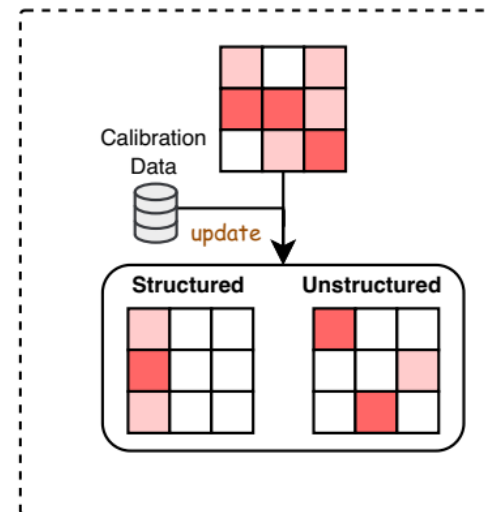
Hardware Accelerators

GPU, NPU, PIM, ...

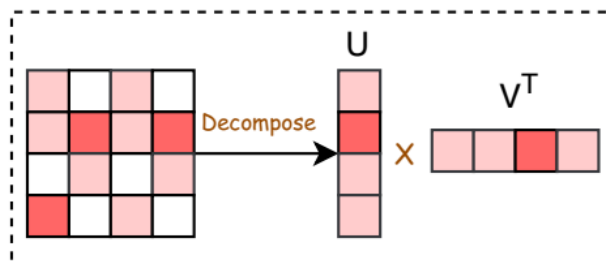
Model Compression Techniques



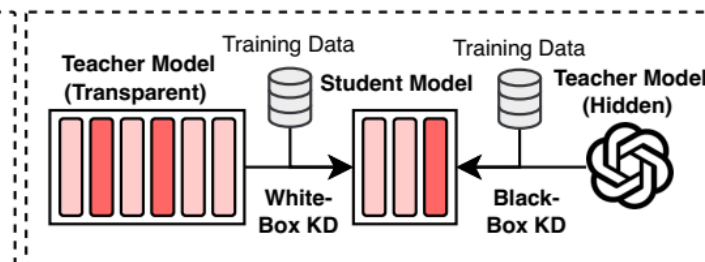
(a) Quantization



(b) Parameter Pruning



(c) Low-Rank Approximation



(d) Knowledge Distillation

Small LLMs

- **Google** Gemini
 - Nano-1: 1.8B, Nano-2: 3.25B
- **Google** Gemma 4
 - E2B(2.3B, +embedding 5.1B)
 - E4B(4.5B, +embedding 8B)
 - 12B
- **Meta**
 - Llama3.2
 - Lightweight: 1B, 3B
 - Multimodal: 11B
 - MobileLLM
 - 125M, 350M for on-device
 - MobileLLM-Pro: 1B
- **MS Phi**
 - Phi-4-mini (3.8B)
- **Mistral**
 - Ministral 3B
- **NVIDIA**
 - Minitron 4B
- **Alibaba**
 - Qwen3: 0.6B, 1.7B, 4B, 8B, 14B

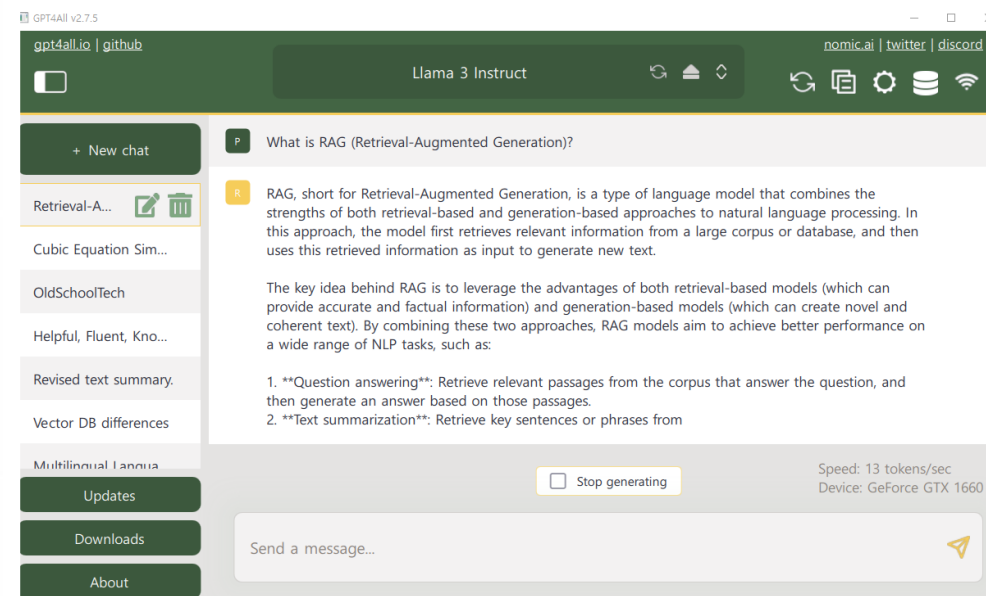
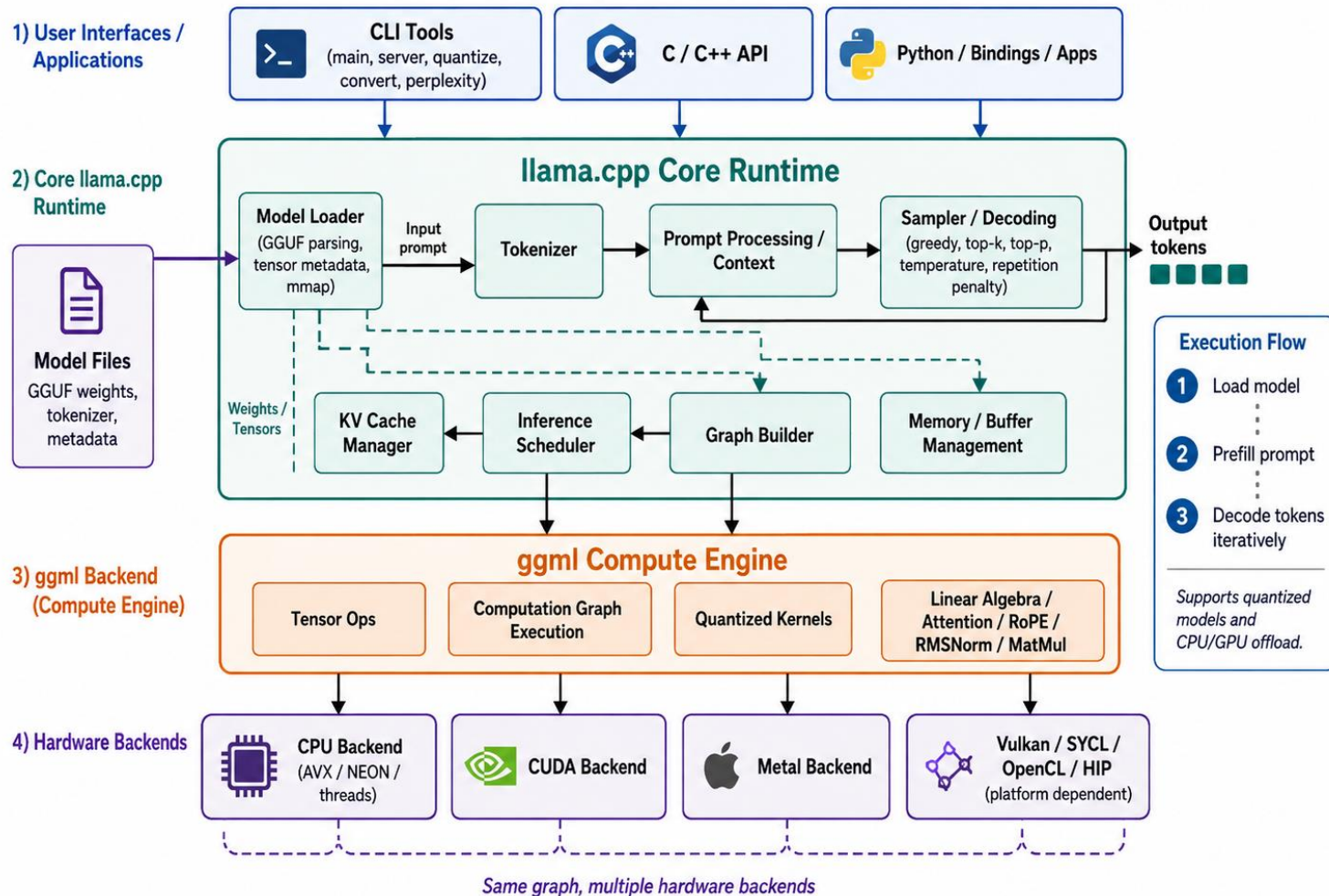
Llama.cpp



- Georgi Gerganov implemented Meta's LLaMa architecture within the versatile and widely-used C/C++ programming languages.
- Broad compatibility (Apple M, Intel AVX512, NVIDIA)
- Performance optimizations inherent to C/C++.
- Integer Quantization (1.5b, 2b, 3b, 4b, 5b, 6b, 8b)
- Support CPU+GPU hybrid inference
- GGUF and GGML: tensor library used by llama.cpp
- Backed by an active open-source community, continuous improvements and collaborative development.
- <https://github.com/ggerganov/llama.cpp>

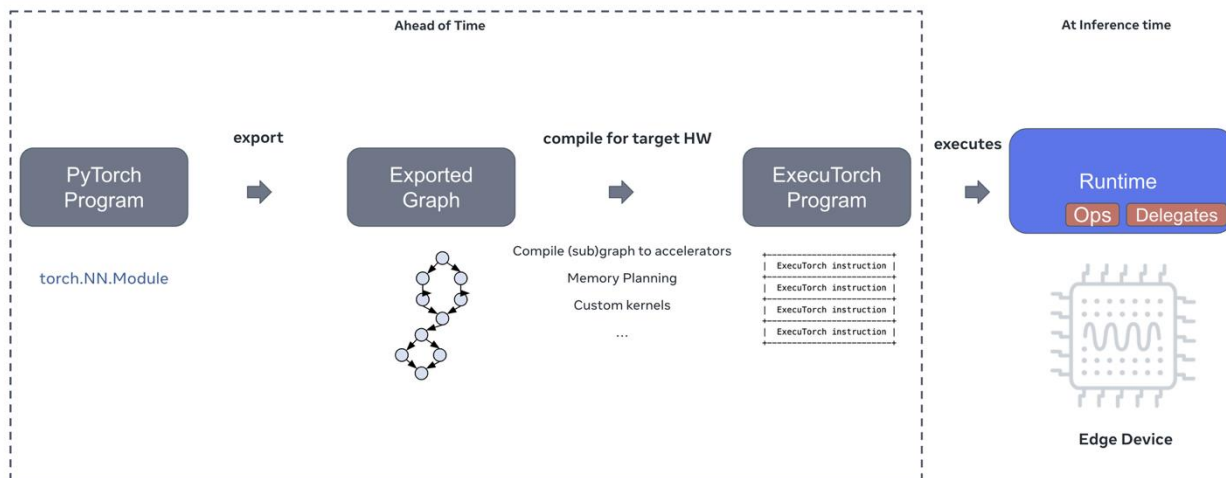
Llama.cpp

Llama.cpp Architecture

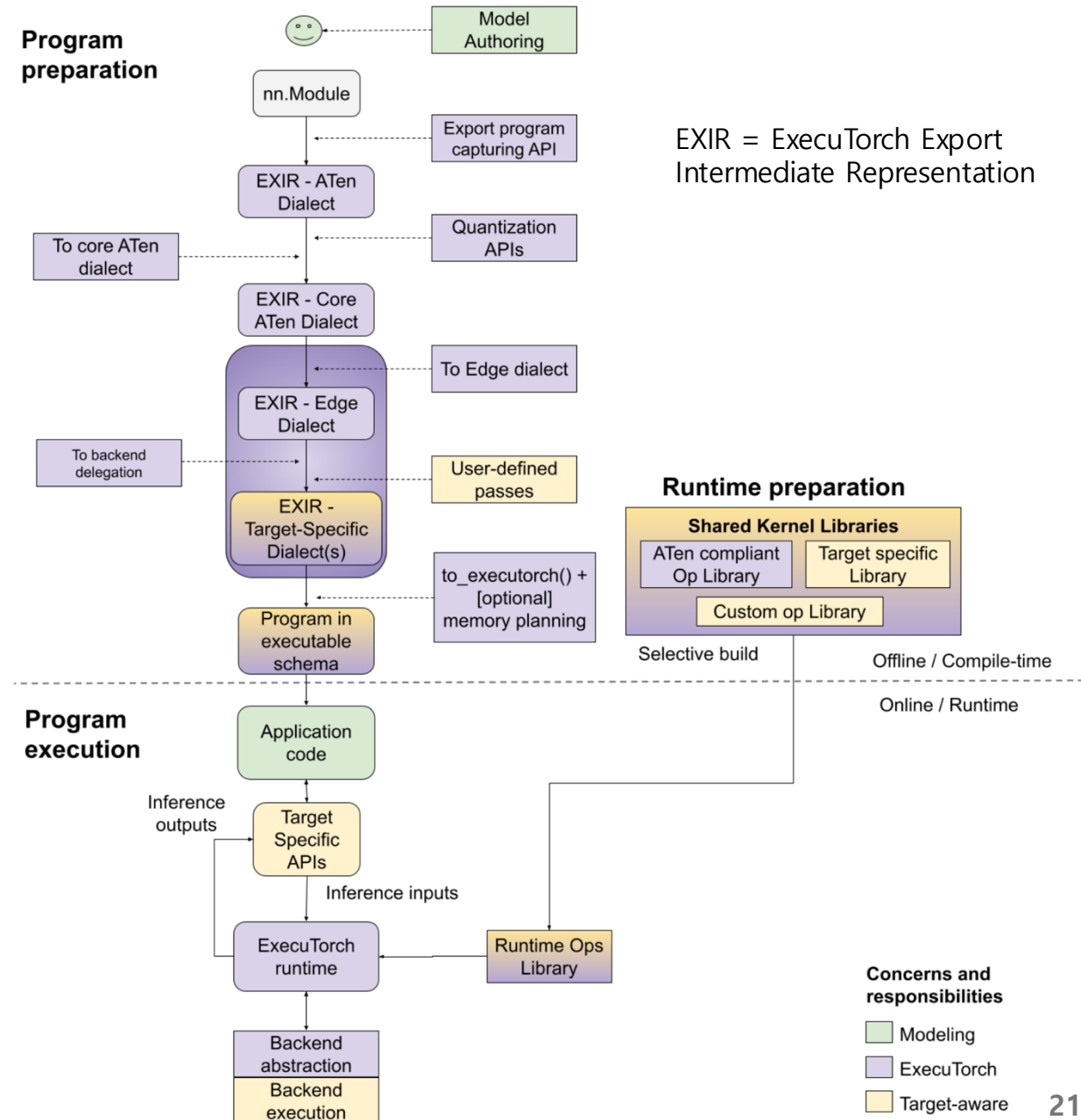


ExecuTorch

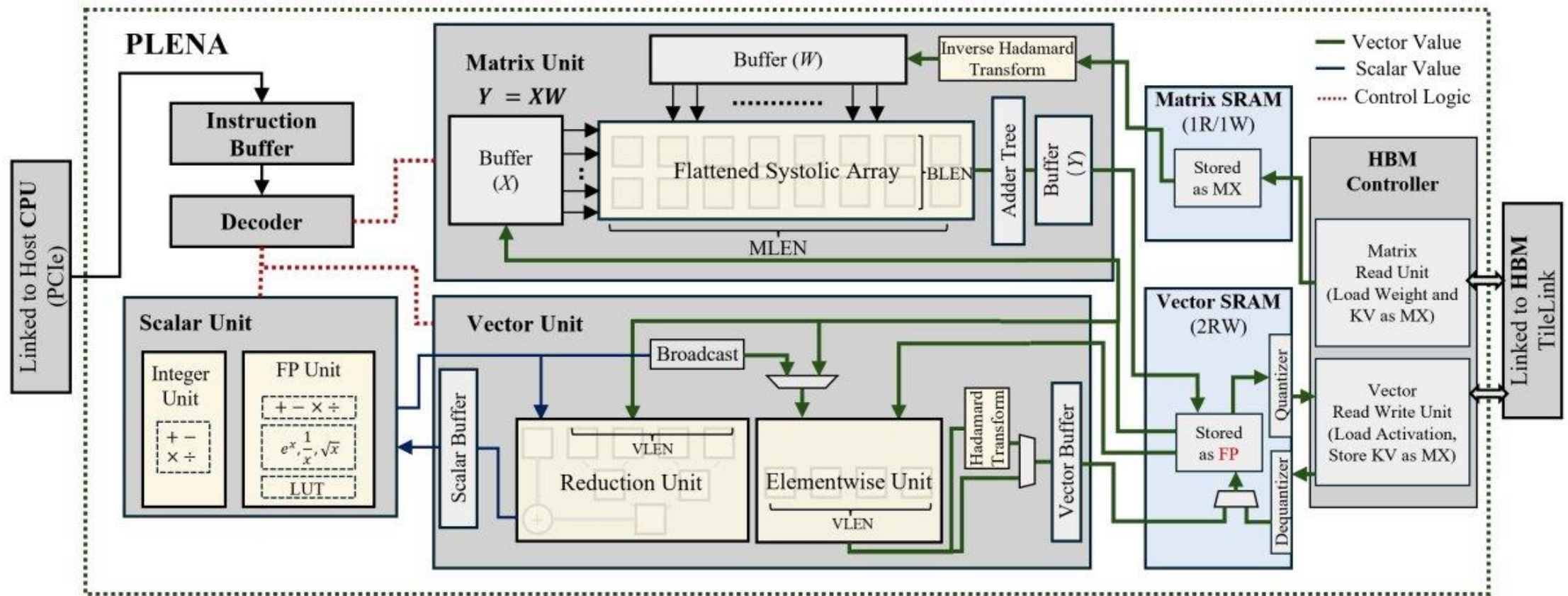
- PyTorch's official, end-to-end solution designed for on-device AI inference
- Ahead-of-Time (AOT) compilation
- Hardware Acceleration (Backend Delegation): Apple, Qualcomm, ARM, Exynos, MediaTek, and Vulkan



THE EXECUTORCH STACK

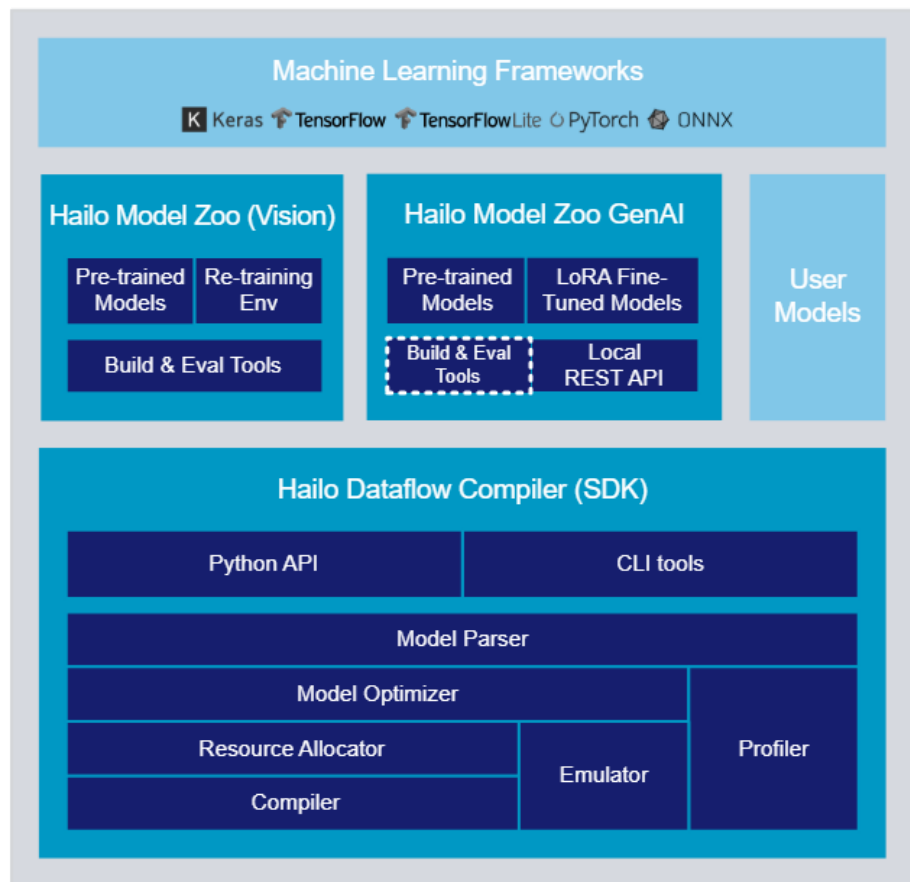


NPU

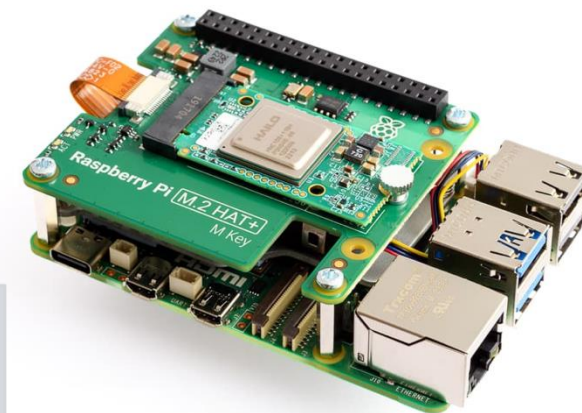
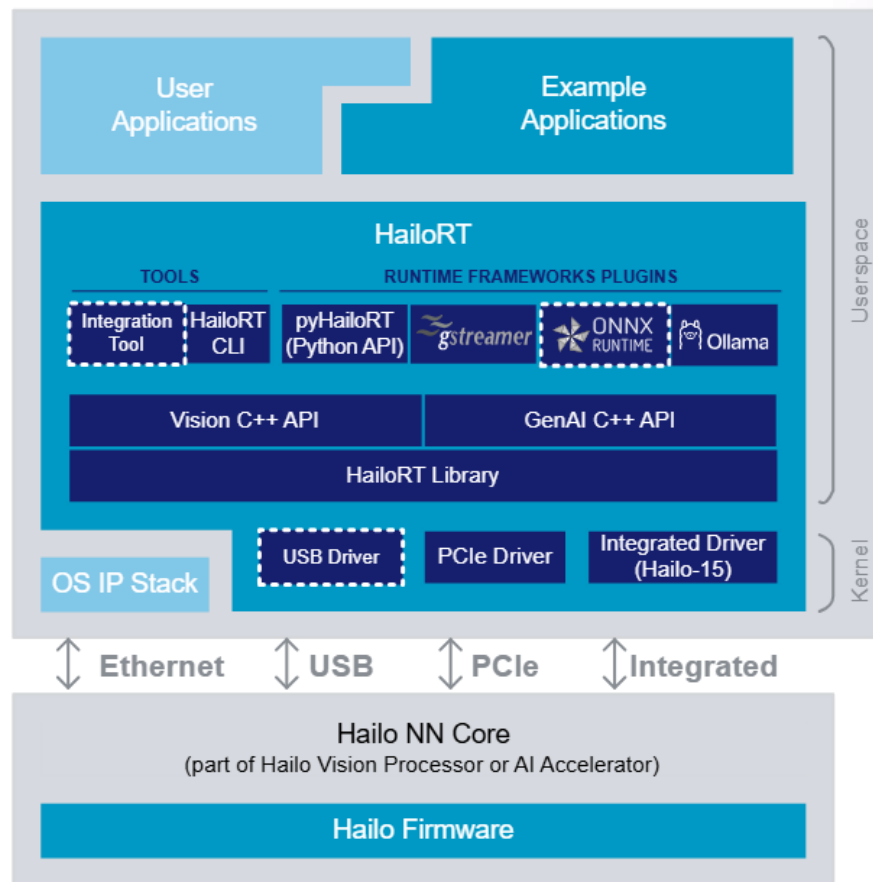


Hailo Software Stack

Model Build Environment

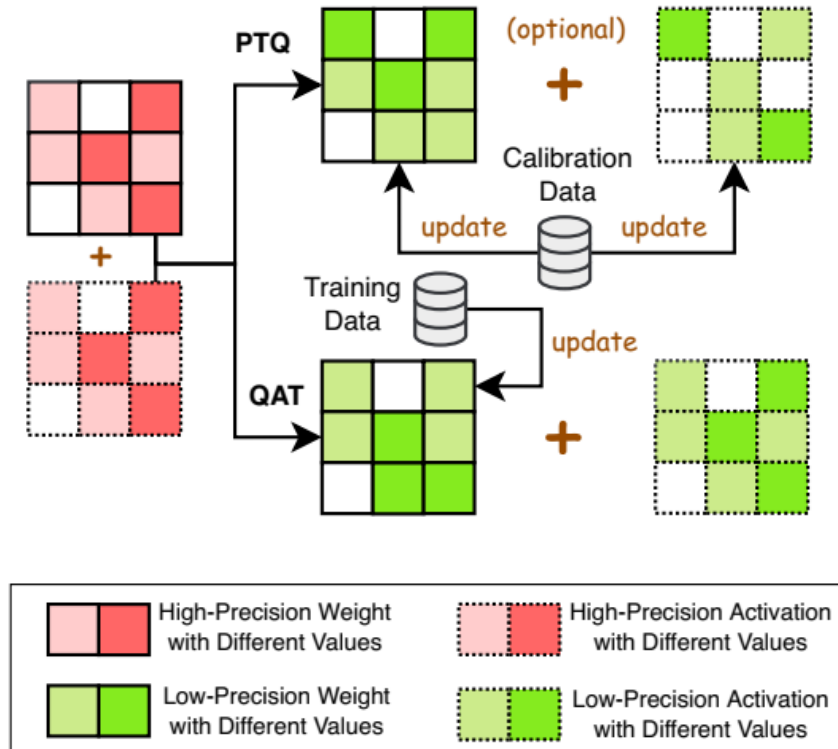


Runtime Environment



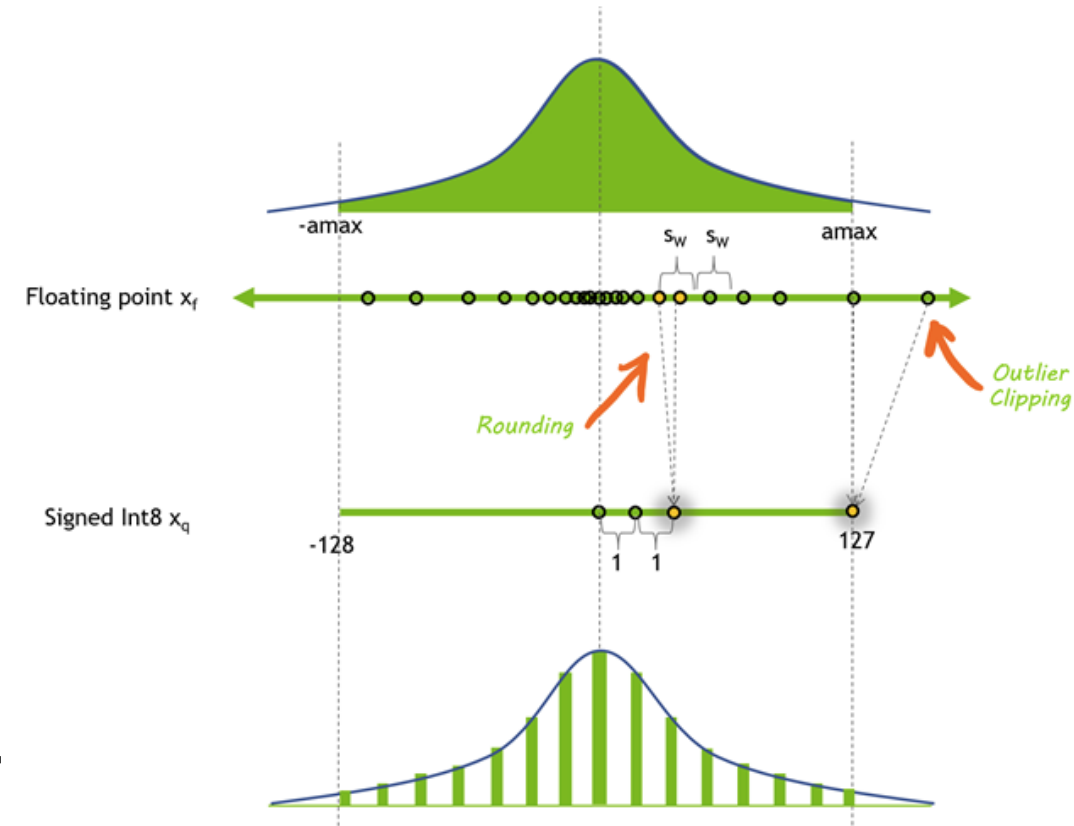
- Hailo software component
- Other software component
- Future support (for Hailo-10H)

Quantization



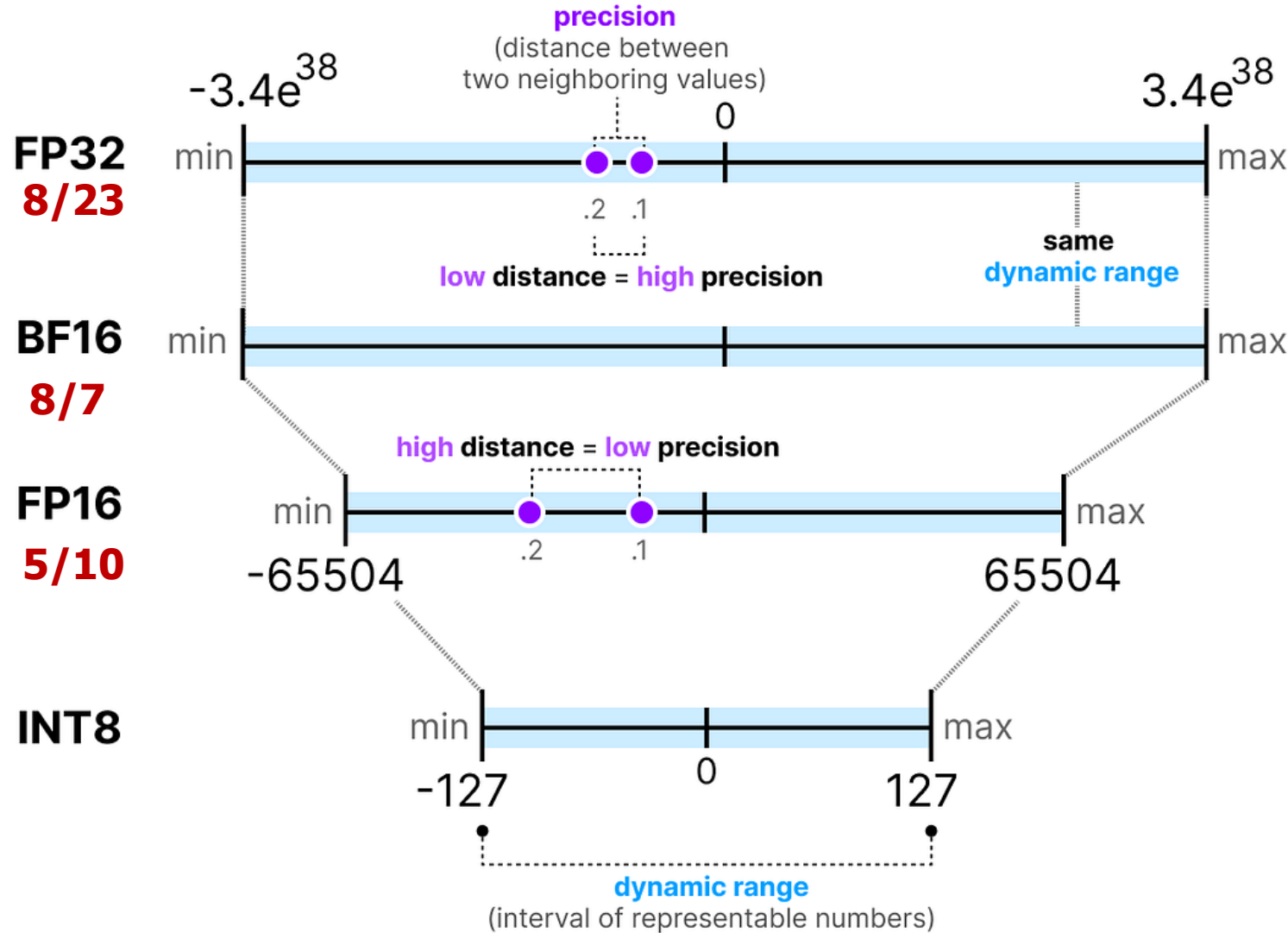
Quantization

- Mapping data to a smaller set of quantization levels
 - FP32 $\rightarrow$ FP16 $\rightarrow$ Int8 $\rightarrow$ Int4 $\rightarrow$ Int2 $\rightarrow$ Binary
- Reduced storage cost, reduced computation requirements.
- Must minimize the quantization error
- The number of **quantization levels** reflects the precision and the number of bits required to represent the data

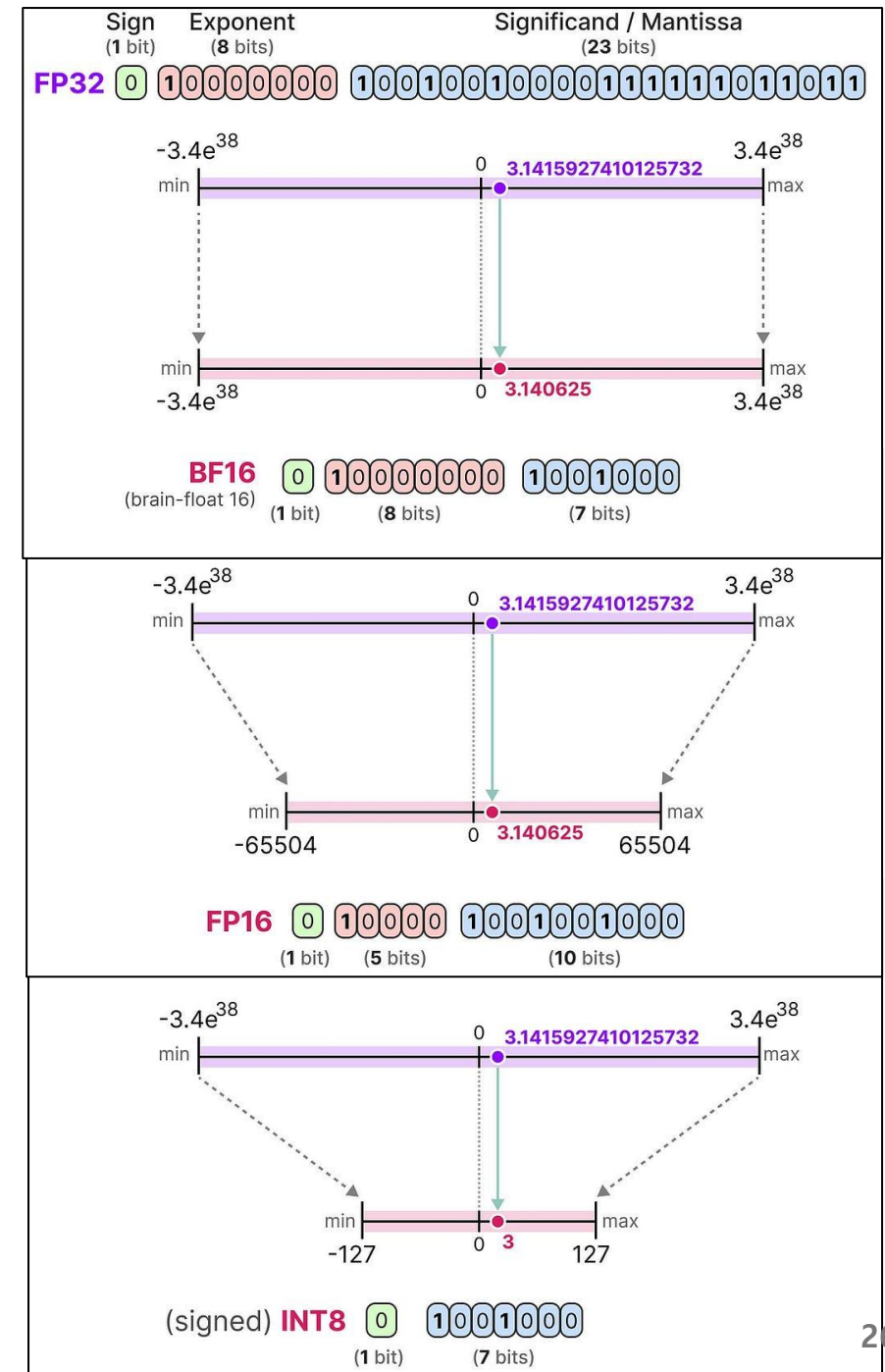


<https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/>

Numeric Data Types



<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>



MX (Microscaling)

- OCP(Open Compute Project) standard Group Quantization
- MXFP8, MXFP6, MXFP4, MXINT8
- MXFP4: E2M1 FP4 + UE8M0 scale (Block 32)
 - Per-block scaling: 32 elements in a block share 8bits scale (E8M0).

S	exp	M
---	-----	---

$$(-1)^S \times 2^{\text{exp} - 1} \times (1 + \frac{M}{2})$$

- UE8M0 scale K (8bits)
 - $2^K \rightarrow$ exponent add

Exp	M=0	M=1
00	0	0.5
01	1	1.5
10	2	3
11	4	6

NVFP4

- MXFP4의 NVIDIA NextGen
- Block size: 16
- FP8 block scale + FP32 Tensor scale
- NVIDIA Hardware native

	MXFP4	NVFP4
실제 element	E2M1 FP4	E2M1 FP4
block size	32개	16개
block scale	E8M0, 8bit	E4M3, 8bit
scale 특성	2^{scale} 정밀도 ↓	정밀도 ↑
global scale	없음	FP32 per-tensor
유효 저장량	4.25 bit/value	약 4.5 bit/value

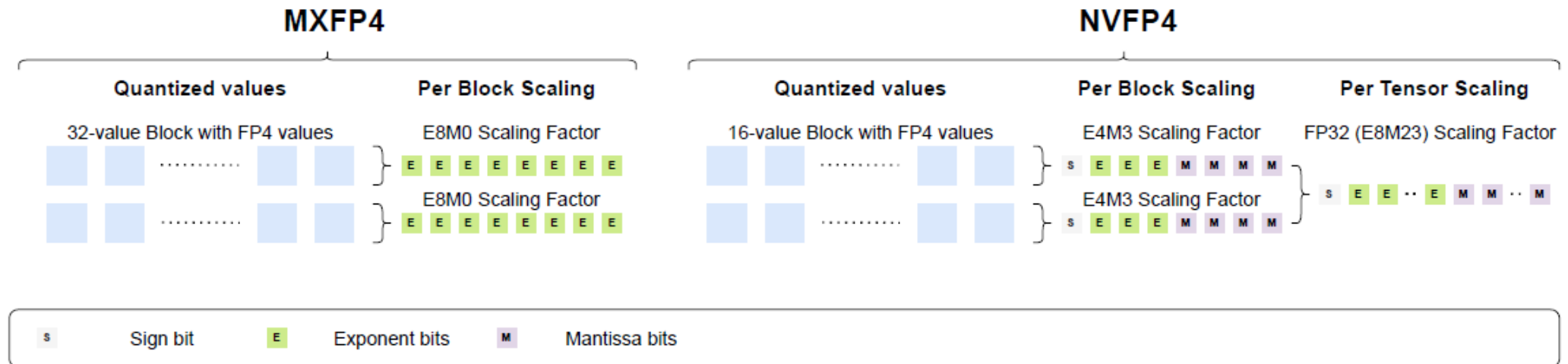


Figure 1: Schematic illustration of the MXFP4 (left) and NVFP4 (right) microscaling formats.

Design Choices in LLM Quantization

A practical design space: largely independent axes with known interactions.

A. Numerical Representation	1. Uniform / Non-uniform	- Uniform: evenly spaced levels, common for INT quantization - Non-uniform: log-scale, NF4, codebook-based vector quantization Codebook / vector quantization belongs here, not under granularity. Uniform (evenly spaced) Non-uniform (e.g., log-scale)	Vector quantization (codebook) Examples: NF4, AQLM, QuIP#, GPTVQ
	2. Symmetric / Asymmetric	- Symmetric: zero-point $z = 0$ - Asymmetric: zero-point derived from range, not usually learned Symmetric ($z = 0$) Asymmetric ($z \neq 0$)	Important LLM notes: - Weights are usually symmetric - Activations are often asymmetric or per-token symmetric - Asymmetric matmul needs zero-point correction terms - INT4 symmetric uses $[-8, 7]$, so one level is effectively wasted
	Format & Bit-width	Options INT8 INT4 INT2 Ternary (-1, 0, +1) FP8 (E4M3, E5M2) FP4 MXFP4 NVFP4 - Scale format matters: FP16 vs power-of-two vs FP8 block scale - Decode is memory-bound: weight-only quantization often helps - Prefill / large batch is compute-bound: W and A both need lower precision to use low-precision tensor cores	
B. Parameter Sharing Scope	4. Granularity	Tensor (per-tensor) Out-channel (per-output-channel) Token (per-token) Head (KV) (per-head) Group (e.g., $g = 128$) Scale should be constant along the reduction dimension so it can move outside the accumulator. - Weight: per-output-channel is hardware-friendly - Activation: per-token is common - Group-wise quantization changes kernel structure: partial accumulation or inner-loop dequantization	
C. Decision Timing and Optimization	5. Static / Dynamic (A / KV only)	- Static: clipping/range fixed by calibration - Dynamic: range recomputed per input or per token Static (calibrated) Dynamic (per-token)	- Weights are essentially always static - Dynamic is often near-free when fused - Static is vulnerable to distribution shift / outlier shift - Dynamic can lose efficiency if kernel fusion is unavailable
	PTQ / QAFT / QAT + Rounding & Calibration	PTQ QAFT QAT Rounding & Calibration - RTN - GPTQ (Hessian-aware) - AdaRound / BRECQ - Data-free vs calibration-based Full QAT is often impractical at LLM scale.	
D. What to Quantize	7. Target Tensors	What can be quantized Weight (W) ↔ Activation (A) ← KV Cache (KV) Example configurations W4A16 W8A8 W4A4 W4A8KV4 ... - Decode = memory-bound → weight quantization is often the first lever - Prefill = compute-bound → quantize both W and A - Long context = KV-bound → quantize or compress KV cache	
E. Preprocess and Realization	8. Pre-quantization Transform	Smoothing / migration Rotation Clipping Outlier decomposition	This is the main accuracy-recovery toolbox in modern LLM quantization.
	9. Simulated vs Real Kernel	Fake / simulated quantization Real low-precision kernel	- Simulated accuracy does not guarantee real speedup - Real performance depends on hardware support, dequantization overhead, and kernel fusion - Reproducibility requires reporting both accuracy and measured kernel/system speed
	Practical decision order	Deployment scenario (decode / prefill / long-context) What to quantize (W / A / KV) Hardware-supported formats & granularity Accuracy recovery (transform, rounding, calibration) Validate with real kernels and end-to-end measurement	

Distribution statistics matter, but usually after the deployment and hardware constraints are fixed.

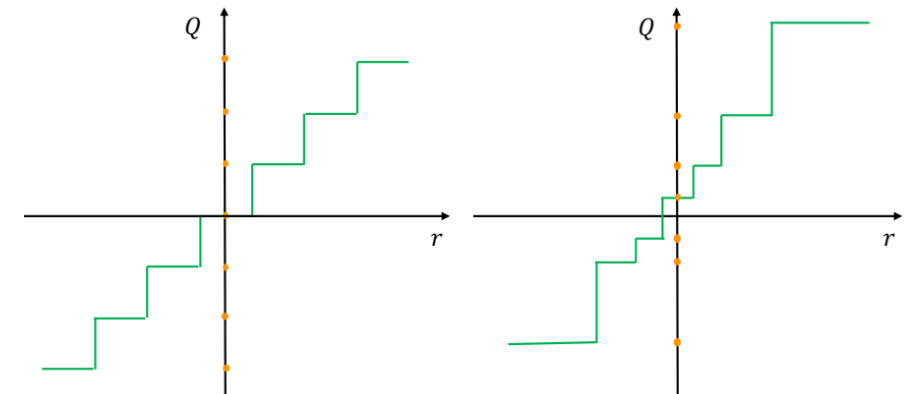
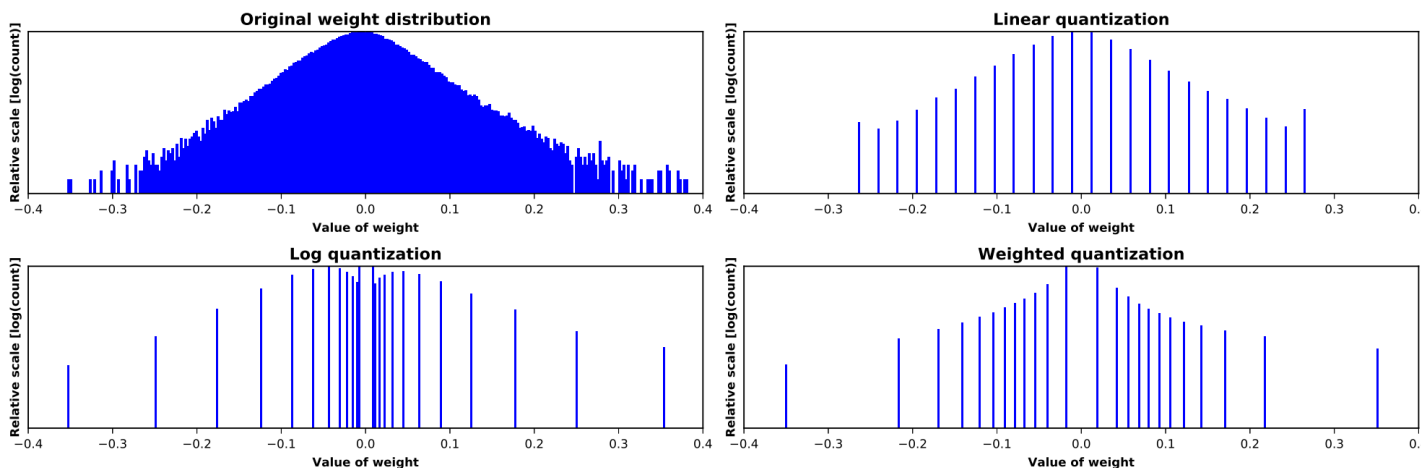
Uniform vs Non-Uniform

- **Uniform Quant (Linear Quant)**

- Affine mapping of integers to real numbers, $r = S(q - Z)$
- Cannot match the realistic data distribution well

- **Non-Uniform Quant (Log Quant)**

- Higher accuracy by focusing more on important value regions
 - 27.8% loss for 4b uniform quant vs. 5% loss for log-2 quant (VGG-16 [LogNet](#))
- Variable step length => difficult to deploy efficiently on real H/W
- Logarithmic approach
 - Exponential variance on step length. Multiplication $\rightarrow$ bit-shift
 - Accumulation needs a full precision storage.



$$Q(r) = X_i, \text{ if } r \in [\Delta_i, \Delta_{i+1})$$

Linear Quantization

$$q = \text{Int}\left(\frac{r}{S}\right) + Z \quad \tilde{r} = S(q - Z)$$

Quantization

Dequantization

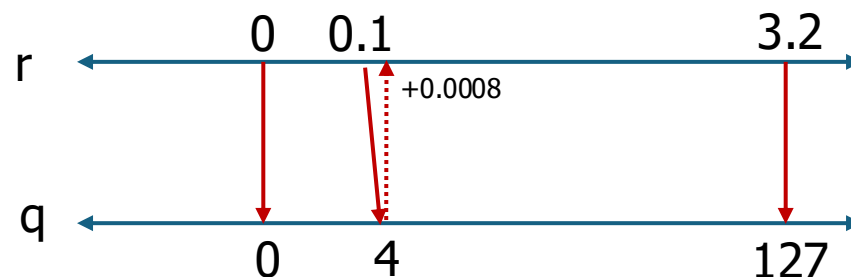
r : real valued input (activation or weight)

S : real valued scaling factor = $\text{max_real} / \text{max_quant}$

Z : integer zero point

Int : rounding operation (e.g., round to nearest and truncation)

$$\mathbf{X}_{\text{quant}} = \text{round}\left(\frac{127}{\max|\mathbf{X}|} \cdot \mathbf{X}\right)$$
$$\mathbf{X}_{\text{dequant}} = \frac{\max|\mathbf{X}|}{127} \cdot \mathbf{X}_{\text{quant}}$$



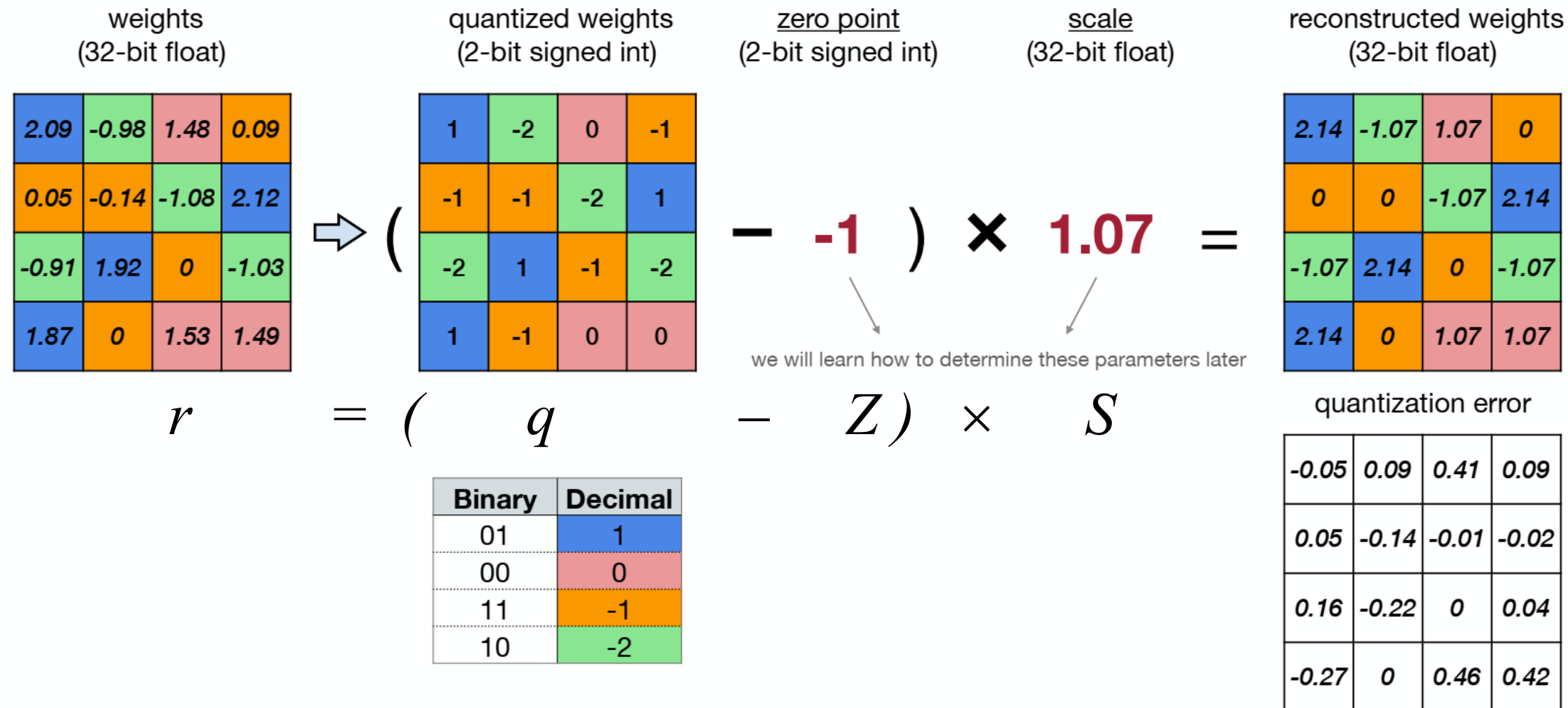
If $Z = 0$, $\max|\mathbf{X}| = 3.2$, 8-bit linear quantization,

A weight of 0.1 is quantized to $\text{round}(0.1 \times 127/3.2) = 4$.

4 is dequantized to $4 \times 3.2/127 = 0.1008$ (quantization error = 0.0008)

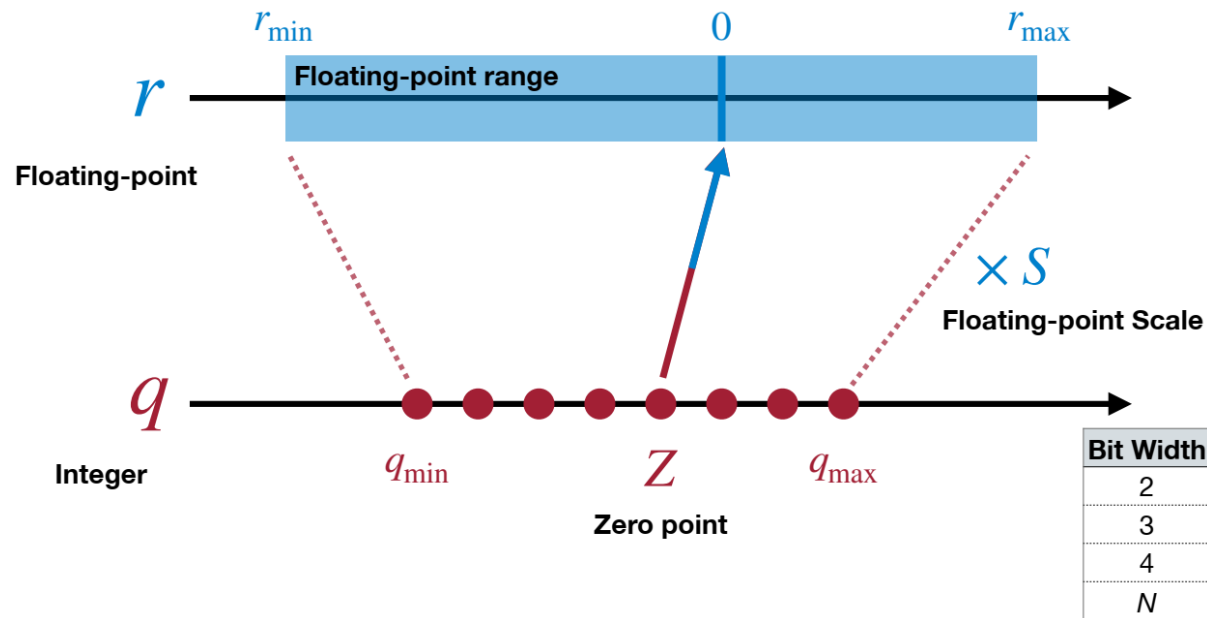
Linear Quantization

An affine mapping of integers to real numbers



Linear Quantization

- An affine mapping of integers to real numbers $r = S(q - Z)$



$$r_{\max} = S(q_{\max} - Z)$$

$$r_{\min} = S(q_{\min} - Z)$$

$$r_{\max} - r_{\min} = S(q_{\max} - q_{\min})$$

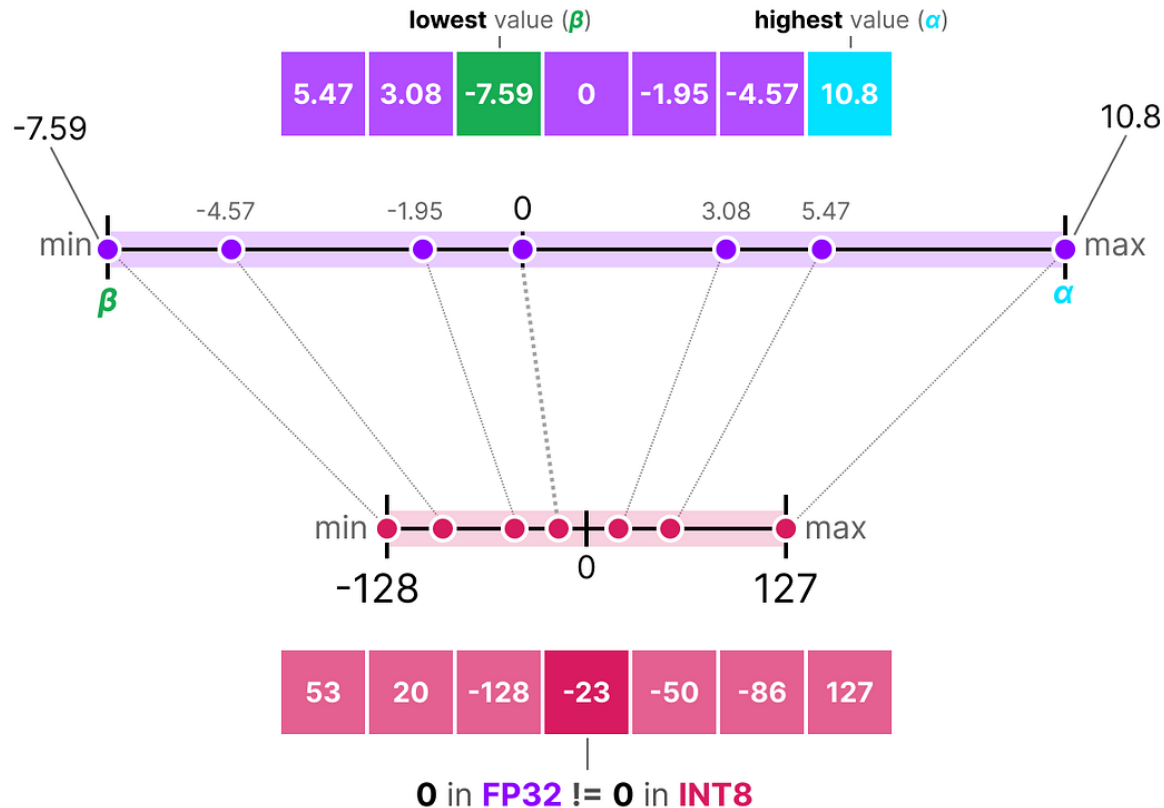
$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

$$r_{\min} = S(q_{\min} - Z)$$

$$Z = q_{\min} - \frac{r_{\min}}{S}$$

$$Z = \text{round}\left(q_{\min} - \frac{r_{\min}}{S}\right)$$

Linear Quantization



$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}} = \frac{10.8 - (-7.59)}{127 - (-128)} = \frac{18.39}{255} = 0.072$$

$$\begin{aligned} Z &= \text{round}\left(q_{min} - \frac{r_{min}}{S}\right) \\ &= \text{round}\left(-128 - \frac{-7.59}{0.072}\right) \\ &= \text{round}(-22.58) = -23 \end{aligned}$$

$$\begin{aligned} \text{Quantize: } q &= \text{round}\left(\frac{r}{S}\right) + Z \\ &= \text{round}\left(\frac{5.47}{0.072}\right) - 23 = 53 \end{aligned}$$

$$\begin{aligned} \text{Dequantize: } r &= S(q - Z) \\ &= 0.072(53 + 23) = 5.472 \end{aligned}$$

Linear Quantized Matrix Multiplication

$$Y = WX$$

$$S_Y(q_Y - Z_Y) = S_W(q_W - Z_W) \cdot S_X(q_X - Z_X) \quad r = S(q - Z)$$

$$q_Y = \frac{S_W S_X}{S_Y} (q_W - Z_W)(q_X - Z_X) + Z_Y$$

$$q_Y = \underbrace{\frac{S_W S_X}{S_Y}}_{\text{The only non-integer}} \underbrace{(q_W q_X - Z_W q_X - \overbrace{Z_X q_W + Z_W Z_X}^{\text{Precompute}})}_{\substack{\text{N-bit Integer Multiplication} \\ \text{32-bit Integer Addition/Subtraction}}} \underbrace{+ Z_Y}_{\substack{\text{N-bit Integer} \\ \text{Addition}}}$$

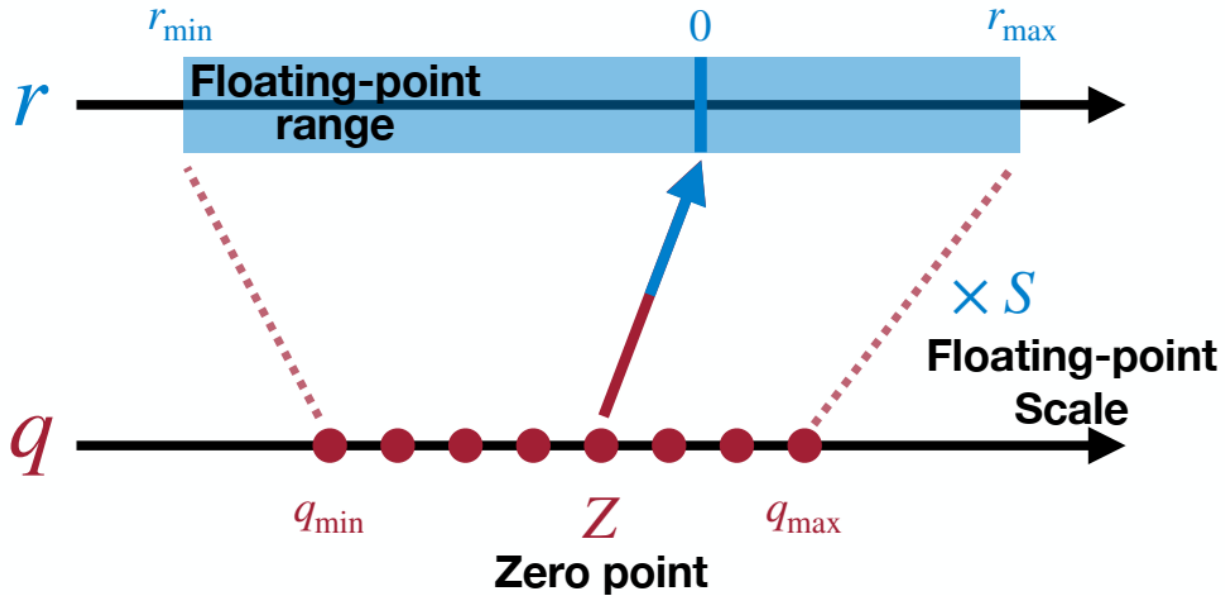
Empirically, $\frac{S_W S_X}{S_Y} \in (0,1)$.

$$\frac{S_W S_X}{S_Y} = 2^{-n} M_0, M_0 \in [0.5,1)$$

(bit shift & fixed-point multiplication)

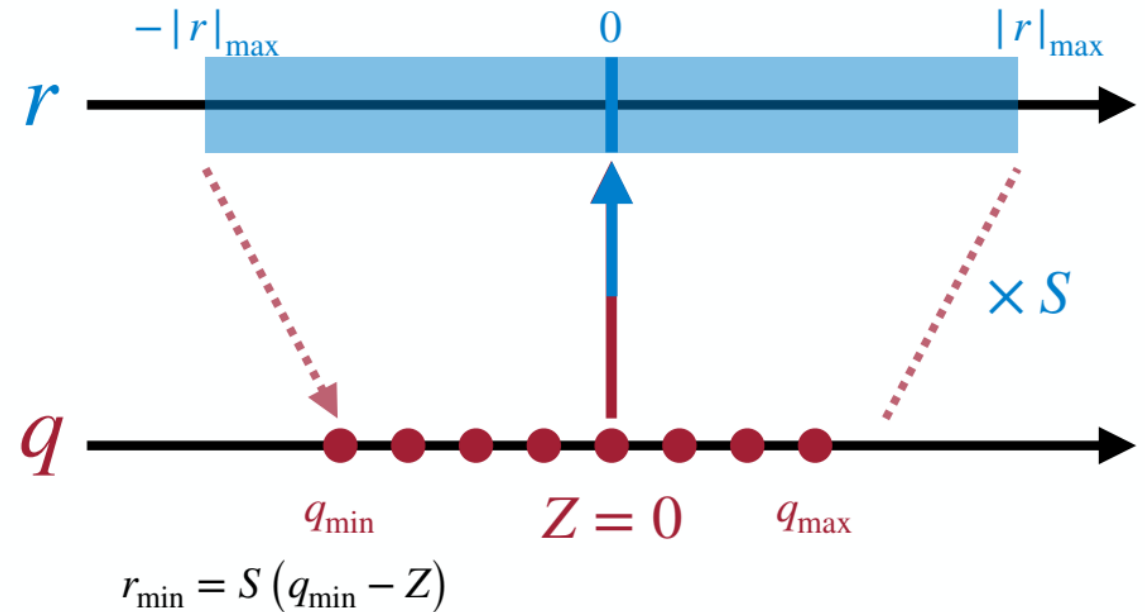
Asymmetric vs. Symmetric

Asymmetric Linear Quantization



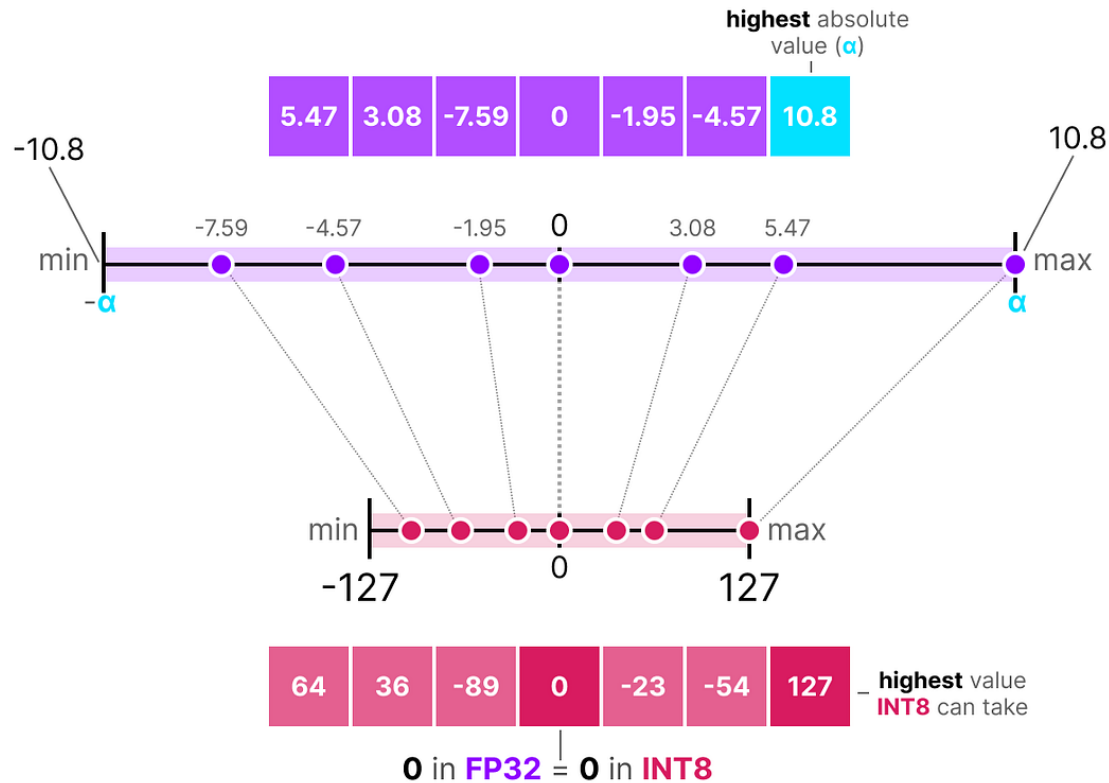
- The quantization range is fully used.
- The implementation is complex.

Symmetric Linear Quantization



- The quantized range will be wasted for biased float range.
- The implementation is simpler.

Symmetric Quantization



$$S = \frac{r_{max}}{q_{max}} = \frac{10.8}{127} = 0.085$$

$$\text{Quantize: } q = \text{round}\left(\frac{r}{s}\right)$$

$$= \text{round}\left(\frac{5.47}{0.085}\right) = 64$$

$$\text{Dequantize: } r = Sq$$

$$= 0.085 \times 64 = 5.44$$

FP32				INT8				FP32				FP32 (original)				FP32 (dequantized)				Quantization error		
5.47	3.08	-7.59		64	36	-89		5.44	3.06	-7.54		5.47	3.08	-7.59		5.44	3.06	-7.54		.03	.02	.05
0	-1.95	-4.57	quantize →	0	-23	-54	dequantize →	0	-1.96	-4.59		0	-1.95	-4.57	-	0	-1.96	-4.59	=	0	-.01	-.02
10.8	3.02	-1.92		127	36	-23		10.8	3.06	-1.96		10.8	3.02	-1.92		10.8	3.06	-1.96		0	-.04	-.04

Quant/Dequant Overhead

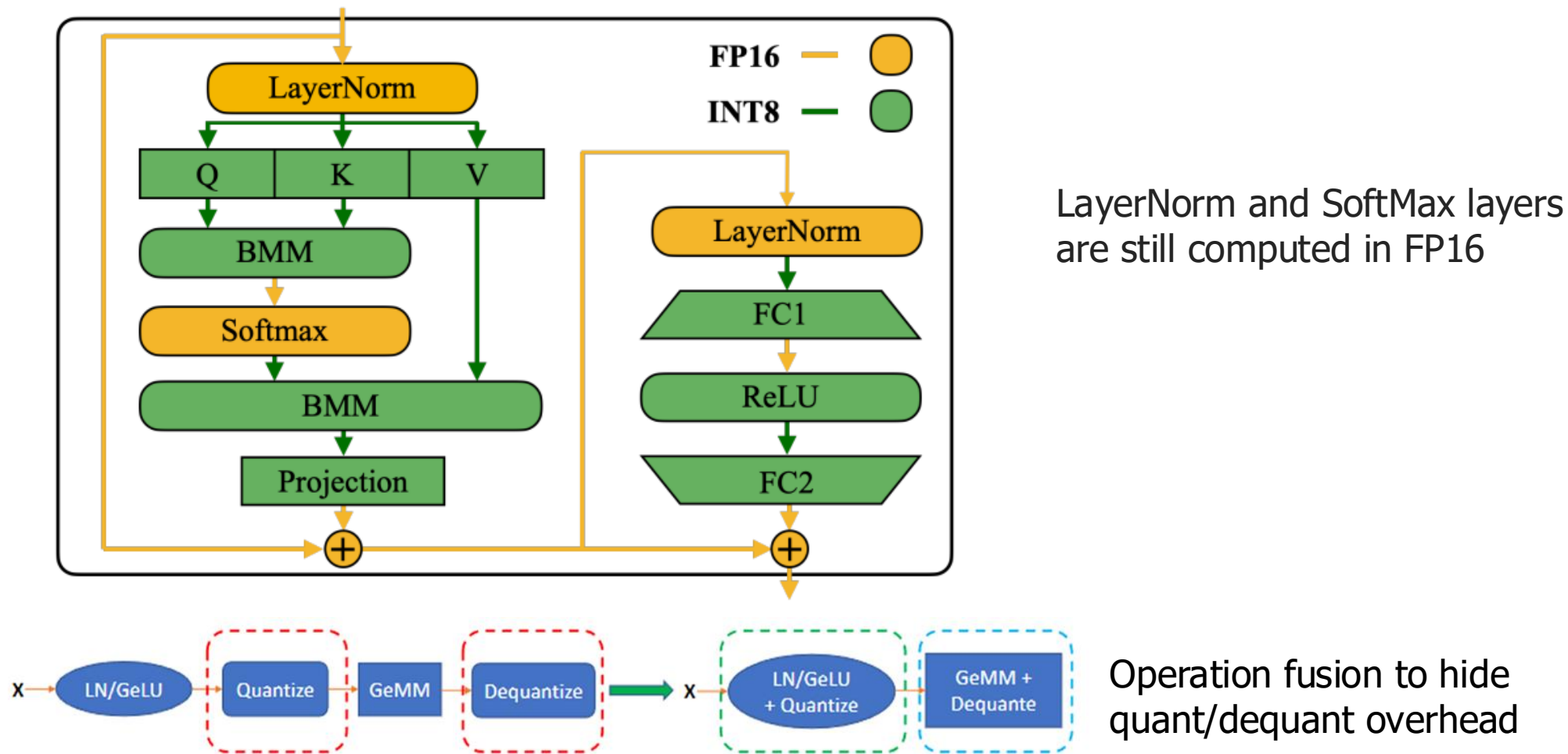
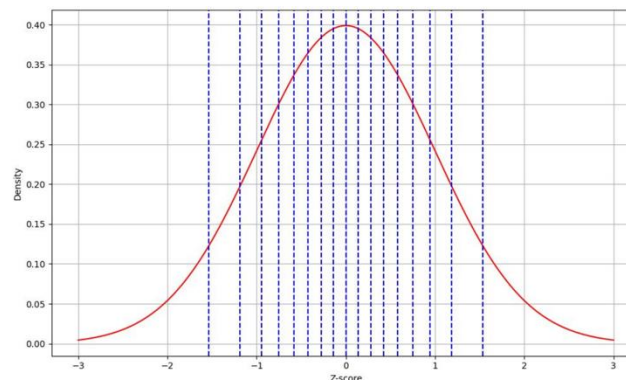


Figure 3: The illustration of normal (left) and our fused (right) INT8 GeMM.

Non-Uniform Quantization - NF4

- NormalFloat (NF) data type builds on **Quantile Quantization**
 - Information-theoretically optimal data type
 - Ensures each quantization bin has an equal number of values assigned from the input tensor
- Assume the standard normal distribution
- Codebook-based



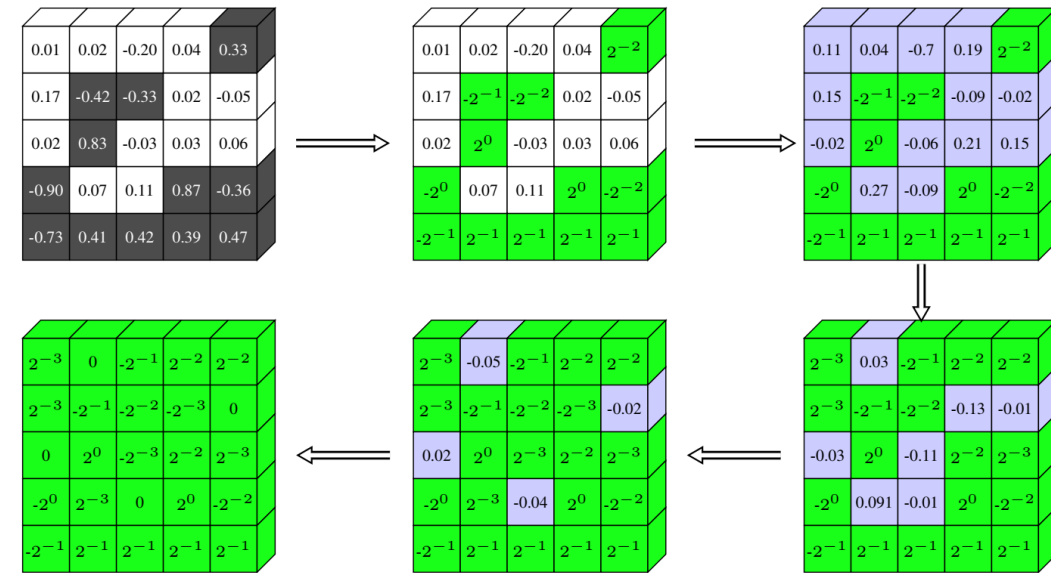
Index (4-bit)	NF4 Quantile Level
0000 (0)	-1
0001 (1)	-0.6961928
0010 (2)	-0.5250929
0011 (3)	-0.3949175
0100 (4)	-0.2844414
0101 (5)	-0.1847734
0110 (6)	-0.09105
0111 (7)	0
1000 (8)	0.0795802
1001 (9)	0.1609302
1010 (10)	0.2471467
1011 (11)	0.3392461
1100 (12)	0.4407098
1101 (13)	0.5580939
1110 (14)	0.7076249
1111 (15)	1

Table 4: Mean 5-shot MMLU test accuracy for LLaMA 7-65B models finetuned with adapters on Alpaca and FLAN v2 for different data types. Overall, NF4 with double quantization (DQ) matches BFloat16 performance, while FP4 is consistently one percentage point behind both.

LLaMA Size Dataset	Mean 5-shot MMLU Accuracy								Mean
	7B		13B		33B		65B		
	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	
BFloat16	38.4	45.6	47.2	50.6	57.7	60.5	61.8	62.5	53.0
Float4	37.2	44.0	47.3	50.0	55.9	58.5	61.3	63.3	52.2
NFloat4 + DQ	39.0	44.5	47.5	50.7	57.3	59.2	61.8	63.9	53.1

Non-Uniform Quantization – 2^k

- Incremental network quantization (INQ)
 - At each iteration, the unquantized weights are partitioned into two disjoint groups (random or magnitude-aware)
 - One group is quantized into **powers of two** or zero while the other remained in high precision
 - Retraining to compensate the accuracy loss.
 - The portion of quantized weights gradually increases until it covers all weights.



Non-Uniform Quantization

- Binarycode-based Quantization
 - a real-number vector $\mathbf{r} \in \mathbb{R}^n$ is quantized into m binary vectors
 - $\mathbf{r} \approx \sum_{i=1}^m \alpha_i \mathbf{b}_i$
 - Scaling factors $\alpha_i \in \mathbb{R}$, Binary vectors $\mathbf{b}_i \in \{-1, +1\}^n$.

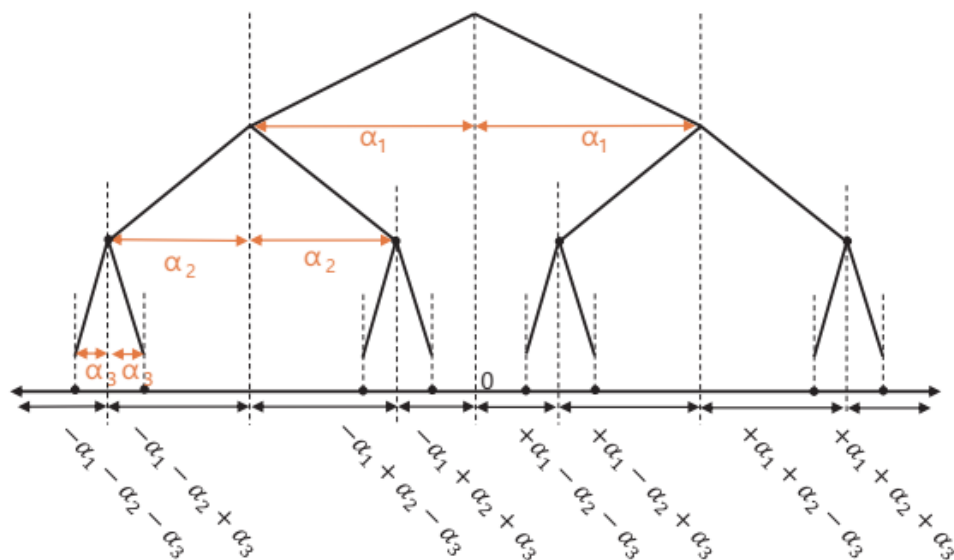


Fig. 3. An illustration of binary-coding-based quantization when the number of quantization bits is 3.

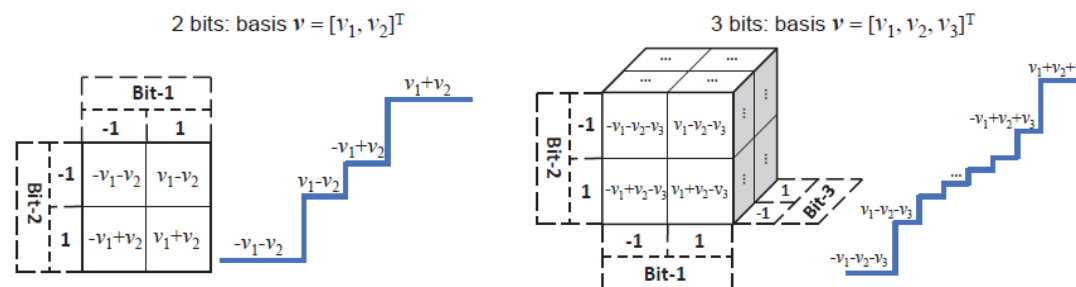
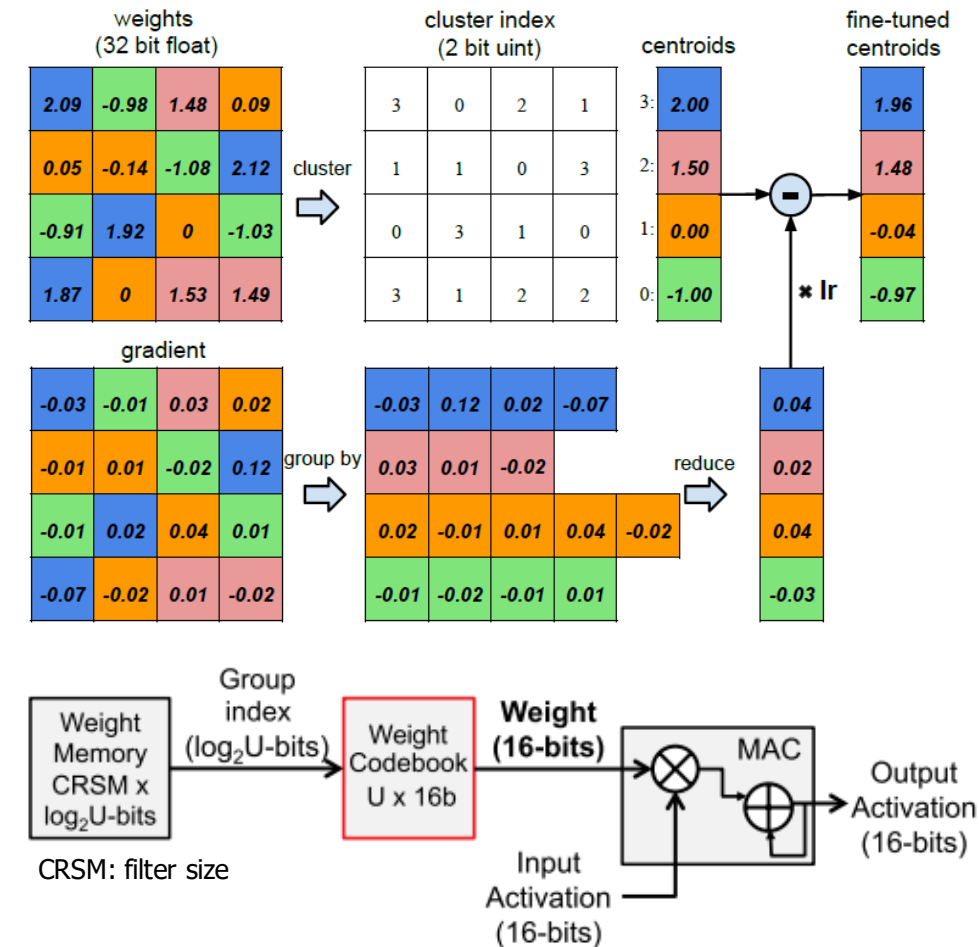


Fig. 2: Illustration of our learnable quantizer on the 2-bit (left) and 3-bit (right) cases. For each case, the left figure shows how quantization levels are generated by the basis vector, and the right figure illustrates the corresponding quantization function.

[LQ-Nets](#) (ECCV'18)

Learned Quantization (Codebook)

- Weight sharing
 - Weights are grouped using a hashing function or a k-means algorithm
 - One value is assigned to each group
 - Reduce # of unique weights in a filter
- Codebook-based mapping
 - Each group of weights has an index
 - The index is stored for each position in the filter rather than a weight value.
 - Two-step process to fetch the weight
- Reduce the cost of weight access
 - if bitwidth of group index < bitwidth of weight
 - not reduce the precision of the MAC computation itself



Learned Quantization

Table 1: The compression pipeline can save $35\times$ to $49\times$ parameter storage with no loss of accuracy.

Network	Top-1 Error	Top-5 Error	Parameters	Compress Rate
LeNet-300-100 Ref	1.64%	-	1070 KB	$40\times$
LeNet-300-100 Compressed	1.58%	-	27 KB	
LeNet-5 Ref	0.80%	-	1720 KB	$39\times$
LeNet-5 Compressed	0.74%	-	44 KB	
AlexNet Ref	42.78%	19.73%	240 MB	$35\times$
AlexNet Compressed	42.78%	19.70%	6.9 MB	
VGG-16 Ref	31.50%	11.32%	552 MB	$49\times$
VGG-16 Compressed	31.17%	10.91%	11.3 MB	

Vector Quantization

• QuIP#

① Randomized Hadamard Transform

- $\tilde{W} = UWV^T$
- $U = H_m S_U, V = H_n S_V$
- H: normalized Hadamard matrix,
- S: random ± 1 diagonal matrix
- outlier-heavy distribution $\rightarrow$ more Gaussian / isotropic distribution

② E₈ Vector Quantization

- $w = [w_1, w_2, \dots, w_8] \in R^8, w \rightarrow c_k \in \mathcal{C}$
- 256-entry codebook (not 64K entries)

1.2	-0.4	0.1	2.8	-1.5	0.6	-0.2	1.0
-----	------	-----	-----	------	-----	------	-----



0.52	-0.68	0.44	0.17	-0.39	0.63	-0.58	0.29
------	-------	------	------	-------	------	-------	------



0.50	-0.75	0.50	0.25	-0.50	0.75	-0.50	0.25
------	-------	------	------	-------	------	-------	------



16-bit codeword (2 bits per weight)

00110101	0100101	1
LUT index (8 bit)	Sign (7 bit)	Shift (1 bit)

From Codebook

1/4	1	1/4	0	3/4	1/2	3/4	0
-----	---	-----	---	-----	-----	-----	---



1/4	-1	1/4	0	-3/4	1/2	-3/4	0
-----	----	-----	---	------	-----	------	---



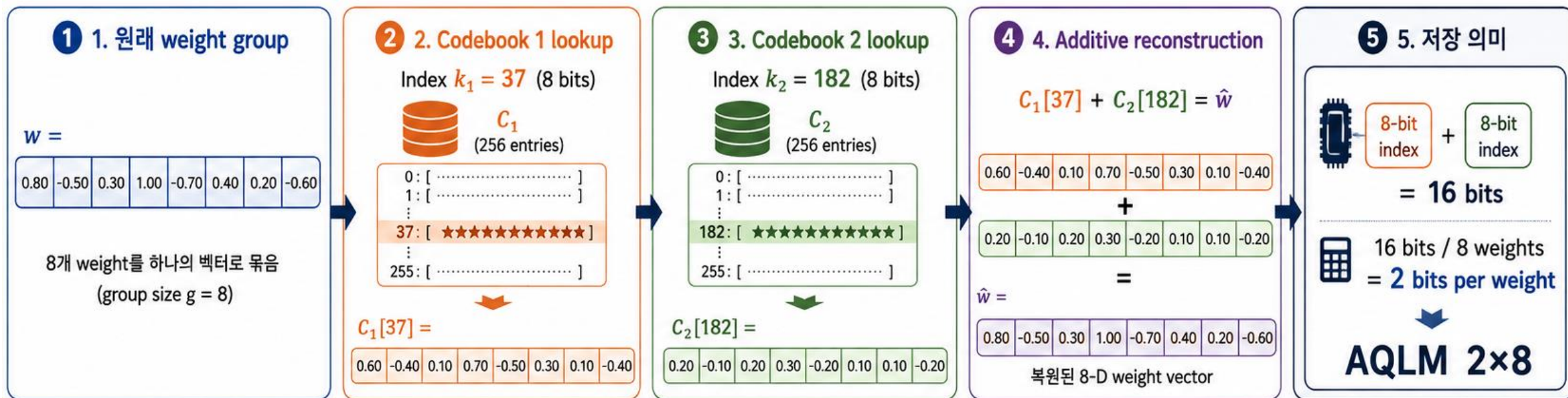
1/2	-3/4	1/2	1/4	-1/2	3/4	-1/2	1/4
-----	------	-----	-----	------	-----	------	-----

Vector Quantization

- AQLM
 - Learned multi-codebook, Additive VQ

$$\mathbf{w}_g \approx C_1[k_1] + C_2[k_2] + \cdots + C_M[k_M]$$

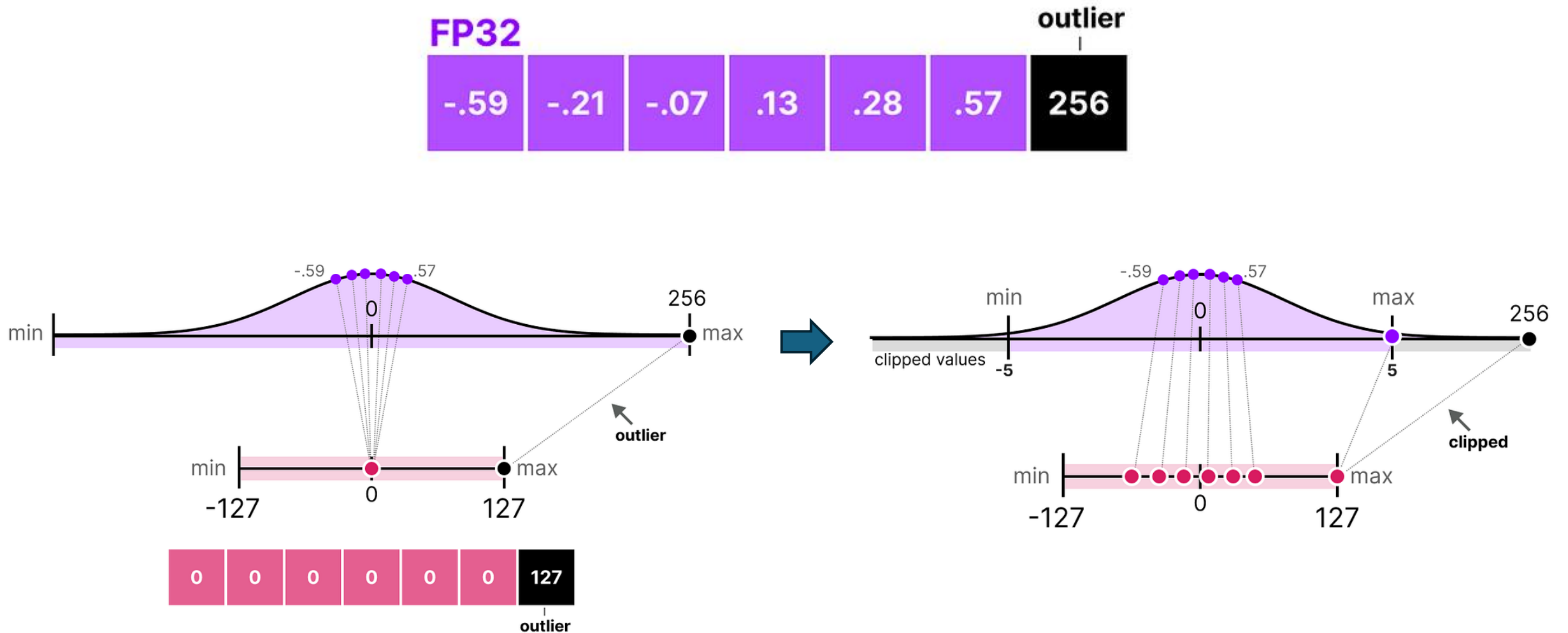
8개 weight를 두 개의 8-bit index로 표현하는 learned additive vector quantization



Post-Training Quantization (PTQ)

- Perform the quantization and the adjustments of the weights, **without any fine-tuning**
- A pre-trained model is **calibrated** using calibration data (e.g., a small subset of training data) to **compute the clipping ranges and the scaling factors**.
- Low overhead, but lower accuracy as compared to QAT, especially for low-precision
- How should we get the optimal linear quantization parameters (S , Z)?

Range Mapping and Clipping

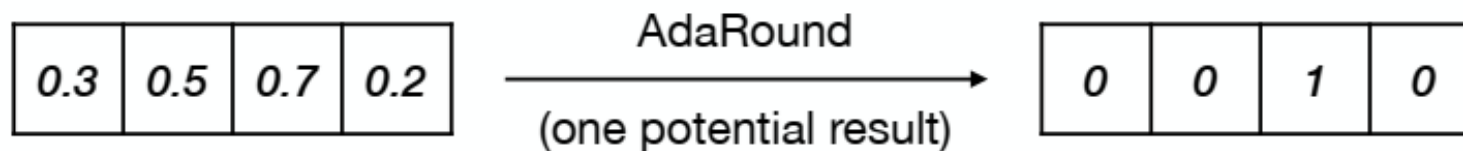
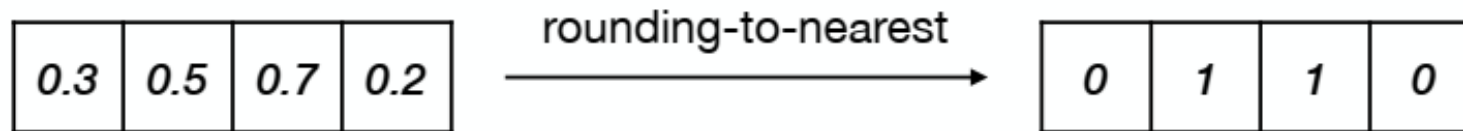


Static vs Dynamic Quantization

- Static quantization
 - Clipping range (min/max) is pre-calculated and static during inference.
 - No runtime computational overhead.
 - Weights are essentially always static.
 - Activations can also be determined via calibration data.
 - Vulnerable to distribution shift.
 - Pre-calculation method
 - minimizing Mean Squared Error (MSE) between original unquantized weight distribution and the corresponding quantized values or entropy
- Dynamic quantization
 - Clipping range is dynamically calculated for each activation map during runtime
 - real-time computation of the signal statistics (min, max, percentile, etc.)
 - Higher accuracy

Rounding

- RTN: project to the nearest discrete level
- Adaptive rounding: AdaRound, BRECQ
 - $q_i = \left\lfloor \frac{w_i}{\Delta} \right\rfloor + h(\alpha_i)$, $h(\alpha_i) \in \{0, 1\}$
 - Find $h(\alpha_i)$ to minimize $\|WX - \hat{W}(\alpha)X\|^2$
 - Using Calibration data

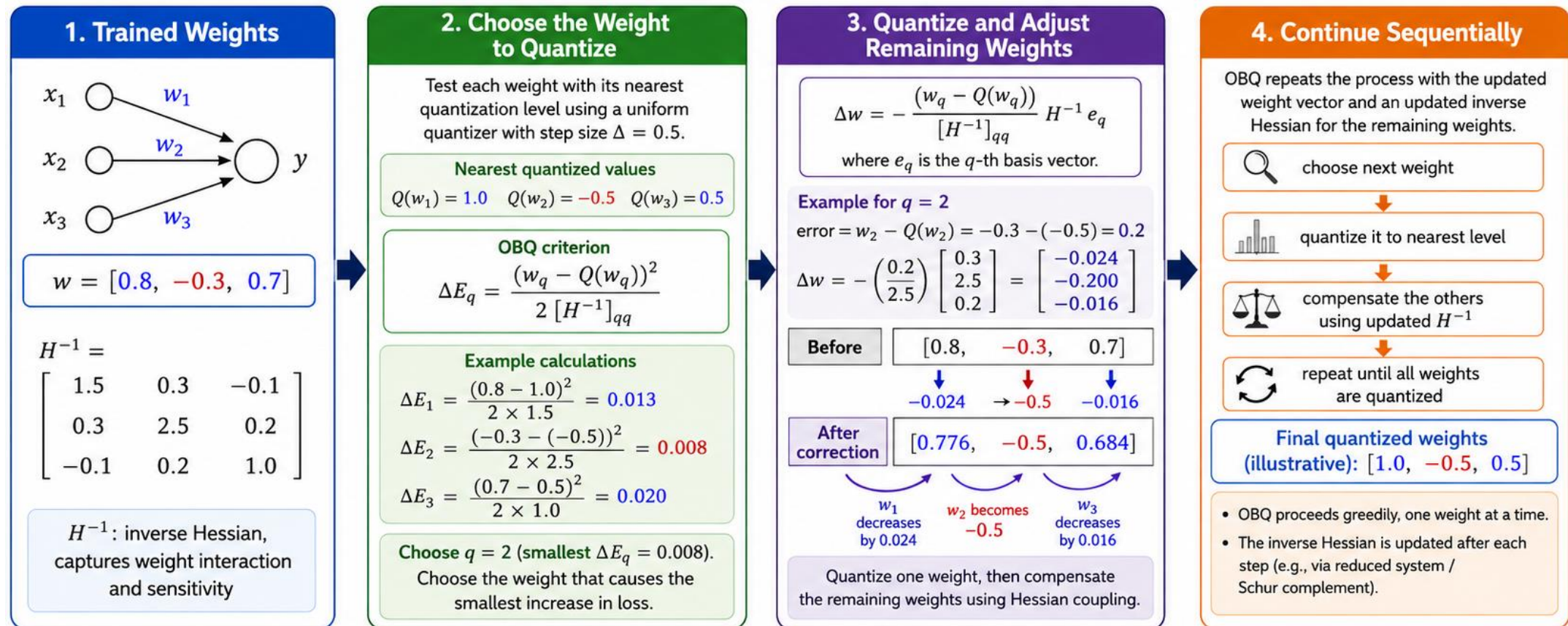


[Up or Down? Adaptive Rounding for Post-Training Quantization \[ICML'20\]](#)

[BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction](#)

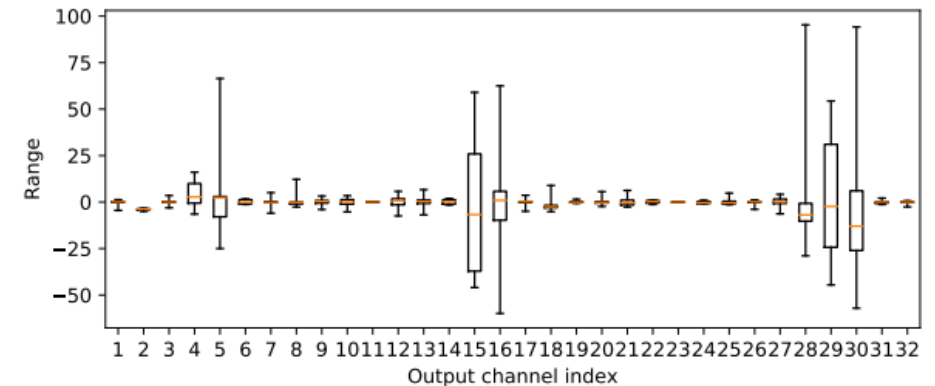
Optimal Brain Quantization (OBQ)

- A second-order PTQ method minimizing the increase in loss
 - Uses **Hessian** (curvature) information, not just nearest rounding.
 - Selects the weight with the smallest estimated loss increase.
 - After quantizing one weight, it adjusts the others to reduce output error.



Per-Channel Weight Quantization

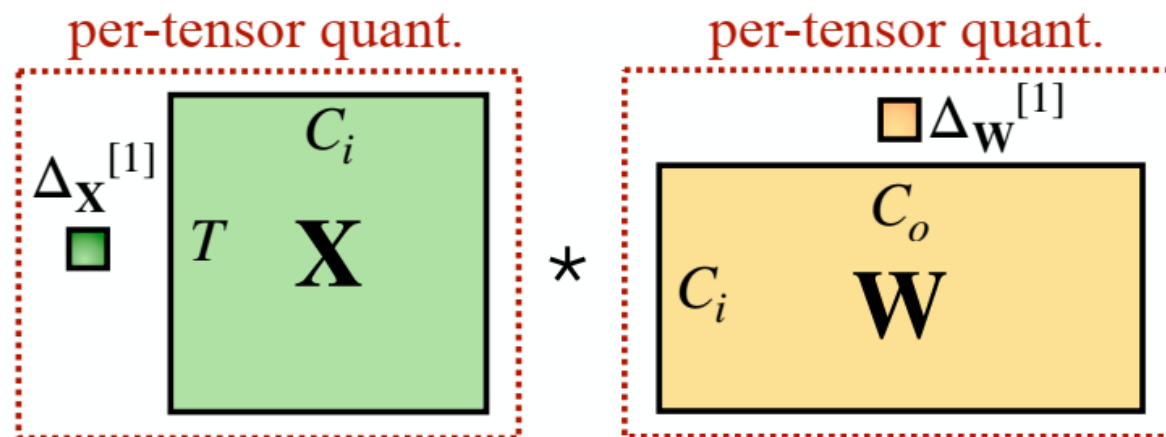
- Using single scale for whole weight tensor (**Per-Tensor** Quantization)
 - works sufficiently well for large models
 - leads to significant accuracy drops for small models
- Common failure results from
 - Large differences (more than $100\times$) in ranges of weights for different output channels
 - Outlier weight values that make all remaining weights less precise after quantization
- Solution: **Per-Channel** Quantization



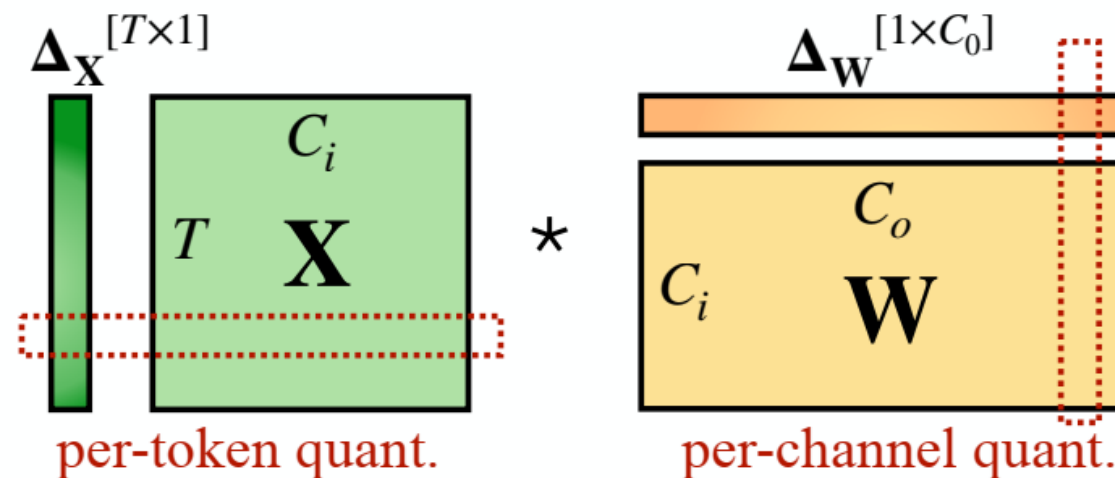
strong differences btwn channel weight ranges

[Data-Free Quantization Through Weight Equalization and Bias Correction \[ICCV 2019\]](#)

Per-Channel Weight Quantization



(a) per-tensor quantization



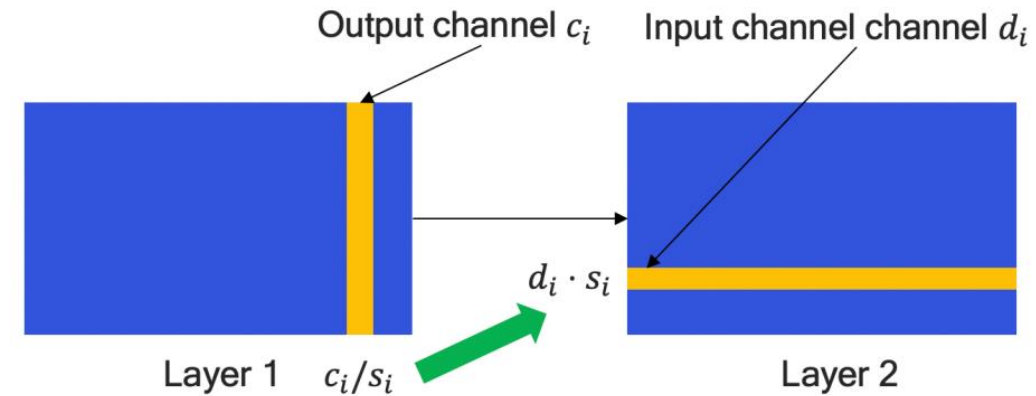
(b) per-token + per-channel quantization

Weight Equalization

- Rescaling to **equalize the weight ranges** of different output channels
- Scaling equivariance
 - scaling **down** layer k weights of output channel c_i by s_i
 - scaling **up** layer $k + 1$ weights of input channel d_i by s_i
- How to determine s_i

$$s_i = \frac{1}{r_{d_i}^{k+1}} \sqrt{r_{c_i}^k r_{d_i}^{k+1}}$$

- $r_{c_i}^k$: quantization weight range of output channel i (c_i) in Layer k
- $r_{d_i}^{k+1}$: quantization weight range of input channel i (d_i) in Layer $k + 1$



$$\tilde{r}_{c_i}^k = \frac{r_{c_i}^k}{s_i} = \frac{r_{c_i}^k \cdot r_{d_i}^{k+1}}{\sqrt{r_{c_i}^k r_{d_i}^{k+1}}} = \sqrt{r_{c_i}^k r_{d_i}^{k+1}}$$
$$\tilde{r}_{d_i}^{k+1} = r_{d_i}^{k+1} \cdot s_i = \sqrt{r_{c_i}^k r_{d_i}^{k+1}}$$

Activation function must be linear. (e.g., ReLU)

Post-Training INT8 Linear Quantization

Activation	Symmetric	Asymmertric			Asymmertric	
	Per-Tensor	Per-Tensor			Per-Tensor	
	Minimize KL-Divergence	Exponential Moving Average (EMA)			mean +/- 6 * std (from BatchNorm)	
Weight	Symmetric	Asymmetric	Asymmetric	Symmetric	Asymmetric	
	Per-Tensor	Per-Tensor	Per-Channel	Per-Channel	Per-Tensor	
	-	-	-	-	Weight Equalization Bias Correction	
Neural Network	GoogleNet	-0.45%	0%	0%	0%	-
	ResNet-50	-0.13%	-0.6%	-0.6%	-0.6%	-
	ResNet-152	-0.08%	-1.7%	-1.8%	-1.8%	-
	MobileNetV1	-	-70.8%	-0.6%	-11.8%	-0.3%
	MobileNetV2	-	-71.8%	-2.2%	-2.1%	-0.5%

Smaller models seem to not respond as well to post-training quantization, presumably due to their smaller representational capacity

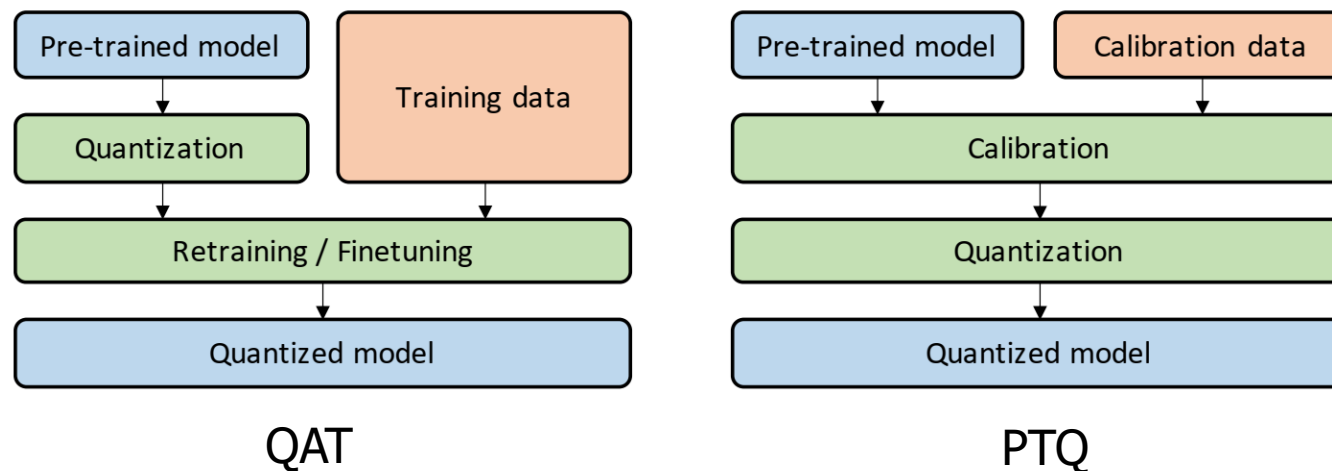
➔ How should we improve performance of quantized models?

Quantization-Aware Training (QAT)

How should we improve performance of quantized models?

Fine-tuning Methods

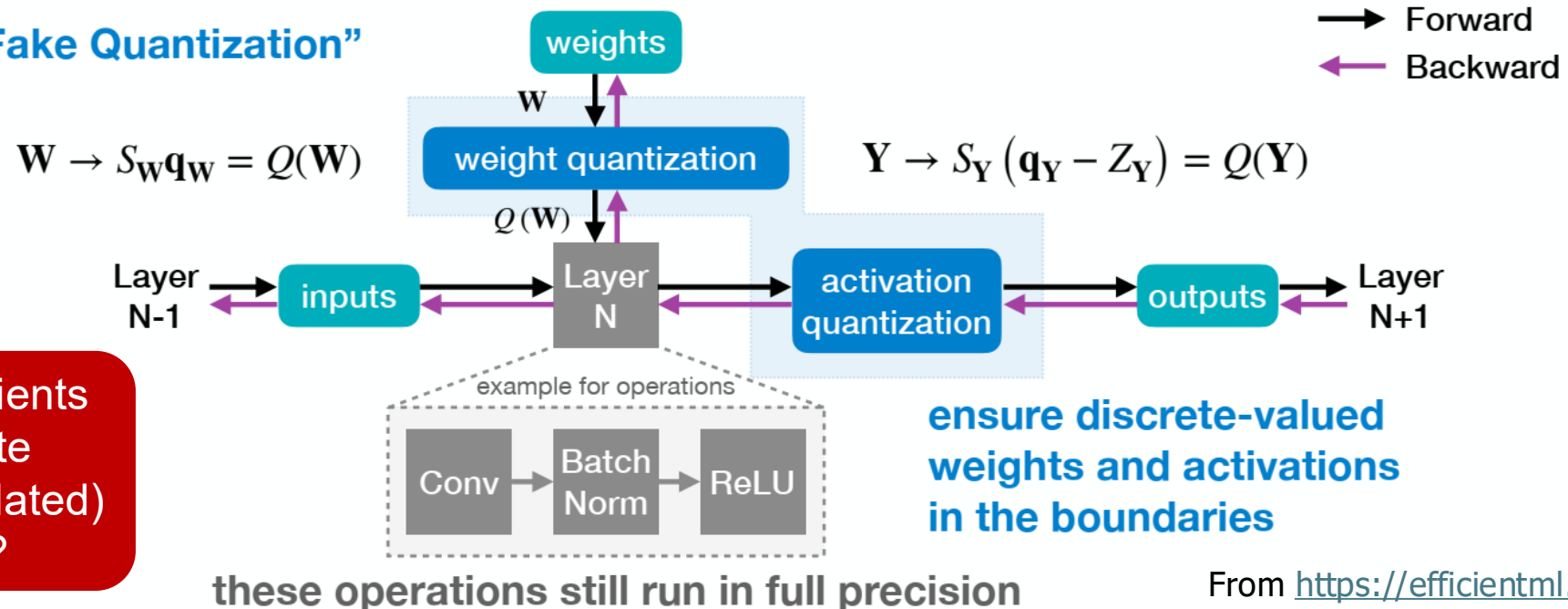
- Quantization may introduce a perturbation to the trained model parameters
- Need to adjust the parameters in the NN after quantization
- **Quantization-Aware Training (QAT)**
 - A pre-trained model is quantized and then fine-tuned using training data to adjust parameters and recover accuracy degradation



Quantization-Aware Training

- Train the model taking quantization into consideration
- A full precision copy of the weights W is maintained throughout the training
- The small gradients are accumulated without loss of precision
- Once the model is trained, only the quantized weights are used for inference

“Simulated/Fake Quantization”

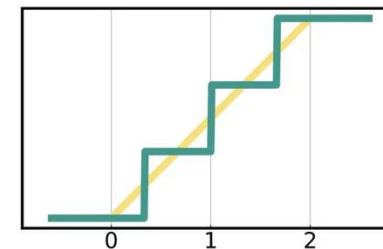
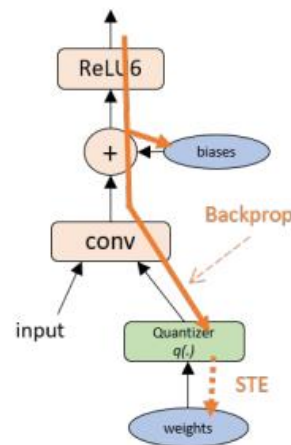


How should gradients back-propagate through the (simulated) quantization?

Gradient of Quantization Function

- Quantization function is **nondifferentiable** with respect to its input
 - Gradient presented as delta-spikes at the transition points
 - Not friendly to the SGD learning
- Straight Through Estimator (STE)**
 - Ignore the rounding operation and approximates it with an **identity** function
 - Simply bypasses the quantization function during the gradient BP.
 - Take the gradient at the quantized value and perform the update on the original real value
 - Works well in practice, except for ultra low-precision quantization such as binary quantization

$$\frac{\partial \mathbf{w}_q}{\partial \mathbf{w}} = \frac{\partial q(\mathbf{w})}{\partial \mathbf{w}} \stackrel{STE}{\approx} \frac{\partial I(\mathbf{w})}{\partial \mathbf{w}} = 1$$
$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial L(\mathbf{w}_q)}{\partial \mathbf{w}_q} \cdot \frac{\partial \mathbf{w}_q}{\partial \mathbf{w}} \stackrel{STE}{\approx} \frac{\partial L(\mathbf{w}_q)}{\partial \mathbf{w}_q}$$



Forward pass:
 $X_q = \text{Round}(\text{Clip}(0, X_r, 2^n - 1))$

Backward pass:
 $\frac{\partial \mathcal{L}_{\text{loss}}}{\partial X_r} \stackrel{STE}{=} \frac{\partial \mathcal{L}_{\text{loss}}}{\partial X_q}$

Quantization-Aware Training

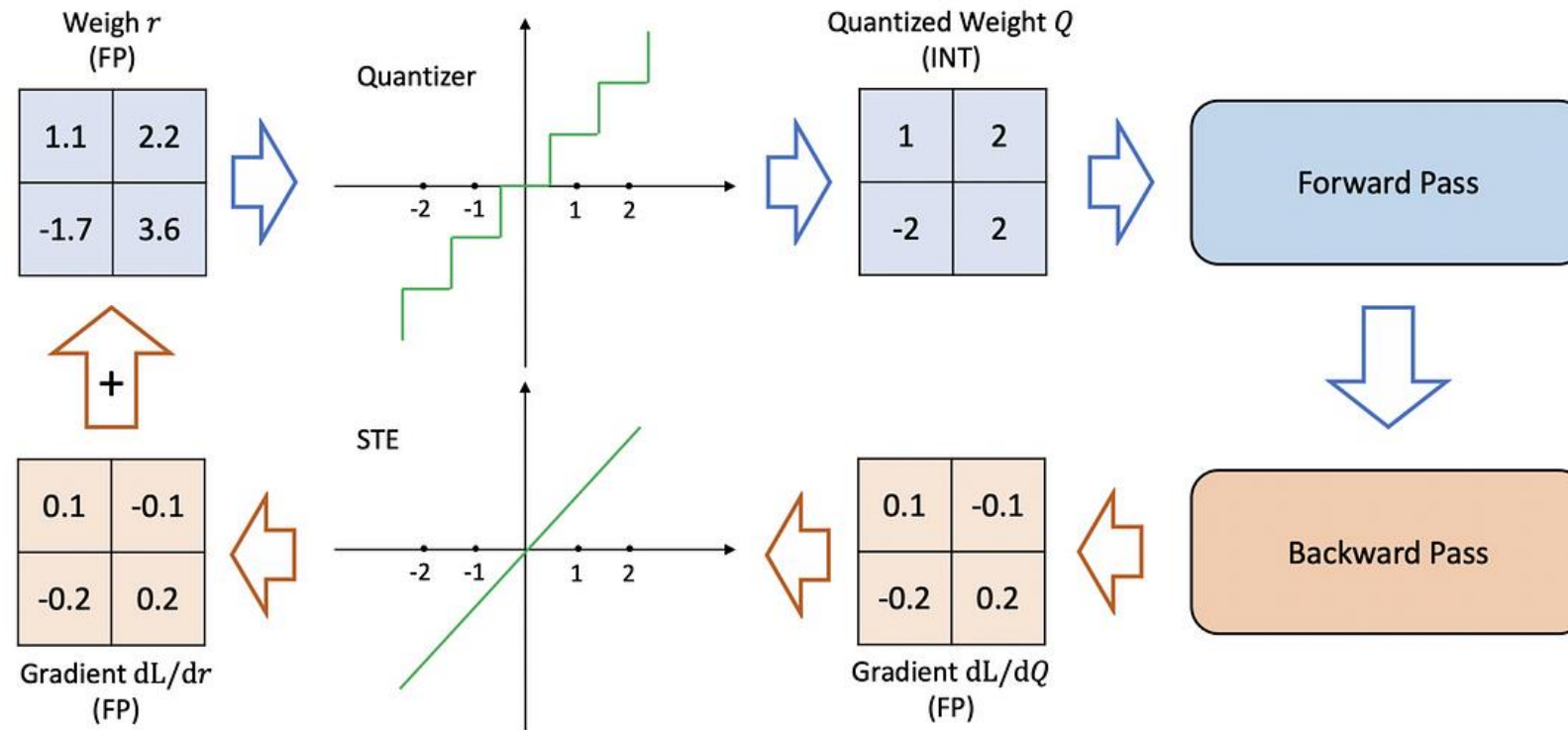
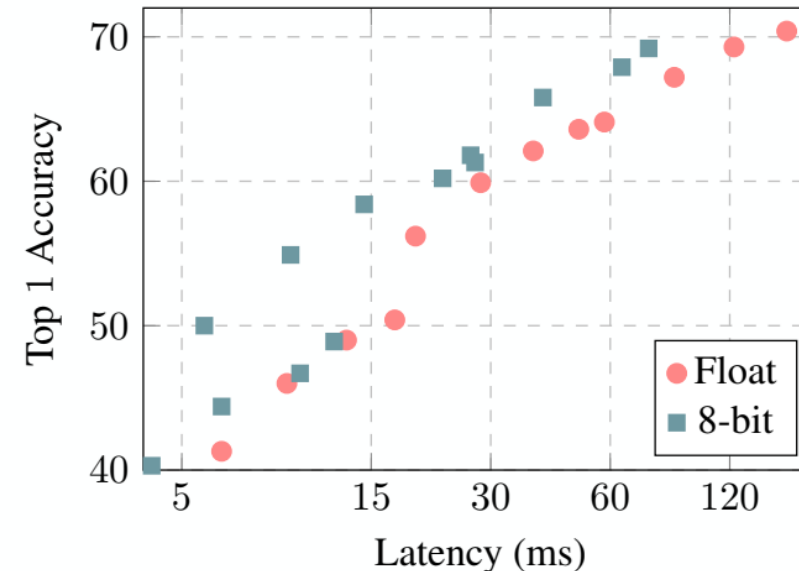


Figure 5: Illustration of Quantization-Aware Training procedure, including the use of Straight Through Estimator (STE).

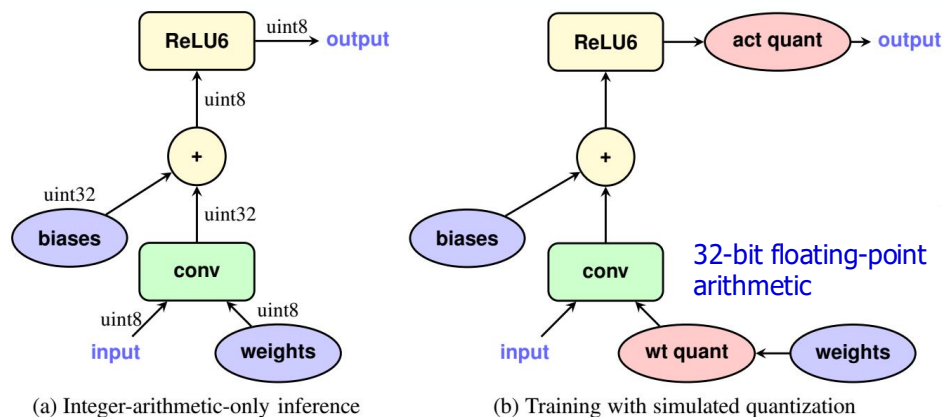
INT8 Linear Quantization-Aware Training

- An affine mapping of integers to real numbers $r = S(q - Z)$

Neural Network	ResNet-50	Inception-V3
Floating-point Accuracy	76.4%	78.4%
8-bit Integer-quantized Accuracy	74.9%	75.4%



Latency-vs-accuracy tradeoff of float vs. integer-only MobileNets on ImageNet using Snapdragon 835 big cores.



Weight update during training

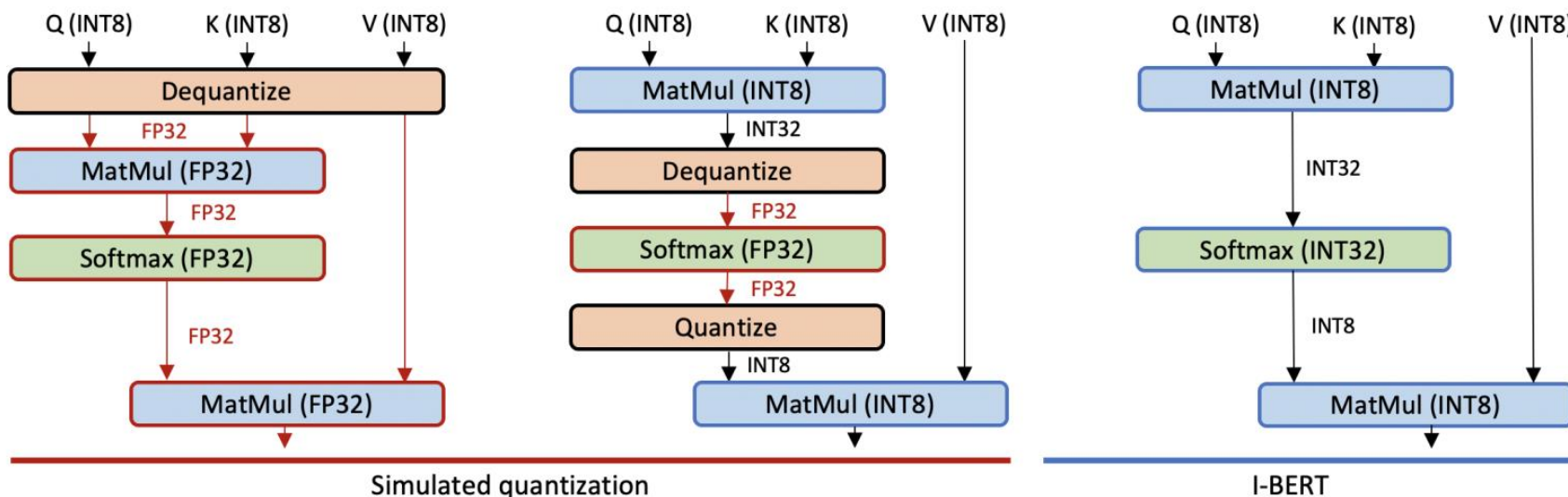
$$w_{float} = w_{float} - \eta \frac{\partial L}{\partial w_{out}} \cdot I_{w_{out} \in (w_{min}, w_{max})}$$

$$w_{out} = SimQuant(w_{float})$$

INT8 Linear Quantization-Aware Training

Neural Network	Floating-Point	Post-Training Quantization		Quantization-Aware Training	
		Asymmetric	Symmetric	Asymmetric	Symmetric
		Per-Tensor	Per-Channel	Per-Tensor	Per-Channel
MobileNetV1	70.9%	0.1%	59.1%	70.0%	70.7%
MobileNetV2	71.9%	0.1%	69.8%	70.9%	71.1%
NASNet-Mobile	74.9%	72.2%	72.1%	73.0%	73.0%

Approximation for Nonlinear OPs



Integer-Only Approximation for non-linear operations of BERT

$$\text{GELU}(x) := x \cdot \frac{1}{2} \left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right],$$

$$\text{where } \text{erf}(x) := \frac{2}{\sqrt{\pi}} \int_0^x \exp(-t^2) dt.$$



$$\text{i-GELU}(x) := x \cdot \frac{1}{2} \left[1 + L\left(\frac{x}{\sqrt{2}}\right) \right]$$

$$L(x) = \text{sgn}(x) [a(\text{clip}(|x|, \text{max} = -b) + b)^2 + 1]$$

$$\text{Softmax}(\mathbf{x})_i = \frac{\exp(x_i - x_{\max})}{\sum_{j=1}^k \exp(x_j - x_{\max})}$$

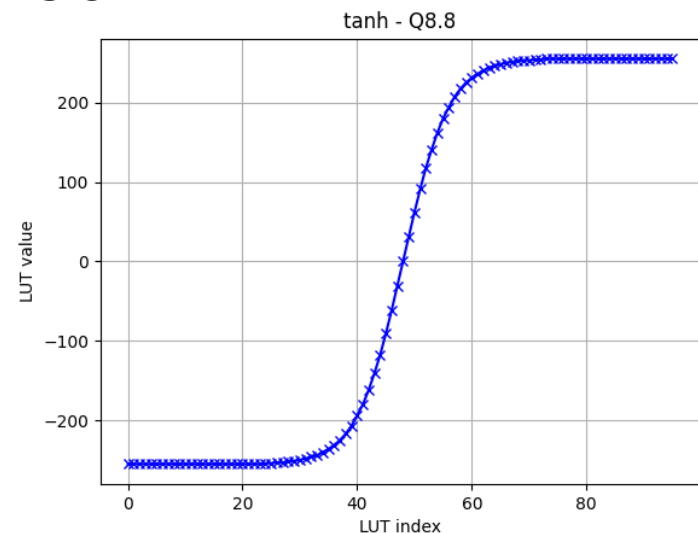
$$\tilde{x}_i = x_i - x_{\max} \quad \tilde{x} = (-\ln 2)z + p$$

$$\exp(\tilde{x}) = 2^{-z} \exp(p) = \exp(p) \gg z$$



$$\text{i-exp}(\tilde{x}) := L(p) \gg z$$

$$L(p) = 0.3585(p + 1.353)^2 + 0.344 \approx \exp(p)$$

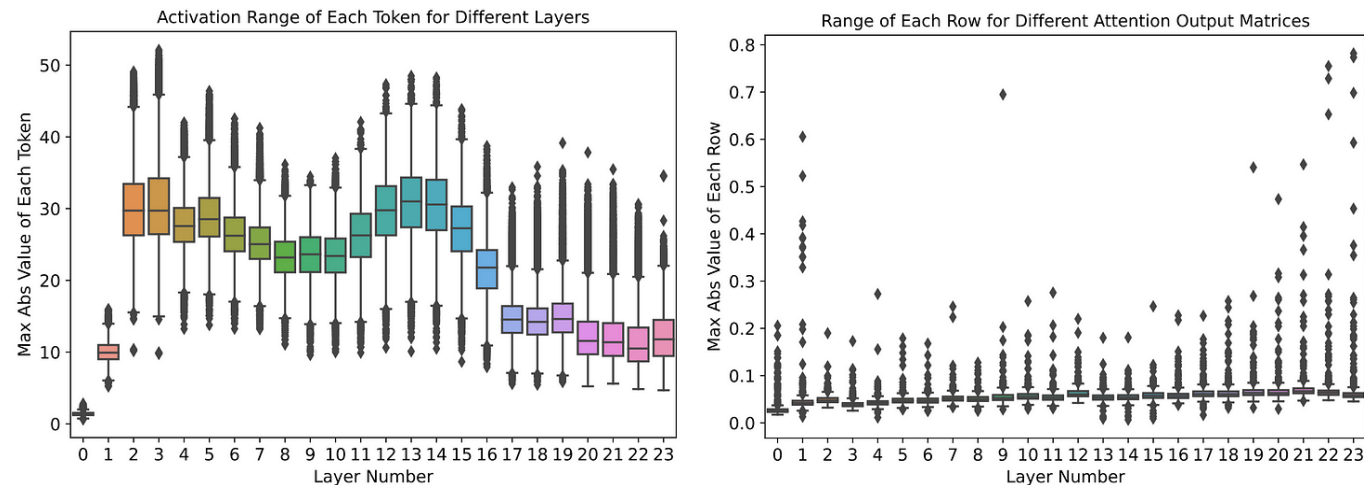


LLM Quantization

- CNN Quantization
 - **FP32** → Int8 → Int4 → 2/1b (XNOR-Net, BinaryNet)
 - QAT is reasonable
 - **Bert** quantization also followed CNN's approaches
- LLM Quantization
 - **FP16** → Int8 → Int4 → 1b (BitNet)
 - PTQ is popular (Llama-2 have the cost of >\$20 million (278억) to train)
 - QLora: Quantization at fine-tuning for downstream task
 - Weight-only, Weight-Activation, KV Cache

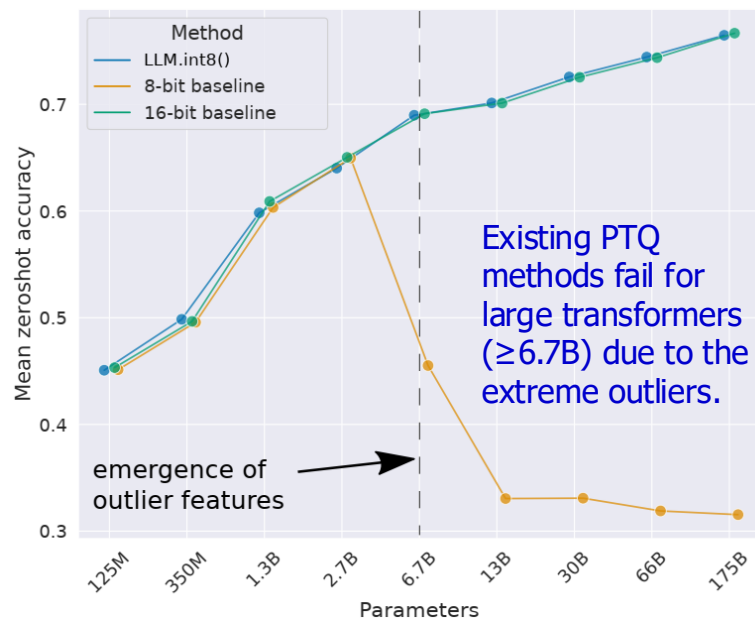
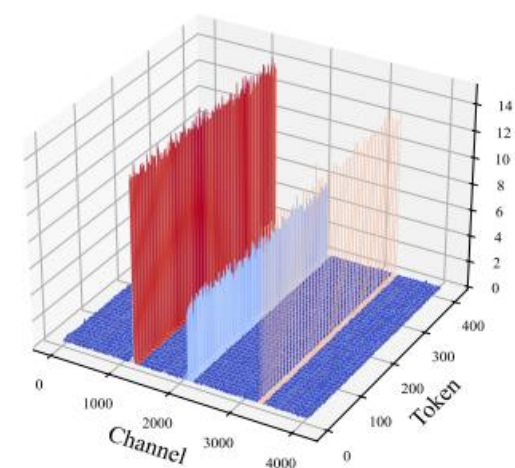
Weight/Activation Variance

- Highly dynamic range in the activations (tokens) between layers
 - A very diverse distribution of activations depending on input token
 - **Token-wise, Dynamic**
 - Calibrate token-wise min/max ranges dynamically during inference
- Weight matrices have long-tailed ranges
 - Each head is independently trained and has significantly different distributions.
 - **Group-wise quantization**

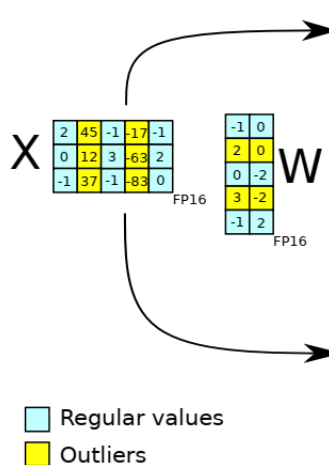


LLM.int8 - Mixed Precision

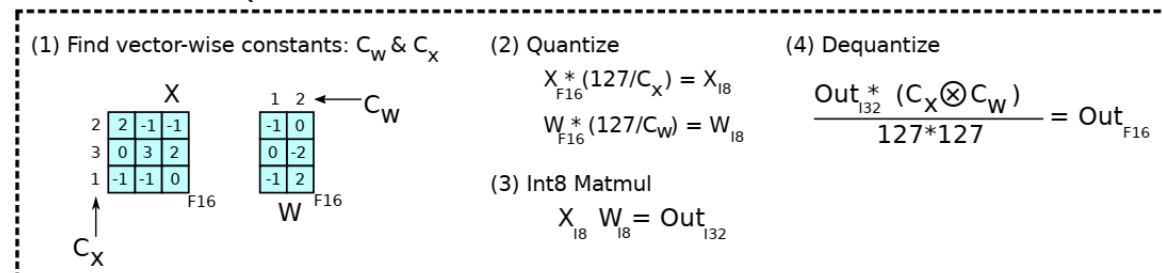
- **W8A8**
- **Vector-wise** quantization granularity.
 - scaling by row and column-wise absolute maximum (C_x, C_w)
- **Separate outlier and normal values into two matrices**
 - **Outlier** feature matrices are multiplied in 16-bit



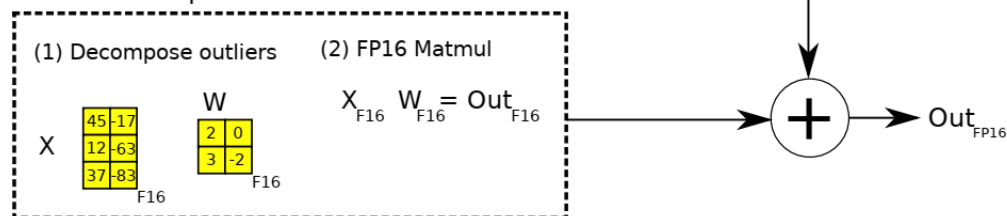
LLM.int8()



8-bit Vector-wise Quantization

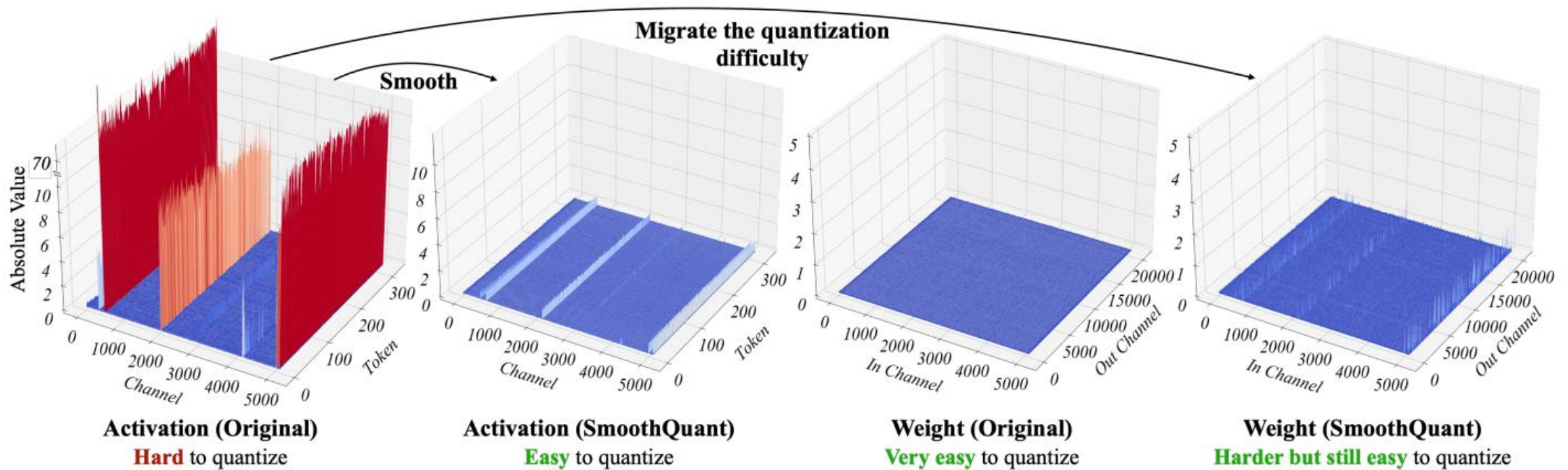


16-bit Decomposition

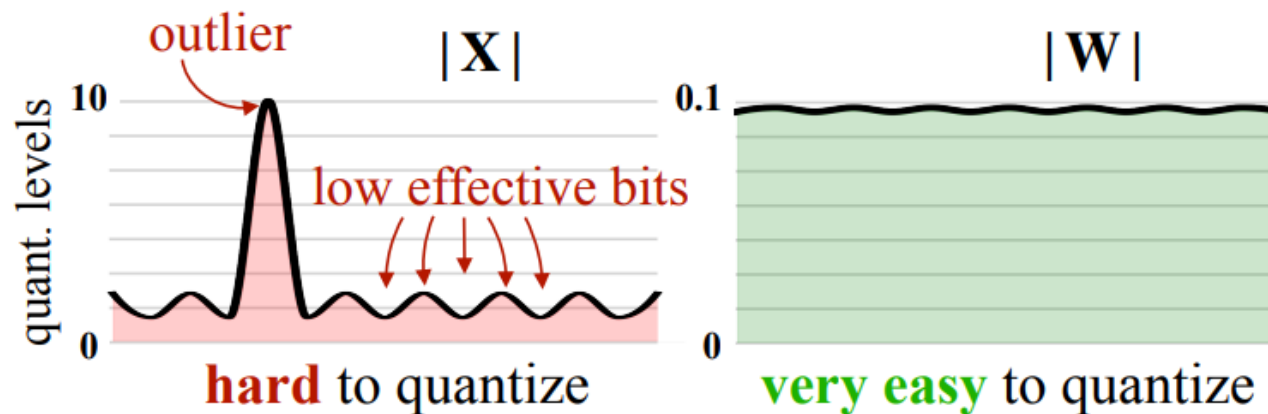


SmoothQuant

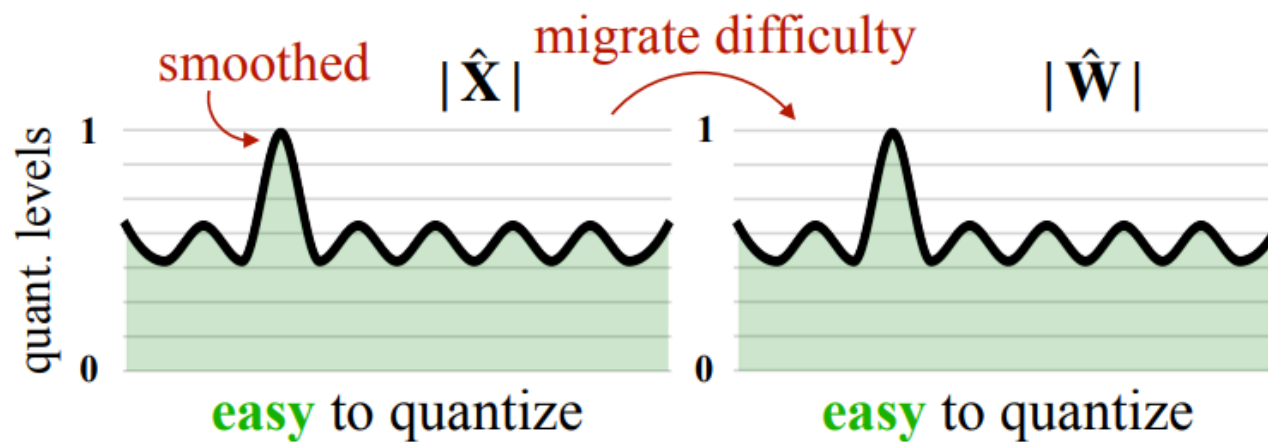
- Outliers persist in fixed channels.
- Transfer appropriate outliers from activation to weight offline through channel-wise smoothing scales



SmoothQuant



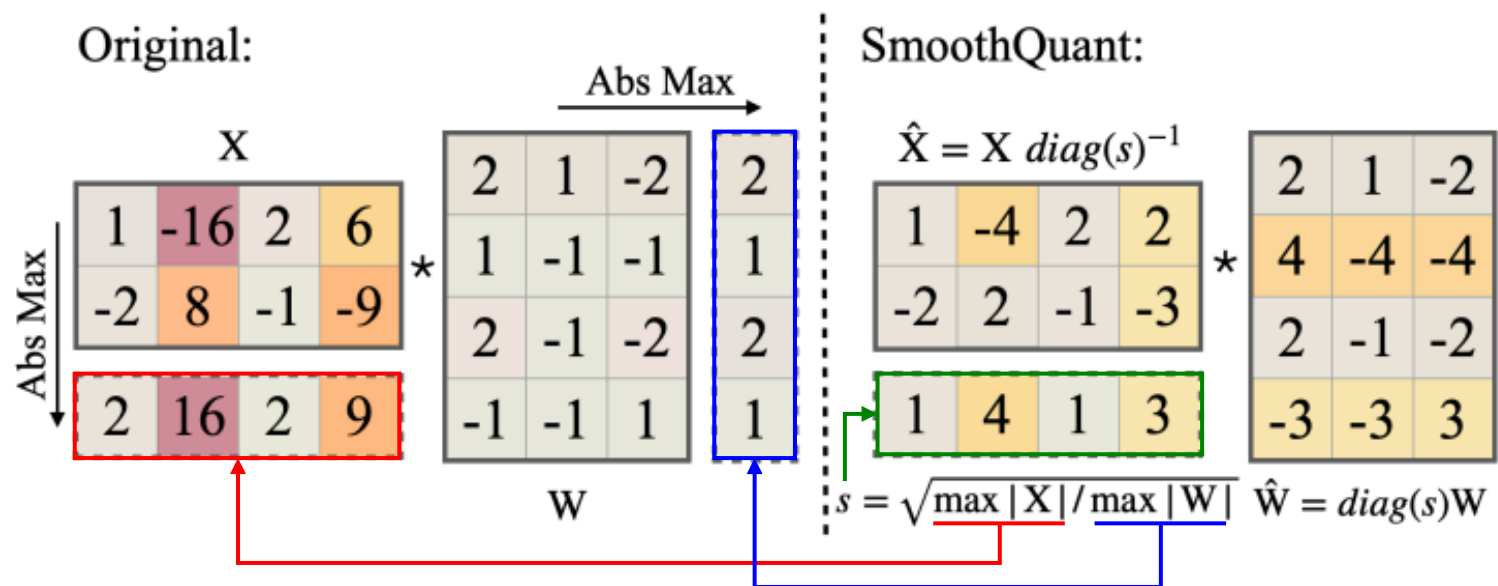
(a) Original



(b) SmoothQuant

SmoothQuant

- Smoothing activation to reduce quantization error



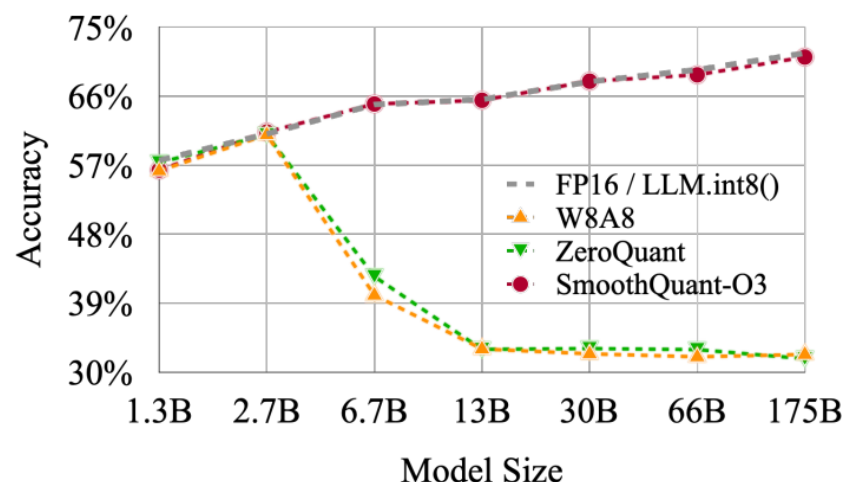
$$Y = XW = (X \text{diag}(s)^{-1}) \cdot (\text{diag}(s)W) = \hat{X}\hat{W}$$

SmoothQuant

- Well maintains the accuracy without fine-tuning.
- Can both accelerate inference and halve the memory footprint.

Method	Weight	Activation
W8A8	per-tensor	per-tensor dynamic
ZeroQuant	group-wise	per-token dynamic
LLM.int8()	per-channel	per-token dynamic+FP16
Outlier Suppression	per-tensor	per-tensor static
SmoothQuant-O1	per-tensor	per-token dynamic
SmoothQuant-O2	per-tensor	per-tensor dynamic
SmoothQuant-O3	per-tensor	per-tensor static

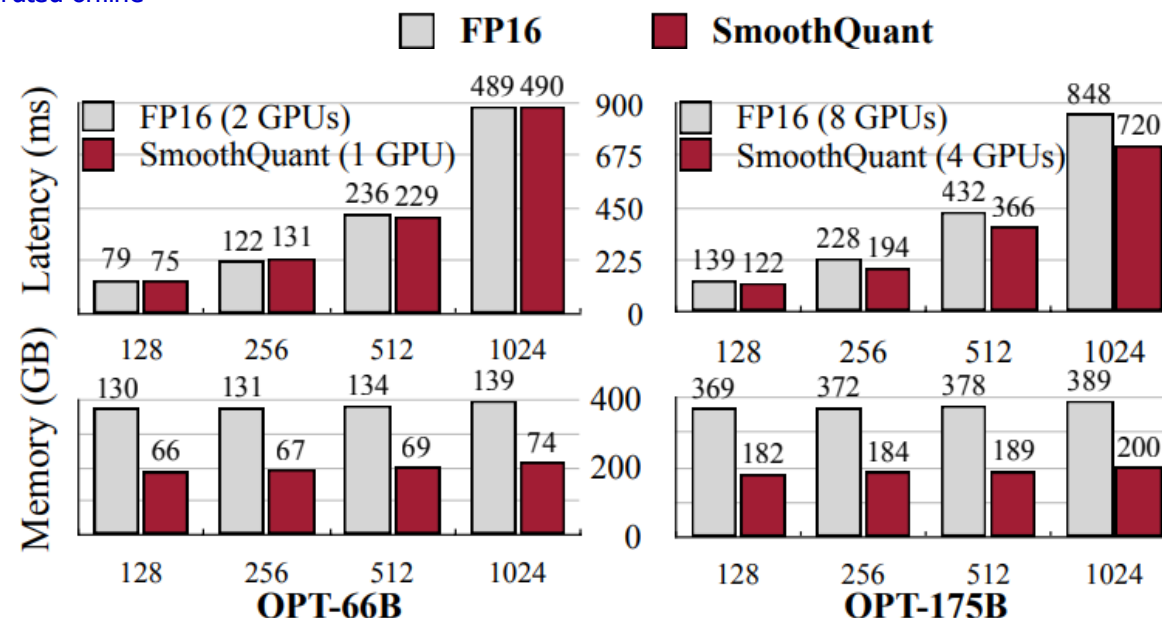
→ The scaling factor is calibrated offline



Equal performance to LLM.int8() without the need for mixed-precision decomposition of the activation.

Wiki PPL↓	7B	13B	30B	65B
FP16	11.51	10.05	7.53	6.17
W8A8 SmoothQuant	11.56	10.08	7.56	6.20

losslessly quantize LLaMA families

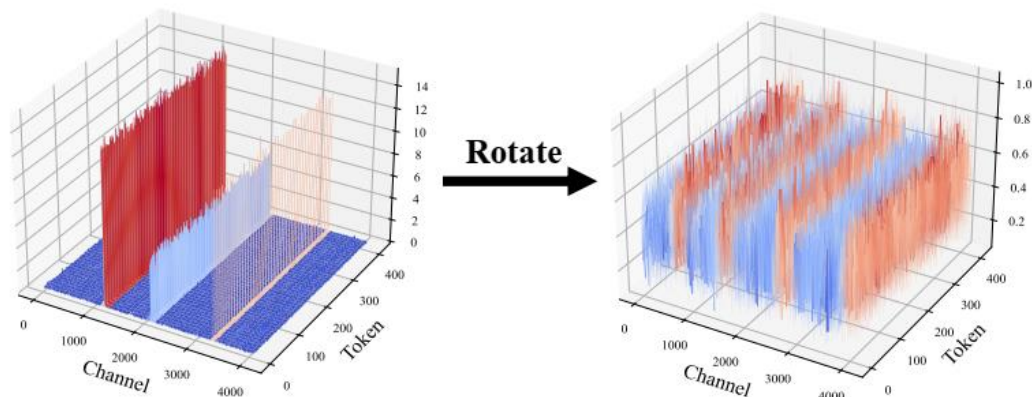


QuaRot

- Make the activations easy to quantize by rotating them using randomized Hadamard transformation (remove outlier features)
- Enable 4-bit LLM inference by quantizing all weights, activations, and KV caches (cf. SmoothQuant cannot implement A4W4)

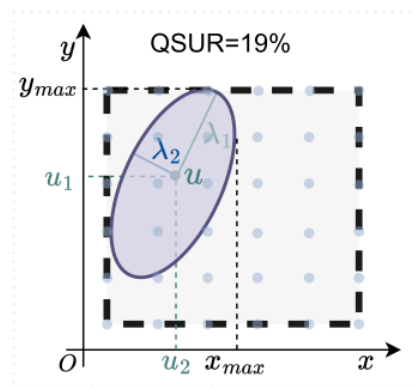
$$\mathbf{Y} = (\mathbf{X}\mathbf{R})(\mathbf{R}^{-1}\mathbf{W}^{-1}) = \mathbf{X}\mathbf{W}^T$$

A rotation matrix is an orthogonal matrix $\mathbf{R}$ satisfied $\mathbf{R}\mathbf{R}^T = \mathbf{I}$ and $|\mathbf{R}| = 1$

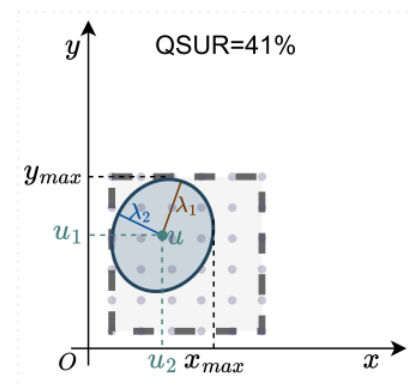


[QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs \[NeurIPS'24\]](#)

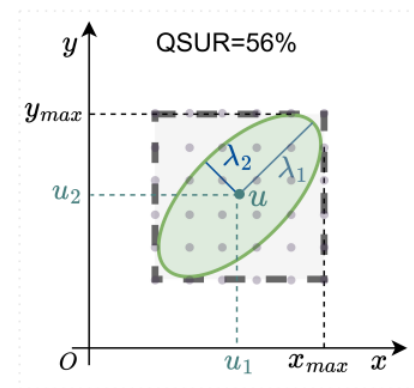
[Rotated Runtime Smooth: Training-Free Activation Smoother for accurate INT4 inference \[ICLR'25\]](#)



(a) Original



(b) Smooth Transform

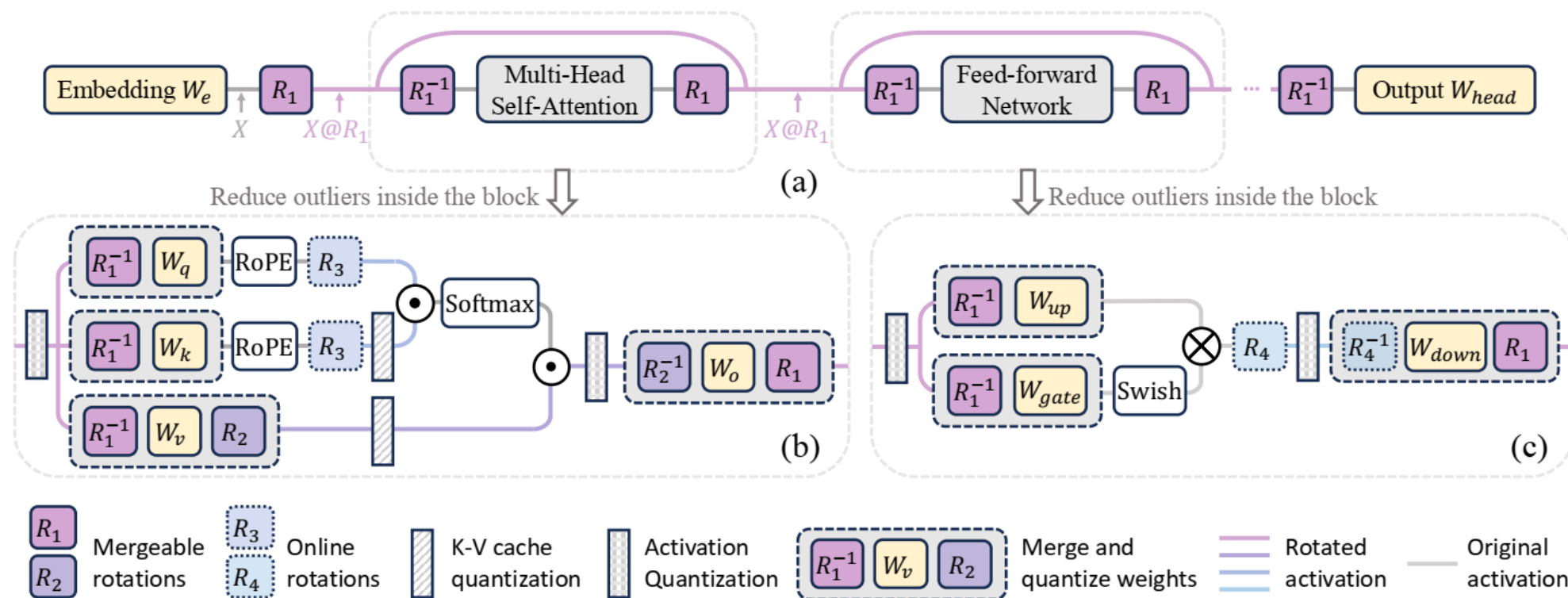


(c) Rotation Transform

[OstQuant: Refining Large Language Model Quantization with Orthogonal and Scaling Transformations for Better Distribution Fitting \[ICLR'25\]](#)

SpinQuant

- Optimize the rotation matrix to minimize the final loss of the quantized network
- Cayley SGD for optimizing orthonormal matrices.



Weight-Only Quantization

- **W8A8** quantization is still memory-bounded.
- **W4A16** low-bit weight-only quantization
 - No efficiency gains in the multiplication operations, due to the **mixed-precision interactions** with the non-quantized activations.
 - **Reduced memory traffic** can speed up inference

GPTQ

- Based on OBQ (Optimal Brain Quantization)
- **Layer-Wise** Quantization
 - minimizes **local layer-output reconstruction**.
 - The curvature of the reconstruction loss can be estimated from the **activation covariance**
- Quantize rows of W independently
- Quantize column-by-column (**fixed order**)
- Perform the update of $\mathbf{H}_F^{-1}$ only d_{col} times (once per column)
- $\mathbf{H}_F^{-1} = \mathbf{U}^T \mathbf{U}$ (Cholesky decomposition)
- Lazy Batch-Updates
 - Apply the update to columns in the block

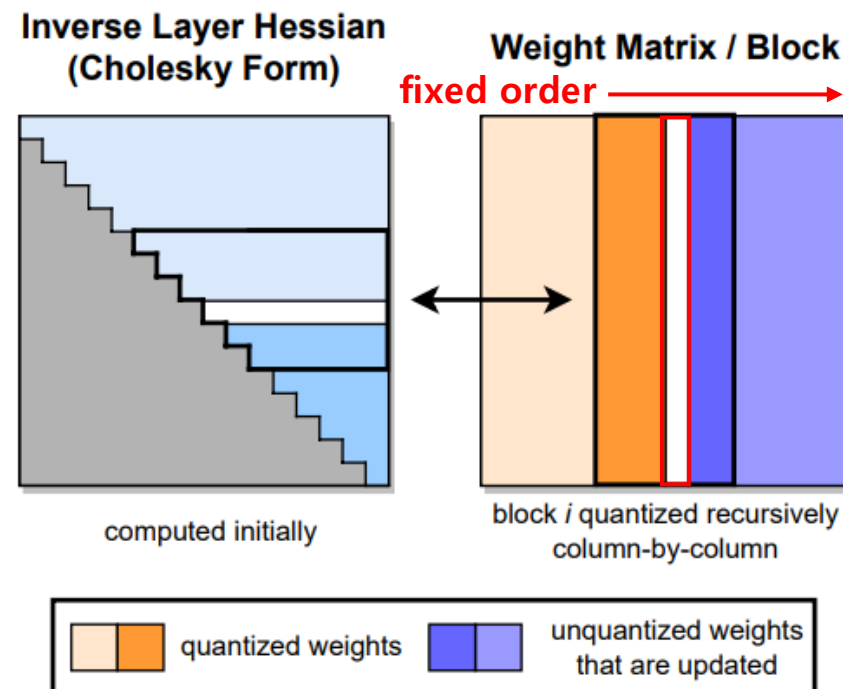
$$L_{\text{rec}} = \|WX - \widehat{W}X\|_F^2 = \|\delta W X\|_F^2,$$

$$L_{\text{rec,row}} = \|\delta w X\|_2^2 = \delta w X X^T \delta w^T$$

$$\Delta L \approx \frac{1}{2} \delta w H \delta w^T, H_{\text{row}} \propto X X^T,$$

$$H_1 = H_2 = H_3 = \dots = X X^T$$

(row에 무관하게 동일 H)



GPTQ

- Uniform quantizer: nearest multiple of 0.5
- GPTQ quantizes columns sequentially; after quantizing column q , it compensates the remaining columns using H^{-1} .

Initial Weight W

0.80	-0.24	0.50	0.10
-0.20	0.40	-0.60	0.30
0.70	-0.10	0.20	-0.50

Initial inverse Hessian $H_{U_1}^{-1}$

1.00	-0.30	0.10	0.00
-0.30	1.00	0.15	0.05
0.10	0.15	1.10	0.20
0.00	0.05	0.20	0.90

Schur-complement update:

$$H_{U_{k+1}}^{-1} = B - \frac{hh^T}{d}$$

where $d = (H_{U_k}^{-1})_{1,1}$,

h = first row (w/o first col), B = trailing block.

Stage 1

Quantize column 1.

$$q_1 = \begin{bmatrix} 1.0 \\ 0.0 \\ 0.5 \end{bmatrix}, \quad e_1 = w_1 - q_1 = \begin{bmatrix} -0.20 \\ -0.20 \\ 0.20 \end{bmatrix}$$

$W^{(1)}$ (after Stage 1)

1.00	-0.30	0.52	0.10
0.00	0.34	-0.58	0.30
0.50	-0.04	0.18	-0.50

Update rule (propagate e_1 to cols 2–4):

$$W_{:,j} \leftarrow W_{:,j} - e_1 \frac{(H_{U_1}^{-1})_{1,j}}{(H_{U_1}^{-1})_{1,1}}, \quad j = 2, 3, 4$$

Uses row 1 of $H_{U_1}^{-1}$:

factors = $[-0.30, 0.10, 0.00]$

$H_{U_2}^{-1}$ (after Stage 1)

0.91	0.18	0.05
0.18	1.09	0.20
0.05	0.20	0.90

Stage 2

Quantize column 2.

$$q_2 = \begin{bmatrix} -0.5 \\ 0.5 \\ 0.0 \end{bmatrix}, \quad e_2 = w_2 - q_2 = \begin{bmatrix} 0.20 \\ -0.16 \\ -0.04 \end{bmatrix}$$

$W^{(2)}$ (after Stage 2)

1.00	-0.50	0.48	0.09
0.00	0.50	-0.55	0.31
0.50	0.00	0.19	-0.50

Update rule (propagate e_2 to cols 3–4):

$$W_{:,j} \leftarrow W_{:,j} - e_2 \frac{(H_{U_2}^{-1})_{1,j-2+1}}{(H_{U_2}^{-1})_{1,1}}, \quad j = 3, 4$$

Propagate error to cols 3–4 using $H_{U_2}^{-1}$

(factors = $[0.18, 0.05]$).

$H_{U_3}^{-1}$ (after Stage 2)

1.05	0.19
0.19	0.90

Stage 3

Quantize column 3.

$$q_3 = \begin{bmatrix} 0.5 \\ -0.5 \\ 0.0 \end{bmatrix}, \quad e_3 = w_3 - q_3 = \begin{bmatrix} -0.02 \\ -0.05 \\ 0.19 \end{bmatrix}$$

$W^{(3)}$ (after Stage 3)

1.00	-0.50	0.50	0.09
0.00	0.50	-0.50	0.32
0.50	0.00	0.00	-0.53

Update rule (propagate e_3 to col 4):

$$W_{:,4} \leftarrow W_{:,4} - e_3 \frac{(H_{U_3}^{-1})_{1,2}}{(H_{U_3}^{-1})_{1,1}}$$

Propagate error to col 4

(factor = 0.19).

$H_{U_4}^{-1}$ (after Stage 3)

0.86

Stage 4

Quantize column 4.

$$q_4 = \begin{bmatrix} 0.0 \\ 0.5 \\ -0.5 \end{bmatrix}$$

$W^{(4)} = \hat{W}$

1.00	-0.50	0.50	0.00
0.00	0.50	-0.50	0.50
0.50	0.00	0.00	-0.50

Final Quantized Weight $\hat{W}$

★ Naïve RTN would give row1,col2 = 0.0 , but GPTQ compensation changes it to -0.5 .

Cholesky factorization eliminates the need for explicit Schur-complement updates of H^{-1} .

AWQ

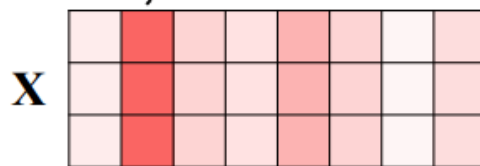
- Observation: Weights are not equally important
 - Keeping **only 1% of salient weight channels** in FP16 can greatly improve perplexity (43.2 $\rightarrow$ 13.0)
 - Salient weights are determined by **activation** distribution, not weight.
 - But the mixed-precision format is not hardware-efficient.

W_{FP16}			
+1.2	-0.2	-2.4	-3.4
-2.5	-3.5	+1.9	+1.4
-0.9	+1.6	-2.5	-1.9
-3.5	+1.5	+0.5	-0.1
+1.8	-1.6	-3.2	-3.4
+2.4	-3.5	-2.8	-3.9
+0.1	-3.8	+2.4	+3.4
+0.9	+3.3	-1.9	-2.3

Q

$Q(W)_{INT3}$			
+1	+0	-2	-3
-3	-4	+2	+1
-1	+2	-3	-2
-4	+2	+1	+0
+2	-2	-3	-3
+2	-4	-3	-4
+0	-4	+2	+3
+1	+3	-2	-2

determine the salient weights by activation



bad hardware efficiency $\rightarrow$

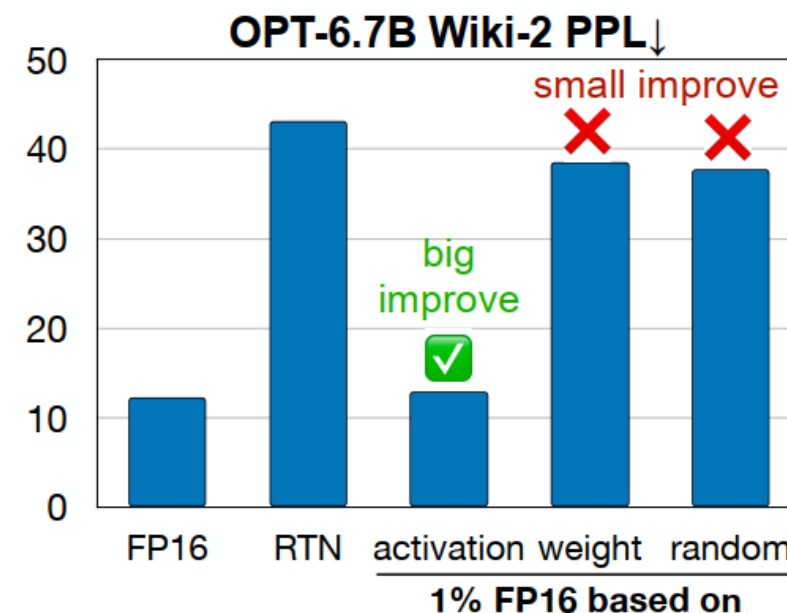
$Q(W)_{MixPrec}$			
+1	+0	-2	-3
-2.5	-3.5	+1.9	+1.4
-1	+2	-3	-2
-4	+2	+1	+0
+2	-2	-3	-3
+2	-4	-3	-4
+0	-4	+2	+3
+1	+3	-2	-2

FP16 channel

*

(a) RTN quantization (**PPL 43.2**)

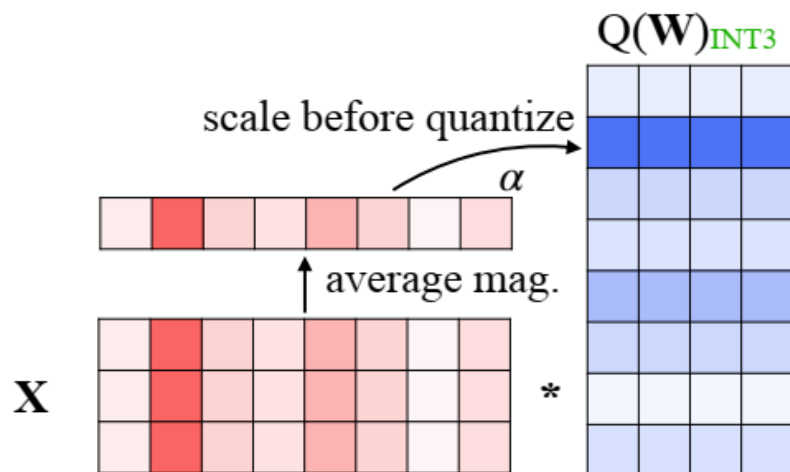
(b) Keep 1% salient weights in FP16 (**PPL 13.0**)



AWQ

- Protecting salient weights by per-channel scaling
 - Multiplying the salient channels with $s > 1$ reduces its quantization error

$$\mathbf{WX} \rightarrow Q(\mathbf{W} \cdot \mathbf{s})(\mathbf{s}^{-1} \cdot \mathbf{X})$$

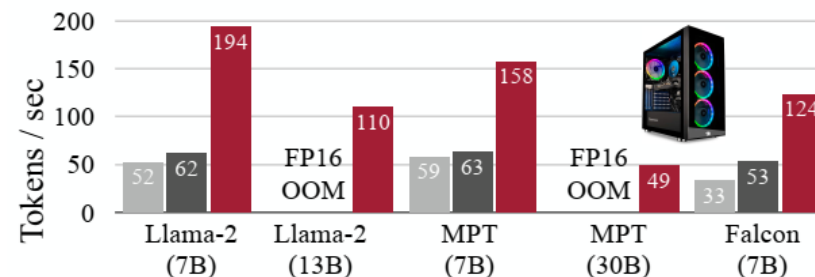


(c) Scale the weights before quantization (PPL 13.0)

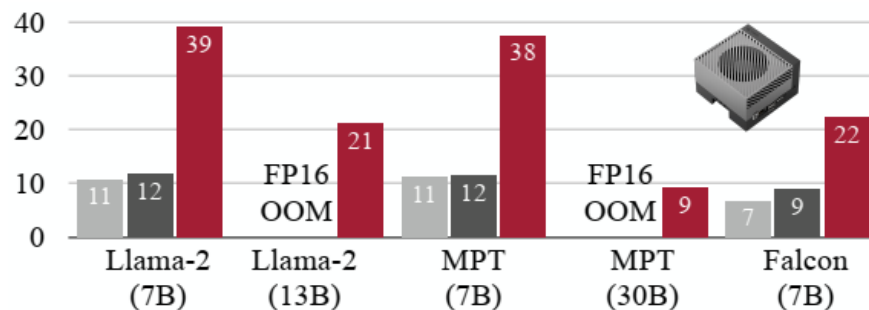
$$s = s_X^\alpha = \left(\frac{1}{N} \sum |X| \right)^\alpha \quad 0 \leq \alpha \leq 1$$

Find per-layer optimal α
Channel group-wise Quantization

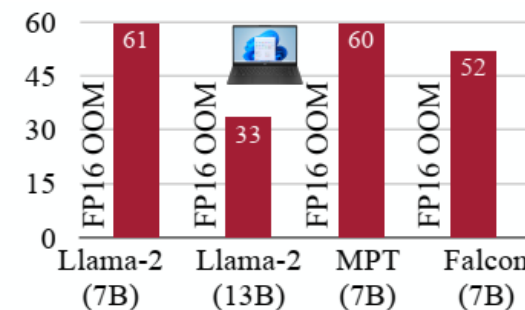
■ Huggingface (FP16) ■ Ours (FP16) ■ Ours (AWQ, W4A16)



(a) RTX 4090 desktop GPU



(b) Jetson Orin mobile GPU



(c) RTX 4070 laptop GPU

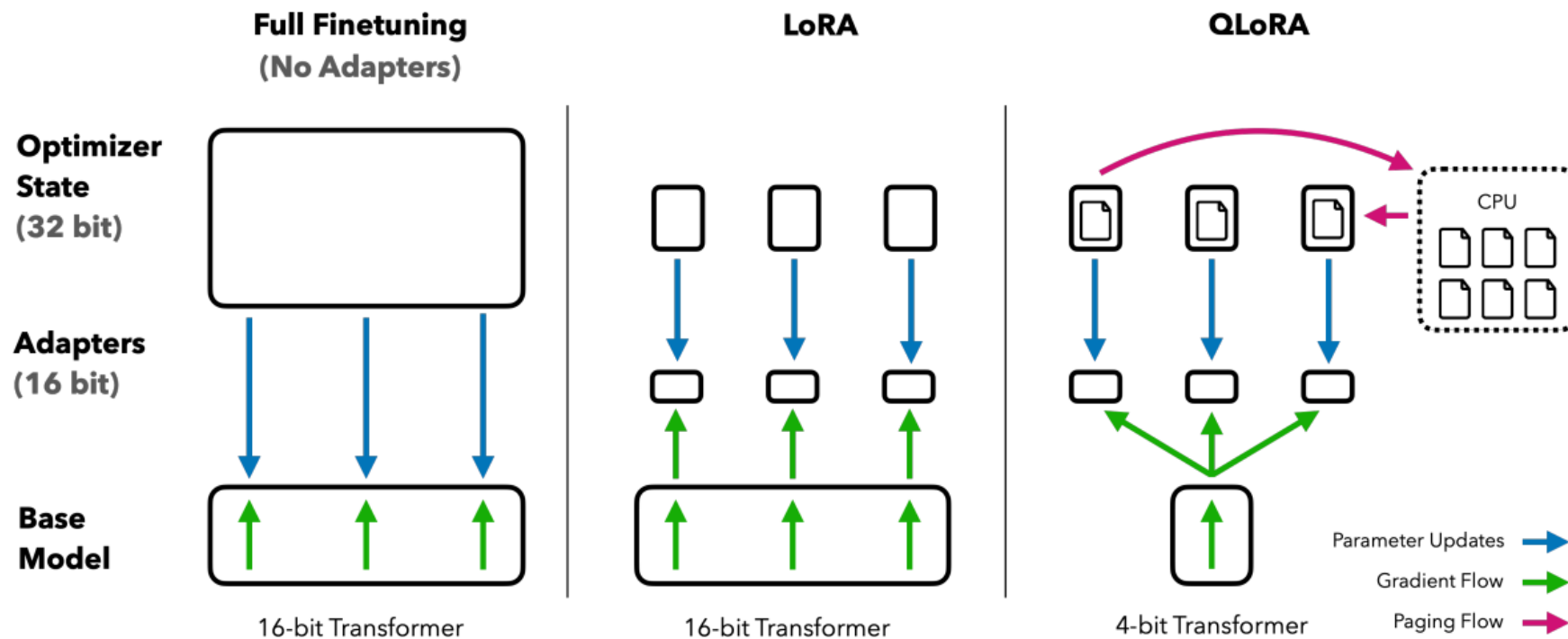
AWQ

- Better perplexity than GPTQ (w/ and w/o reordering) on LLaMA & Llama-2 models.
 - Reconstruction process of GPTQ leads to an over-fitting issue to the calibration set and may not preserve the generalist abilities of LLMs.

PPL↓		Llama-2			LLaMA			
		7B	13B	70B	7B	13B	30B	65B
FP16	-	5.47	4.88	3.32	5.68	5.09	4.10	3.53
INT3 g128	RTN	6.66	5.52	3.98	7.01	5.88	4.88	4.24
	GPTQ	6.43	5.48	3.88	8.81	5.66	4.88	4.17
	GPTQ-R	6.42	5.41	3.86	6.53	5.64	4.74	4.21
	AWQ	6.24	5.32	3.74	6.35	5.52	4.61	3.95
INT4 g128	RTN	5.73	4.98	3.46	5.96	5.25	4.23	3.67
	GPTQ	5.69	4.98	3.42	6.22	5.23	4.24	3.66
	GPTQ-R	5.63	4.99	3.43	5.83	5.20	4.22	3.66
	AWQ	5.60	4.97	3.41	5.78	5.19	4.21	3.62

QLoRA = LoRA+Quant

- LoRA with **quantized base model weights**
- **NormalFloat (NF4)**
- Double quantization (apply quantization on scaling factors) to further reduce base model size
- Paged Optimizers with CPU offloading



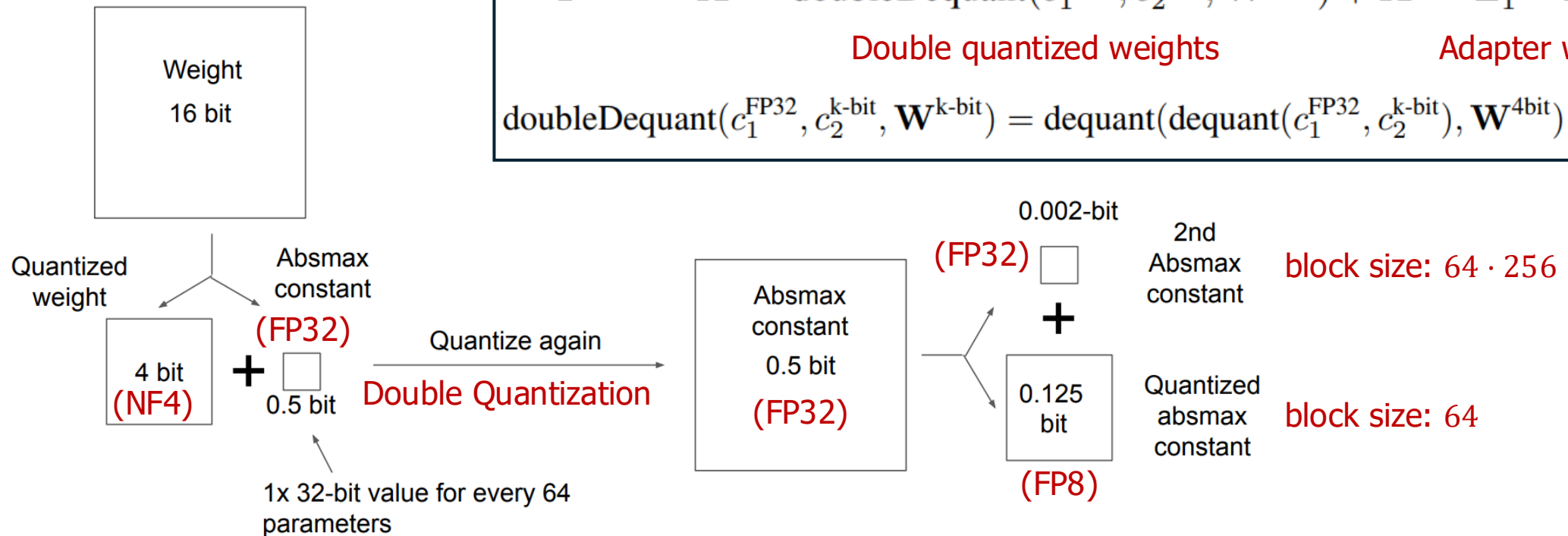
Double Quantization

- Quantize the quantization constants
 - Bits per parameter: $\frac{32}{64} = 0.5$ bits $\rightarrow \frac{8}{64} + \frac{32}{64 \cdot 256} = 0.127$ bits
- Reduce the memory footprint of quantization constants.
- Compute in BF16

$$\mathbf{Y}^{\text{BF16}} = \mathbf{X}^{\text{BF16}} \text{doubleDequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}, \mathbf{W}^{\text{NF4}}) + \mathbf{X}^{\text{BF16}} \mathbf{L}_1^{\text{BF16}} \mathbf{L}_2^{\text{BF16}}$$

Double quantized weights Adapter weights

$$\text{doubleDequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}, \mathbf{W}^{\text{k-bit}}) = \text{dequant}(\text{dequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}), \mathbf{W}^{\text{4bit}}) = \mathbf{W}^{\text{BF16}}$$



Llama.cpp Quantization

계열	형식	기법
Legacy block quant	Q4_0, Q5_0, Q8_0 (symmetric, only scale) Q4_1, Q5_1 (asymmetric, scale+min)	32-weight block 마다 scale/min (FP16)
K-quant	Q2_K_S, Q2_K, Q3_K_S, Q3_K_M, Q3_K_L Q4_K_S, Q4_K_M, Q5_K_S, Q5_K_M, Q6_K	Mixed Precision Double Quant (6bit scale/min) (superblock = 8 blocks = 256 weights)
IQ / I-quant	IQ1_S, IQ1_M IQ2_XXS, IQ2_XS, IQ2_S, IQ2_M IQ3_XXS, IQ3_XS, IQ3_S, IQ3_M IQ4_XS, IQ4_NL (non-linear)	importance-aware (GPTQ) imatrix $I_i \approx E[x_i^2]$ Structured codebook/VQ (XXS < XS < S < M)
[?] Ternary $w \in \{-1, 0, +1\}$	TQ1_0 (base-3 packing, 메모리↓, 복잡) TQ2_0 (2-bit scalar packing)	$[-1, +1, 0, -1, +1] \rightarrow [0, 2, 1, 0, 2]$ $0 \times 3^4 + 2 \times 3^3 + 1 \times 3^2 + 0 \times 3^1 + 2 \times 3^0 = 65$ (8b으로 표현)
FP4	MXFP4, NVFP4	FP4 + shared scale
ultra-low-bit	Q1_0, Q2_0	단순 1/2-bit block quant